

**Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Ecole Nationale d'Ingénieurs de Carthage**



Rapport de Projet de Fin d'Etudes

Spécialité : Génie Informatique

Élaboré par : MEDDEB Rayen

**Conception et Développement d'un Système de
Gestion du Transport (LUMTMS 0.1)**

Réalisé au sein de : Lumiere Logistique



Encadré par :

Encadrant académique : Mme Samia Saidi

Encadrant industriel : M. Hakim Ben Chikh

Année universitaire : 2024–2025

Dédicace

À ma famille, à mes amis, et à tous ceux qui ont éclairé mon chemin.

À mes parents, Kais et Hannene,

Pour votre amour inconditionnel, votre confiance sans faille et les valeurs de persévérance et de rigueur que vous m'avez transmises. Votre soutien constant a été le socle de ma réussite.

À ma fiancée, Farah,

Mon soutien infaillible et ma plus grande source de bonheur. Ta patience, ton amour et tes encouragements ont été mon ancre dans les moments de doute et ma lumière au quotidien. Ce travail est aussi le tien.

À mes frères, Fedi et Wala Eddine,

Pour votre affection fraternelle, votre patience et les moments de complicité qui ont rendu cette aventure plus légère.

À mes amis, Ghazi, Khalil et Hakim,

Pour votre amitié sincère, votre écoute et votre présence indéfectible. Votre joie de vivre a été une bouffée d'air frais dans les moments les plus intenses.

À mes enseignants,

Pour avoir éveillé ma curiosité, nourri mon esprit critique et partagé votre savoir avec passion et bienveillance.

Ce travail est le fruit de ces liens précieux.
Je vous le dédie en témoignage de ma profonde gratitude.

Remerciements

Au terme de ce projet de fin d'études, qui marque l'aboutissement de mon parcours d'ingénieur, je souhaite adresser mes plus sincères remerciements à ceux qui m'ont accompagné et soutenu.

Ma reconnaissance va d'abord à **l'École Nationale d'Ingénieurs de Carthage (ENICarthage)**, pour la qualité de son enseignement et pour m'avoir fourni les outils intellectuels et humains nécessaires à ma réussite.

Je remercie très chaleureusement mon encadrante académique, **Madame Samia Saïdi**. Sa disponibilité, la pertinence de ses conseils et son suivi rigoureux ont été des atouts précieux pour la bonne conduite de ce travail. Son exigence bienveillante m'a poussé à donner le meilleur de moi-même.

J'exprime également ma profonde gratitude à mon encadrant professionnel, **Monsieur Hakim Be-chikh**, pour la confiance qu'il m'a témoignée en me confiant ce projet au sein de *Lumière Logistique*. Ses orientations claires et son expertise métier ont été un guide essentiel.

Enfin, je souhaite remercier l'ensemble des collaborateurs de *Lumière Logistique* pour leur accueil, leur esprit d'équipe et leur disponibilité. Leur soutien a grandement facilité mon intégration et la réalisation de ce système.

À toutes ces personnes, j'exprime ma plus vive reconnaissance.

Table des matières

Table des matières

Dédicace	i
Remerciements	ii
Table des matières	iii
Liste des figures	x
Liste des tableaux	xiii
Liste des abréviations	xv
Introduction Général	1
1 Problèmes du Transport et de la Livraison – État des Lieux et Évolution	2
1.1 Introduction	2
1.2 Situation Actuelle du Transport et de la Logistique	2
1.3 Limites de la Gestion Traditionnelle du Transport	4
1.4 Les Tendances Actuelles des TMS	5
1.5 Conclusion	7
2 Contexte du Projet	8
2.1 Introduction	8
2.2 Présentation de l'organisme d'accueil	8
2.2.1 Présentation de l'entreprise	8
2.2.2 Organigramme de l'entreprise	9

2.3	Présentation du projet	10
2.3.1	Problématique	10
2.4	La Solution Propriétaire Utilisée par Lumière Logistique : Akenea TMS	11
2.5	Critique de la Solution Utilisée à Lumière Logistique (Akenea TMS)	12
2.6	Solution proposée : Lumière Transport Management System (LUMTMS)	12
2.7	Méthode de travail et planification	14
2.7.1	Approches de gestion de projet	14
2.7.2	Choix de la méthode : SCRUM	15
2.7.3	Modélisation fonctionnelle et technique avec UML	16
2.8	Conclusion	16
3	Sprint 0 – Étude Préliminaire	17
3.1	Introduction	17
3.2	Analyse des besoins	17
3.2.1	Identification des acteurs	17
3.2.2	Définition des besoins fonctionnels	18
3.2.2.1	Gestion des Comptes et de l'Authentification	18
3.2.2.2	Gestion des Ordres de Transport (OT)	18
3.2.2.3	Gestion des Voyages	18
3.2.2.4	Gestion des Ressources Matérielles	19
3.2.2.5	Gestion des Chauffeurs	19
3.2.2.6	Gestion des Clients et Articles	19
3.2.2.7	Outils de Consultation et d'Analyse	19
3.2.2.8	Interaction intelligente via Chatbot	20
3.2.2.9	Déploiement, Automatisation et DevOps	20
3.2.3	Définition des besoins non-fonctionnels	20
3.3	Diagramme de cas d'utilisation globale	20
3.4	Product Backlog	21
3.5	Planification des itérations – Répartition en sprints	23
3.6	Conception architecturale	24
3.6.1	Conception et implémentation de l'architecture microservices	25
3.6.2	Architecture logique	26
3.6.3	Architecture physique	27
3.7	Choix et exigences technologiques	28
3.7.1	Environnement matériel	29
3.7.2	Environnement logiciel	29
3.7.2.1	Logiciels de développement	29
3.7.2.2	Outils de développement	30
3.7.2.3	Langages de programmation	30

3.7.2.4	Outils de déploiement et d'hébergement	31
3.8	Conclusion	31
4	Sprint 1 – Gestion des Utilisateurs et Profils, Authentification	32
4.1	Introduction	32
4.2	Objectifs du Sprint	32
4.3	Spécification et Analyse des Besoins	33
4.3.1	Sprint Backlog	33
4.3.2	Diagramme de cas d'utilisation du Sprint	34
4.4	Description textuelle des cas d'utilisation	35
4.4.1	Description textuelle du cas d'utilisation "S'authentifier"	35
4.4.2	Description textuelle du cas d'utilisation "Modifier profil"	36
4.4.3	Description textuelle du cas d'utilisation "Ajouter un compte utilisateur" . . .	37
4.4.4	Gestion des autres actions liées aux comptes utilisateurs	38
4.5	Conception	39
4.5.1	Diagramme de classes	39
4.5.2	Diagrammes de séquence	39
4.5.2.1	Diagramme de séquence "S'authentifier"	39
4.5.2.2	Diagramme de séquence " modifier profil "	40
4.5.2.3	Diagramme de séquence "Ajouter un compte utilisateur"	41
4.6	Réalisation	42
4.6.1	Interface d'authentification (login)	42
4.6.2	Interface de gestion des utilisateurs	43
4.6.3	Interface du profil utilisateur	44
4.7	Conclusion	44
5	Sprint 2 – Gestion des Articles et des Clients	45
5.1	Introduction	45
5.2	Objectifs du Sprint	45
5.3	Spécification et Analyse des Besoins	46
5.3.1	Sprint Backlog	46
5.4	Diagramme de cas d'utilisation du Sprint	47
5.5	Description textuelle des cas d'utilisation	49
5.5.1	Cas d'utilisation "Ajouter un article"	49
5.5.2	Autres cas d'utilisation de gestion des articles	49
5.5.3	Cas d'utilisation "Ajouter un client"	50
5.5.4	Autres cas d'utilisation de gestion des clients	50
5.6	Conception	51
5.6.1	Diagramme de classes	51
5.6.2	Diagrammes de séquence	51

5.6.2.1	Diagramme de séquence "Ajouter un article"	51
5.6.2.2	Diagramme de séquence de "ajouter un client"	52
5.7	Réalisation	53
5.7.1	Interface de gestion des articles	53
5.7.2	Interface de gestion des clients	55
5.8	Conclusion	57
6	Sprint 3 – Gestion des Chauffeurs et des Camions	58
6.1	Introduction	58
6.2	Objectifs du Sprint	58
6.3	Spécification et Analyse des Besoins	59
6.3.1	Sprint Backlog	59
6.3.2	Diagramme de cas d'utilisation du Sprint	60
6.4	Description textuelle des cas d'utilisation	61
6.4.1	Description textuelle du cas d'utilisation "Ajouter un camion"	61
6.4.2	Description textuelle du cas d'utilisation "Consulter la localisation d'un camion"	62
6.4.3	Autres cas d'utilisation de gestion des camions	64
6.4.4	Description textuelle du cas d'utilisation "Ajouter un chauffeur"	64
6.4.5	Description textuelle du cas d'utilisation "Modifier l'état d'un chauffeur (mettre en repos)"	65
6.4.6	Autres cas d'utilisation de gestion des chauffeurs	66
6.5	Conception	66
6.5.1	Diagramme de classes	66
6.5.2	Diagrammes de séquence	67
6.5.2.1	Diagramme de séquence "Ajouter un camion"	67
6.5.2.2	Diagramme de séquence "Consulter la localisation d'un camion"	68
6.5.2.3	Diagramme de séquence "Ajouter un chauffeur"	69
6.5.2.4	Diagramme de séquence "Modifier l'état d'un chauffeur (mettre en repos)"	70
6.6	Réalisation	71
6.6.1	Interface de gestion des camions	71
6.6.2	Interface de gestion des chauffeurs	74
6.7	Conclusion	76
7	Sprint 4 : Gestion des Ordres de Transport (OT)	77
7.1	Introduction	77
7.2	Objectifs du Sprint	77
7.3	Spécification et Analyse des Besoins	78
7.3.1	Sprint Backlog	78
7.3.2	Diagramme de cas d'utilisation du Sprint	78

7.4	Description textuelle des cas d'utilisation	79
7.4.1	Description textuelle du cas d'utilisation "Importer des Ordres de Transport (OT) depuis Geode"	79
7.4.2	Description textuelle du cas d'utilisation "Exporter des Ordres de Transport (OT) vers Geode"	80
7.4.3	Description textuelle du cas d'utilisation "Créer un Ordre de Transport (OT)"	81
7.4.4	Description textuelle du cas d'utilisation "Dupliquer un Ordre de Transport (OT)"	82
7.4.5	Autres cas d'utilisation de gestion des OT	83
7.5	Conception	83
7.5.1	Diagramme de classe de sprint	83
7.5.2	Diagrammes de séquence	85
7.5.2.1	Diagramme de séquence "Importer des OTs"	85
7.5.3	Diagramme de séquence de "Exporter des OTs"	86
7.5.4	Diagramme de séquence "Créer un OT"	87
7.5.5	Diagramme de séquence "dupliquer un OT"	88
7.6	Réalisation	88
7.6.1	Interface de gestion des OT	89
7.6.2	Interface de création d'un OT	89
7.6.3	Structure d'un fichier PLA	90
7.7	Conclusion	91
8	Sprint 5 – Gestion des voyages et des tournées génériques	92
8.1	Introduction	92
8.2	Objectifs du sprint	92
8.3	Analyse et spécification	93
8.3.1	Sprint Backlog	93
8.3.2	Diagramme de cas d'utilisation de sprint 5	94
8.4	Description textuelle des cas d'utilisation	95
8.4.1	Description textuelle du cas d'utilisation "Créer manuellement un voyage"	96
8.4.2	Description textuelle du cas d'utilisation "Déplanifier un voyage"	96
8.4.3	Description textuelle du cas d'utilisation "Créer une tournée générique"	97
8.4.4	Description textuelle du cas d'utilisation "Création automatique d'un voyage"	98
8.4.5	Autres cas d'utilisation	99
8.5	Conception	99
8.5.1	Diagramme de classe de sprint	99
8.5.2	Diagrammes de séquence	100
8.5.2.1	Diagramme de séquence "Créer un voyage"	100
8.5.2.2	Diagramme de séquence "Déplanifier un voyage"	101

8.5.2.3	Diagramme de séquence “Créer une tournée générique”	102
8.5.2.4	Diagramme de séquence “Création automatique d’un voyage à partir d’une tournée générique”	103
8.6	Réalisation	104
8.6.1	Interfaces de planification des voyages	104
8.6.2	Interface de gestion des tournées génériques	106
8.7	Conclusion	107
9	Sprint 6 : Chatbot et Outils de consultation et d’analyse	108
9.1	Introduction	108
9.2	Objectif du sprint	108
9.3	Analyse et spécification	108
9.3.1	Diagramme des cas d’utilisation	108
9.3.2	Backlog du sprint	109
9.3.3	Description textuelle des cas d’utilisation	110
9.3.3.1	Description textuelle du cas d’utilisation ”Consulter le planning journalier”	110
9.3.3.2	Description textuelle du cas d’utilisation ”Consulter le tableau de bord”	111
9.3.3.3	Description textuelle du cas d’utilisation ”Interagir avec le chatbot”	112
9.3.4	Principe de fonctionnement du script Python du chatbot intelligent	112
9.3.4.1	Flux de traitement d’une question	112
9.3.4.2	Composants technologiques clés	113
9.4	Conception	113
9.4.1	Diagrammes de séquence	113
9.4.1.1	Diagramme de séquence - Consulter le planning journalier	113
9.4.1.2	Diagramme de séquence - Consulter le tableau de bord analytique (KPIs)	114
9.4.1.3	Diagramme de séquence - Interagir avec le chatbot intelligent	115
9.5	Réalisation	115
9.5.1	Tableau de Bord et Indicateurs de Performance (KPIs)	115
9.5.2	Interface de Consultation du Planning	117
9.5.3	Interface du Chatbot Intelligent	117
9.6	Conclusion	120
10	Sprint 7 – Déploiement et Automatisation du Système	121
10.1	Introduction	121
10.2	Objectif du sprint	121
10.3	Backlog du sprint	121
10.4	Dockerisation du backend	123

10.4.1	Cycle de Vie d'une Application avec Docker	123
10.4.2	Configuration des Dockerfiles	124
10.4.2.1	Stratégie de Dockerisation pour Spring Boot	124
10.4.2.2	Exemple de Dockerfile	124
10.4.3	Dockerisation des Services Auxiliaires (MySQL)	125
10.5	Déploiement via Docker Compose	126
10.6	Hébergement du système sur un serveur de l'entreprise	127
10.6.1	Architecture de Déploiement	127
10.6.2	Déploiement Unifié avec Docker Compose	128
10.6.3	Étapes d'hébergement	128
10.7	Mise en place d'une pipeline CI/CD avec Jenkins	129
10.7.1	Objectifs de la pipeline	129
10.7.2	Configuration de l'Environnement Jenkins	129
10.7.2.1	Installation et Configuration Initiale	129
10.7.2.2	Configuration des Plugins	130
10.7.2.3	Gestion des Credentials et Sécurité	130
10.7.3	Description des étapes de la pipeline (Jenkinsfile)	130
10.8	Résultats et Démonstration	131
10.8.1	Screenshots des containers en exécution	131
10.8.2	Exécution de la pipeline CI/CD	132
10.9	Conclusion	133
	Conclusion générale et perspectives	134
	Bibliographie	134
	Résumé	138
	Abstract	139

Liste des figures

Table des figures

2.1	Logo Lumière Logistique	9
2.2	Organigramme de Lumière Logistique.	10
3.1	Diagramme de cas d'utilisation globale du TMS.	21
3.2	Diagramme de Gantt - Planification des Sprints	24
3.3	Architecture générale à trois niveaux	24
3.4	Architecture logique de l'application LUMTMS	27
3.5	Architecture physique de l'application LUMTMS	28
4.1	Diagramme de cas d'utilisation du Sprint 1	34
4.2	Diagramme de classes du Sprint 1	39
4.3	Diagramme de séquence - Authentification	40
4.4	Diagramme de séquence - "Modifier profil"	41
4.5	Diagramme de séquence - "Ajouter un compte utilisateur"	42
4.6	Interface de connexion de l'application LUMTMS	43
4.7	Interface de gestion des utilisateurs	43
4.8	Interface du profil utilisateur	44
5.1	Diagramme de cas d'utilisation du Sprint 2	48
5.2	Diagramme de classes du Sprint 2	51
5.3	Diagramme de séquence - "Ajouter un article"	52
5.4	Diagramme de séquence - "ajouter un client"	53
5.5	Interface de gestion des articles	54
5.6	Interface ajouter article	54
5.7	Interface de gestion des clients	55

5.8	Interface ajouter des clients	56
5.9	Interface consulter client	56
6.1	Diagramme de cas d'utilisation du Sprint 3	61
6.2	Diagramme de classes du Sprint 3	67
6.3	Diagramme de séquence – "Ajouter un camion"	68
6.4	Diagramme de séquence – "Consulter la localisation d'un camion"	69
6.5	Diagramme de séquence – "Ajouter un chauffeur"	70
6.6	Diagramme de séquence – "Modifier l'état d'un chauffeur (mettre en repos)"	71
6.7	Interface de gestion des camions	72
6.8	Interface d'ajout d'un camion	72
6.9	Interface de consultation d'un camion	73
6.10	Interface de consultation de la localisation des camions	73
6.11	Interface de consultation de la localisation d'un camion	74
6.12	Interface de gestion des chauffeurs	75
6.13	Interface d'ajout d'un chauffeur	75
7.1	Diagramme de cas d'utilisation du Sprint 4	79
7.2	Diagramme de classe du Sprint 4	84
7.3	Diagramme de séquence - Importation des OT	85
7.4	Diagramme de séquence - Exportation des OT	86
7.5	Diagramme de séquence - Création d'un OT	87
7.6	Diagramme de séquence - Duplication d'un OT	88
7.7	Interface de gestion des Ordres de Transport	89
7.8	Interface de création d'un Ordre de Transport	90
7.9	Exemple de structure d'un fichier d'importation PLA	90
8.1	Diagramme des cas d'utilisation du Sprint 5	95
8.2	Diagramme de classe du Sprint 5	100
8.3	Diagramme de séquence – Création d'un voyage	101
8.4	Diagramme de séquence – Déplanification d'un voyage	102
8.5	Diagramme de séquence – Création d'une tournée générique	103
8.6	Diagramme de séquence – Création automatique d'un voyage	104
8.7	Interface de planification des voyages	105
8.8	Interface de gestion des voyages	105
8.9	Interface du chemin du voyage sur carte	106
8.10	Interface de gestion des tournées génériques	107
9.1	Diagramme des cas d'utilisation du Sprint 6	109
9.2	Diagramme de séquence - Consulter le planning journalier	114
9.3	Diagramme de séquence - Consulter le tableau de bord (KPIs)	114

9.4	Diagramme de séquence - Interagir avec le chatbot	115
9.5	Tableau de bord avec les KPIs principaux	116
9.6	Détail des indicateurs de performance	116
9.7	Vue du planning des voyages	117
9.8	Interaction avec le chatbot pour obtenir des informations	118
9.9	Exemple de réponse structurée du chatbot	119
10.1	Cycle de vie d'une application avec Docker	124
10.2	Users-service Dockerfile	125
10.3	Configuration du service MYSQL dans le fichier Docker Compose	126
10.4	Docker-compose.yml	127
10.5	Liste des conteneurs Docker en exécution sur le serveur	132
10.6	Visualisation de la pipeline CI/CD dans Jenkins Blue Ocean	133

Liste des tableaux

Liste des tableaux

1.1	Exemples de TMS les plus utilisés avec points forts et négatifs	6
3.1	Product Backlog du projet LUMTMS	21
4.1	Sprint Backlog du Sprint 1	33
4.2	Description textuelle du cas d'utilisation "S'authentifier"	35
4.3	Description textuelle du cas d'utilisation "Modifier profil"	36
4.4	Description textuelle du cas d'utilisation "Ajouter un compte utilisateur"	37
5.2	Description textuelle du cas d'utilisation "Ajouter un article"	49
5.3	Description textuelle du cas d'utilisation "Ajouter un client"	50
6.1	Sprint Backlog du Sprint 3	59
6.2	Description textuelle du cas d'utilisation "Ajouter un camion"	62
6.3	Description textuelle du cas d'utilisation "Consulter la localisation d'un camion"	63
6.4	Description textuelle du cas d'utilisation "Ajouter un chauffeur"	64
6.5	Description textuelle du cas d'utilisation " Modifier l'etat d'un chauffeur (mettre en repos)"	65
7.1	Sprint Backlog du Sprint 4	78
7.2	Description textuelle du cas d'utilisation "Importation des OT"	80
7.3	Description textuelle du cas d'utilisation "Exportation des OT"	81
7.4	Description textuelle du cas d'utilisation "Créer un Ordre de Transport (OT)"	82
7.5	Description textuelle du cas d'utilisation "Dupliquer un OT"	83
8.1	Sprint Backlog du Sprint 5	93

8.2	Description textuelle du cas d'utilisation "Créer manuellement un voyage"	96
8.3	Description textuelle du cas d'utilisation "Déplanifier un voyage"	97
8.4	Description textuelle du cas d'utilisation "Créer une tournée générique"	97
8.5	Description textuelle du cas d'utilisation "Création automatique d'un voyage"	98
9.1	Backlog du Sprint 6	109
9.2	Description textuelle - Consulter le planning journalier	110
9.3	Description textuelle - Consulter le tableau de bord	111
9.4	Description textuelle - Interagir avec le chatbot	112
10.1	Backlog du Sprint 7	122

Liste des abréviations

3PL	Third-Party Logistics (Logistique tierce partie)
AI	Artificial Intelligence (Intelligence Artificielle)
API	Application Programming Interface (Interface de Programmation d'Application)
CI/CD	Continuous Integration/Continuous Delivery (Intégration Continue/Livraison Continue)
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
CSS	Cascading Style Sheets (Feuilles de Style en Cascade)
EDI	Electronic Data Interchange (Échange de Données Informatisé)
ERP	Enterprise Resource Planning (Planification des Ressources de l'Entreprise)
ETA	Estimated Time of Arrival (Heure d'Arrivée Estimée)
FMCG	Fast-Moving Consumer Goods (Produits de Grande Consommation)
HTML	HyperText Markup Language (Langage de Balisage d'Hypertexte)
IDE	Integrated Development Environment (Environnement de Développement Intégré)
IoT	Internet of Things (Internet des Objets)
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator (Indicateur de Performance Clé)
LLM	Large Language Model (Grand Modèle de Langage)
LUMTMS	Lumière Logistique Transport Management System
ML	Machine Learning (Apprentissage Automatique)
OT	Ordre de Transport
PME	Petites et Moyennes Entreprises
REST	Representational State Transfer
SaaS	Software as a Service (Logiciel en tant que Service)
SDK	Software Development Kit (Kit de Développement Logiciel)
SQL	Structured Query Language (Langage de Requête Structuré)
TMS	Transport Management System (Système de Gestion du Transport)
UML	Unified Modeling Language (Langage de Modélisation Unifié)
US	User Story (Récit Utilisateur)
WMS	Warehouse Management System (Système de Gestion d'Entrepôt)

XML eXtensible Markup Language (Langage de Balisage Extensible)

Introduction Générale

Dans un environnement économique marqué par la mondialisation, la digitalisation et l'évolution rapide des technologies, le secteur du transport et de la logistique est confronté à des défis majeurs. Les entreprises doivent faire face à des contraintes croissantes en matière de coûts, de délais, de traçabilité et de satisfaction client. Pour répondre à ces enjeux, l'adoption d'outils numériques performants est devenue indispensable.

Parmi ces outils, les *Systèmes de Gestion du Transport* (TMS – *Transport Management Systems*) jouent un rôle fondamental. Ils permettent de planifier, suivre, optimiser et analyser l'ensemble des opérations de transport, tout en assurant une meilleure communication entre les différents acteurs de la chaîne logistique. Les TMS modernes offrent ainsi un gain de temps, une réduction des erreurs humaines et une amélioration globale de la qualité de service.

Le présent rapport s'inscrit dans le cadre d'un Projet de Fin d'Études réalisé au sein de l'entreprise **Lumière Logistique**, spécialisée dans le transport et la livraison de marchandises. L'objectif principal de ce projet est la conception, le développement et le déploiement d'une solution TMS personnalisée, répondant aux besoins spécifiques de l'entreprise, avec pour finalité d'optimiser la gestion des ordres de transport et des ressources.

Ce projet a été mené selon une **méthodologie agile**, avec une division en *sprints* permettant d'assurer une progression itérative, une validation continue et une meilleure adaptation aux besoins fonctionnels évolutifs. L'implémentation technique repose sur des technologies web modernes telles que **Spring Boot** pour le développement du *backend*, **Angular** pour l'interface utilisateur, ainsi que l'intégration de services externes pour enrichir les fonctionnalités de suivi et de communication.

Enfin, une attention particulière a été portée sur l'aspect **déploiement** de l'application, incluant la configuration de l'environnement serveur, la gestion de la base de données, ainsi que les tests fonctionnels et techniques afin de garantir la stabilité et la fiabilité de la solution. *

Chapitre 1

Problèmes du Transport et de la Livraison – État des Lieux et Évolution

1.1 Introduction

Le transport et la livraison de marchandises sont devenus un pilier central de la chaîne logistique moderne. Autrefois limité à la simple mobilité des biens, ce secteur est désormais un levier stratégique de compétitivité et de satisfaction client. La mondialisation, le e-commerce et les attentes croissantes des consommateurs (livraison rapide, traçabilité, personnalisation) renforcent cette importance. Ainsi, le transport n'est plus une fonction opérationnelle isolée mais un élément clé d'une stratégie globale d'optimisation et de digitalisation. Les technologies numériques et les systèmes intelligents favorisent cette transformation. Dans ce contexte, les TMS (Transport Management Systems) apparaissent comme des outils incontournables pour planifier, suivre et optimiser les opérations logistiques. [1].

1.2 Situation Actuelle du Transport et de la Logistique

Le secteur mondial du transport connaît une croissance constante, stimulée par la **mondialisation des échanges**, l'**externalisation croissante des services logistiques**, ainsi que la **transformation des modèles de consommation**. Selon Statista[2], le marché mondial du transport de fret est estimé à plusieurs milliers de milliards de dollars, avec des perspectives de croissance soutenue dans les années à venir.

Cependant, cette croissance s'accompagne de défis majeurs que les entreprises doivent relever :

- **Complexité accrue des chaînes logistiques** : Les réseaux d'approvisionnement sont de plus en plus longs, fragmentés et internationaux. La gestion des différents intervenants (fournisseurs,

transporteurs, plateformes logistiques) nécessite une coordination rigoureuse et en temps réel.

- **Accélération de la digitalisation** : Les entreprises intègrent des technologies avancées telles que l'*Internet des Objets (IoT)*, l'*intelligence artificielle (IA)*, le *machine learning* ou encore la *blockchain*, afin de sécuriser, automatiser et fiabiliser leurs opérations.
- **Pression sur les coûts** : L'optimisation du budget logistique est cruciale. Elle repose notamment sur une meilleure utilisation des ressources (véhicules, chauffeurs, carburant), la réduction des kilomètres parcourus à vide, ainsi que l'automatisation des processus de planification.
- **Nouvelles attentes des clients** : Les clients finaux, qu'ils soient particuliers ou professionnels, exigent des délais de livraison plus courts, un suivi précis des commandes, des options flexibles et une communication fluide.
- **Contraintes réglementaires et environnementales** : Le secteur est de plus en plus encadré par des réglementations relatives à la sécurité, à la conformité douanière, ainsi qu'à la protection de l'environnement. La réduction de l'empreinte carbone constitue désormais un objectif stratégique pour de nombreux acteurs du transport.

Dans ce contexte exigeant, les entreprises doivent s'appuyer sur des **outils performants** pour garder le contrôle sur leurs flux et améliorer leur performance. C'est dans ce cadre que les **Transport Management Systems (TMS)** se positionnent comme une solution incontournable .

sectionQu'est-ce qu'un Transport Management System (TMS) ?

Un **Transport Management System (TMS)** est un système informatique conçu pour **gérer, automatiser** et **optimiser** les opérations de transport. Il s'agit d'un outil stratégique couvrant l'ensemble du processus logistique lié au transport : de la planification à l'exécution, en passant par le suivi, la facturation et l'analyse des performances.

Un TMS offre généralement les fonctionnalités clés suivantes :

- **Planification des tournées** : prise en compte des contraintes telles que les délais de livraison, la capacité des véhicules, la disponibilité des chauffeurs, ou encore les restrictions géographiques.
- **Optimisation des trajets et des coûts** : grâce à des algorithmes d'optimisation (tels que le *Travelling Salesman Problem – TSP* ou le *Vehicle Routing Problem – VRP*), le TMS propose les itinéraires les plus efficaces, réduisant ainsi les coûts opérationnels.
- **Gestion documentaire** : édition automatique des documents de transport (bons de livraison, lettres de voiture, documents douaniers, etc.) et intégration avec les systèmes *ERP*.
- **Suivi en temps réel** : géolocalisation des véhicules, alertes en cas de retard, notifications envoyées aux clients et mise à disposition d'indicateurs de performance.
- **Analyse et reporting** : mise en place de tableaux de bord, statistiques et indicateurs de performance (*KPI*), permettant un pilotage précis et efficace de la performance logistique.

- **Intégration avec un ERP** : un *Enterprise Resource Planning (ERP)* est un système logiciel intégré qui gère les processus métier clés, tels que la comptabilité, les ressources humaines et la gestion des stocks, permettant une synchronisation fluide avec le TMS pour une gestion administrative et logistique cohérente [3].

Enfin, un TMS moderne est souvent **intégré avec d'autres systèmes**, tels que les *ERP* pour la gestion administrative, ou les *WMS* pour la gestion d'entrepôt. Il s'inscrit dans une architecture globale orientée vers la **logistique 4.0** : connectée, réactive et durable [4][5].

1.3 Limites de la Gestion Traditionnelle du Transport

De nombreuses entreprises, notamment les **PME** ou les **transporteurs locaux**, utilisent encore des méthodes traditionnelles pour gérer leurs opérations : *planification manuelle sur papier ou Excel*, *échanges téléphoniques ou e-mails* avec les clients et chauffeurs, et *absence de suivi en temps réel*.

Cette gestion artisanale entraîne plusieurs limitations majeures :

- **Erreurs fréquentes** : dans la planification des tournées, l'affectation des ressources ou le respect des délais.
- **Manque de visibilité** : absence d'informations fiables sur l'état réel des livraisons, ce qui rend la réactivité quasi impossible.
- **Retards imprévus** : mauvaise anticipation des contraintes liées au trafic, à la météo ou aux imprévus opérationnels.
- **Surcoûts logistiques** : causés par une planification inefficace (kilomètres parcourus à vide, livraisons en double, mauvaise allocation des ressources).
- **Insatisfaction client** : due à un manque de communication, de transparence et à une gestion peu flexible des imprévus.

Les **Transport Management Systems (TMS)** permettent de pallier ces limites en **automatisant**, **fiabilisant** et **optimisant** l'ensemble du processus de transport. Ils contribuent ainsi à **réduire les coûts** tout en **améliorant la qualité de service**.

Selon Gartner (2023), les solutions *SAP TM*, *Oracle OTM* et *Manhattan* dominent le marché mondial des TMS, grâce à leur **couverture fonctionnelle étendue** et leur **flexibilité d'intégration** [6][7].

1.4 Les Tendances Actuelles des TMS

L'évolution technologique pousse les éditeurs de **Transport Management Systems (TMS)** à innover constamment afin de répondre aux besoins croissants de performance et de flexibilité. Les principales tendances observées sur le marché sont les suivantes :

- **Cloud et SaaS** : Les TMS sont de plus en plus hébergés dans le cloud, accessibles à distance, avec une facturation à l'usage. Cela permet une mise en œuvre plus rapide, des mises à jour automatiques et une accessibilité universelle.
- **Intelligence Artificielle (IA) et Machine Learning** : Ces technologies sont intégrées pour prédire les retards, proposer des plans alternatifs et améliorer les prises de décision à grande échelle.
- **IoT et tracking intelligent** : Grâce à la connectivité des véhicules et objets, les TMS peuvent suivre en temps réel la température, l'humidité, la position et détecter les incidents éventuels.
- **Mobilité et applications terrain** : Les chauffeurs accèdent à leurs missions via des applications mobiles qui remontent automatiquement l'information vers le TMS.
- **Durabilité environnementale** : Les TMS intègrent désormais des indicateurs d'empreinte carbone, optimisent les trajets pour limiter les émissions, et favorisent les solutions écologiques.

Ces innovations redéfinissent la manière de piloter la logistique transport, tout en renforçant la **transparence**, la **collaboration** et la **réactivité** [8].

Dans le domaine de la gestion du transport, plusieurs solutions logicielles se distinguent par leurs fonctionnalités, leur flexibilité et leur intégration aux systèmes existants. Parmi elles :

SAP Transportation Management [9]

SAP Transportation Management est une solution intégrée de gestion du transport, particulièrement prisée pour son intégration fluide avec l'écosystème SAP. Ce TMS offre des fonctionnalités avancées pour la planification, l'optimisation des itinéraires, la gestion des coûts de transport et le suivi en temps réel. Il est particulièrement adapté aux grandes entreprises cherchant à réduire leurs coûts logistiques grâce à une automatisation poussée et une analyse approfondie des données. Cependant, son coût élevé, la complexité de sa mise en œuvre et sa forte dépendance à l'environnement SAP peuvent poser des défis, notamment pour les organisations n'utilisant pas déjà d'autres modules SAP.

Oracle Transportation Management (OTM) [10]

Oracle Transportation Management (OTM) est un TMS robuste conçu pour gérer efficacement les processus de transport, notamment la planification, l'exécution et la gestion des règlements. Sa flexibilité dans la gestion des taux et des contrats, ainsi que son interface intuitive pour la construction des expéditions, en font une solution prisée par les entreprises ayant des besoins complexes. OTM excelle dans l'automatisation des processus, mais il peut souffrir de problèmes d'intégration avec d'autres systèmes, d'une gestion parfois complexe et d'une implémentation lente. De plus, certains

utilisateurs signalent des bugs logiciels, ce qui peut affecter la fiabilité dans certains contextes.

Manhattan Associates TMS [11]

Manhattan Associates TMS est une solution de gestion du transport reconnue pour sa scalabilité et sa capacité à s'intégrer rapidement dans les opérations de distribution. Ce système est particulièrement adapté aux entreprises recherchant une mise en œuvre rapide et une flexibilité dans la gestion de leurs flux logistiques. Il offre un support robuste pour les opérations de distribution, mais il peut être moins efficace pour répondre à des besoins très spécifiques ou personnalisés. Les évaluations des utilisateurs sont parfois mitigées, certains louant sa facilité d'utilisation, tandis que d'autres pointent des limitations dans des cas d'utilisation complexes.

Cette table contient ces TMS populaires, leurs points forts et leurs limites.

TABLE 1.1 – Exemples de TMS les plus utilisés avec points forts et négatifs

Nom du TMS	Points forts	Points négatifs
SAP Transportation Management	Intégration excellente avec l'écosystème SAP; fonctionnalités complètes pour la planification et l'optimisation; réduction des coûts	Coût élevé; complexité de mise en œuvre; forte dépendance à SAP [9]
Oracle Transportation Management (OTM)	Gestion intuitive des règlements; flexibilité dans la gestion des taux et contrats; construction facile des expéditions; automatisation robuste	Problèmes d'intégration; gestion complexe; implémentation lente; présence de bugs logiciels [10]
Manhattan Associates TMS	Scalable et connectable; implémentation rapide; adapté aux opérations de distribution; support flexible	Moins adapté aux besoins très spécifiques; évaluations utilisateurs mitigées [11]

Bien que performantes, les TMS leaders du marché présentent plusieurs inconvénients :

- **Coût d'entrée très élevé** : achat de licences, frais de configuration, maintenance annuelle.
- **Complexité d'intégration** : nécessite souvent une refonte des processus internes ou une adaptation coûteuse.
- **Peu de flexibilité** : personnalisation limitée sans surcoût important.
- **Temps de mise en œuvre long** : plusieurs mois voire un an pour les projets complets.
- **Courbe d'apprentissage élevée** : les utilisateurs doivent être formés sur des interfaces riches et complexes.
- **Solutions parfois trop génériques** : elles manquent de finesse pour des cas spécifiques comme celui de Lumière Logistique.
- **Dépendance forte au fournisseur** : pour le support, la maintenance, l'évolution du produit.

1.5 Conclusion

Le transport et la logistique représentent aujourd’hui des leviers stratégiques essentiels dans un environnement globalisé, compétitif et en constante évolution. Les **Transport Management Systems (TMS)** s’imposent comme des solutions incontournables pour répondre aux défis actuels : digitalisation, automatisation, suivi en temps réel, maîtrise des coûts et respect des enjeux environnementaux. En offrant une meilleure visibilité et une optimisation des flux, ils renforcent la performance et la satisfaction client. Ce projet de fin d’études, réalisé au sein de *Lumière Logistique*, s’inscrit dans cette dynamique et vise à concevoir une solution TMS adaptée aux besoins spécifiques de l’entreprise, en intégrant les meilleures pratiques du secteur [1].

Chapitre 2

Contexte du Projet

2.1 Introduction

Ce chapitre vise à présenter le contexte du projet de développement du système de gestion des transports (LUMTMS) pour Lumière Logistique, une entreprise tunisienne spécialisée dans la logistique 3PL et la distribution de produits de grande consommation. Dans un secteur marqué par des exigences croissantes en termes d'efficacité, de traçabilité et de réactivité, l'étude du cadre organisationnel, des outils existants et des défis opérationnels est essentielle. Cette analyse permet de poser les bases du projet, en identifiant les limites actuelles, notamment celles d'Akanea TMS, et en justifiant le choix d'une solution personnalisée, LUMTMS, soutenue par une méthodologie agile (SCRUM) et une modélisation UML. Ce chapitre détaille ainsi l'entreprise, la problématique, la solution proposée et les approches méthodologiques retenues.

2.2 Présentation de l'organisme d'accueil

2.2.1 Présentation de l'entreprise

La société « Lumière Logistique » a été créée en 2012 dans le but de reprendre les activités des services logistiques partagés par les différentes entités du Groupe SLAMA, qui compte 591 collaborateurs (managers et ouvriers) sur tous les sites en Tunisie.

Lumière Logistique est un fournisseur de prestations logistiques de type Troisième Partie Logistique (3PL), spécialisé dans le domaine des produits de grande consommation FMCG (Fast-Moving Consumer Goods) et de la distribution.

Lumière Logistique assure l'ensemble de la chaîne logistique, s'appuyant principalement sur une



FIGURE 2.1 – Logo Lumière Logistique

capacité importante d'entreposage et de distribution qui lui a permis d'entretenir et de consolider des relations étroites avec ses multiples clients.

L'entreposage et la distribution constituent les deux axes fondamentaux sur lesquels Lumière Logistique construit sa stratégie et contribue à une logistique performante et intensive.

Cette entreprise est spécialisée dans deux catégories d'activités principales : la première concerne l'activité de logistique qui comprend l'entreposage, la préparation, la livraison, le co-packaging et la para-logistique ; la deuxième concerne l'activité de transit,

Lumière Logistique dispose de plusieurs dépôts de stockage et de distribution, dont les plus importants sont ceux de Sahlin, de Bouargoub et de Mhamdia. Ces sites regroupent les principales fonctions de Lumière Logistique, notamment la gestion de stockage et la distribution des produits.

2.2.2 Organigramme de l'entreprise

L'organigramme est un graphique représentant la structure d'une entreprise, illustrant à la fois les différents éléments de l'organisation et leurs relations hiérarchiques. La Figure présente l'organigramme de la société Lumière Logistique.

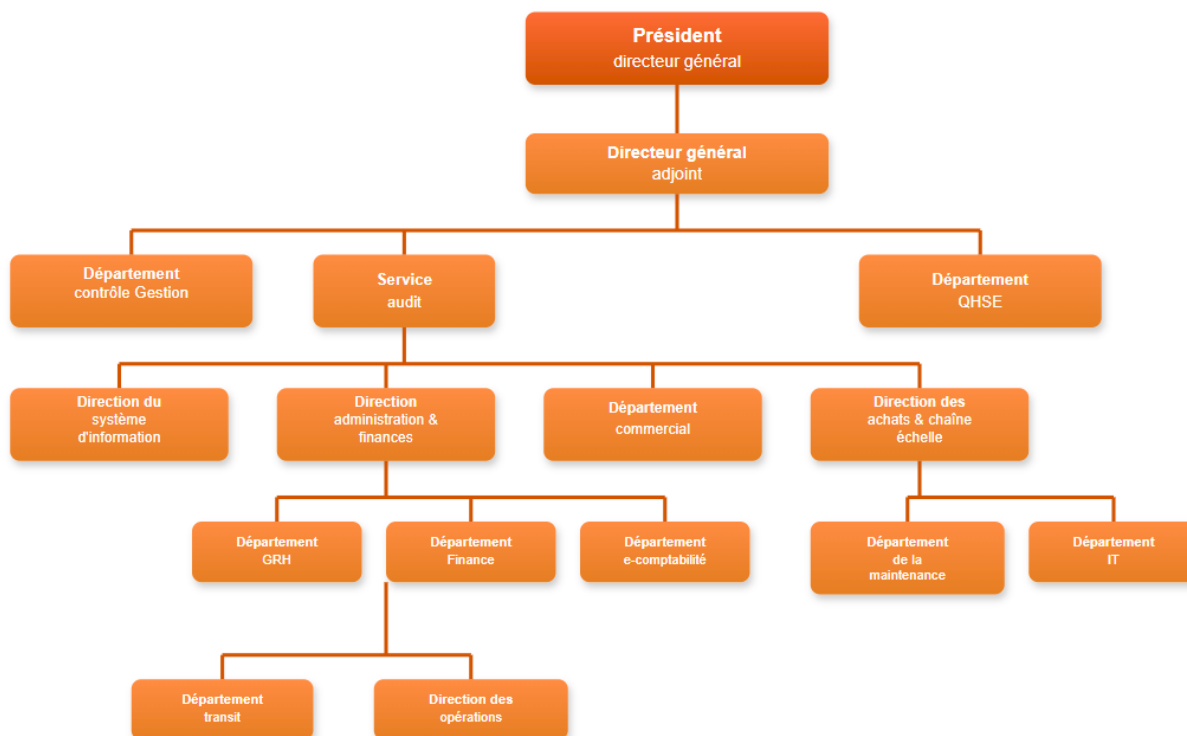


FIGURE 2.2 – Organigramme de Lumière Logistique.

2.3 Présentation du projet

Ce projet s’inscrit dans le cadre d’un stage de fin d’études de six mois réalisé au sein de Lumière Logistique, une entreprise spécialisée dans la gestion du transport et de la chaîne logistique. Dans un contexte marqué par la digitalisation croissante des processus logistiques, la nécessité d’une solution performante et adaptable devient cruciale pour répondre aux exigences du marché et des clients.

2.3.1 Problématique

Lumière Logistique, opérant dans le secteur du transport et de la distribution, fait face à des défis opérationnels significatifs dus à une gestion majoritairement manuelle de ses activités. Malgré l’utilisation d’outils de base, l’organisation globale des opérations reste artisanale, ce qui engendre plusieurs problématiques majeures :

- **Sélection non automatisée des ressources :** La répartition des missions se fait de manière empirique, basée sur l’expérience des agents, sans système d’optimisation des camions ou chauff-

feurs disponibles, ce qui mène à une mauvaise utilisation des ressources et à un manque de rentabilité.

- **Communication inefficace** : L’envoi des informations logistiques se fait par SMS manuels, ce qui multiplie les risques d’oubli, d’erreurs de saisie, et ralentit le processus.
- **Absence de tracking en temps réel** : Les camions ne sont pas suivis dynamiquement, rendant difficile l’anticipation des retards, des incidents, ou des changements de dernière minute.
- **Dépendance aux fichiers Excel** : La gestion des ordres de transport, des tournées et des ressources se fait principalement à l’aide de tableurs non centralisés, sources fréquentes d’erreurs, de doublons et de désynchronisation des données.
- **Absence d’analyse décisionnelle** : Aucun système de reporting automatisé ou de KPI (indicateurs de performance) n’est mis en place, ce qui limite la capacité de l’entreprise à évaluer ses performances, détecter les inefficiences et prendre des décisions éclairées.
- **Risque d’erreurs humaines accru** : Le recours excessif aux traitements manuels (saisie, planification, communication) augmente les risques d’oubli, de doublons, de mauvaises affectations ou de non-conformité aux contraintes clients.
- **Perte de temps opérationnel** : L’ensemble de ces lacunes provoque un gaspillage considérable de temps dans la coordination quotidienne des livraisons, nuisant à la productivité et à la satisfaction client.

Dans ce contexte, il apparaît impératif de concevoir une solution moderne, automatisée, centralisée et évolutive, qui s’aligne avec les exigences métiers de Lumière Logistique tout en améliorant la fiabilité, l’agilité et la rentabilité des opérations de transport.

2.4 La Solution Propriétaire Utilisée par Lumière Logistique : Akenea TMS

Akenea TMS est une suite logicielle complète dédiée à la gestion du transport routier et international, développée par l’entreprise française Akenea, fondée en 1983[13]. Spécialisée dans les solutions supply chain, logistique et douane, Akenea est reconnue pour ses produits performants et modulaires.

Lumière Logistique utilise Akenea TMS pour la gestion de ses opérations de transport. Parmi les points positifs de cette solution, on note sa robustesse dans la planification des tournées, son intégration potentielle avec d’autres systèmes logistiques, et sa capacité à gérer des volumes importants de données logistiques. Elle offre également des fonctionnalités avancées pour l’optimisation des itinéraires et la conformité réglementaire, ce qui contribue à une meilleure efficacité opérationnelle lorsqu’elle est pleinement exploitée.

Cependant, Lumière Logistique ne dispose que de la licence pour le module de planification, en raison

des coûts élevés associés aux autres modules.

2.5 Critique de la Solution Utilisée à Lumière Logistique (Akenea TMS)

Malgré les avantages potentiels d'Akenea TMS, son utilisation actuelle chez Lumière Logistique est limitée au module de planification, ce qui entraîne plusieurs contraintes et inefficacités majeures :

- **Coût très élevé des licences** : Les frais pour débloquer l'ensemble des modules (tracking, facturation, gestion documentaire, etc.) sont prohibitifs, limitant l'accès aux fonctionnalités avancées.
- **Utilisation partielle** : Seule la planification est exploitée, ce qui diminue fortement la rentabilité globale de l'outil et laisse de côté des aspects cruciaux comme le suivi et l'analyse.
- **Aucune automatisation des communications** : Les envois d'informations (SMS/emails) restent manuels, augmentant les risques d'erreurs et les délais.
- **Absence totale de suivi en temps réel des camions** : Sans module dédié, il est impossible d'anticiper les retards ou incidents, impactant la réactivité.
- **Pas de génération de rapports personnalisés** : L'absence d'outils d'analyse rend difficile l'évaluation des performances et la prise de décisions basées sur des données.
- **Choix du camion et du chauffeur basé sur l'expérience des agents** : Sans optimisation automatisée, la sélection reste subjective et inefficace.
- **Aucune visualisation des trajets sur carte** : Cela complique la planification visuelle et l'ajustement en temps réel.
- **Pas de chatbot ou d'assistance intelligente** : L'interface manque de support interactif, rendant l'utilisation plus ardue pour les agents.
- **Lenteur et dépendance aux fichiers Excel** : Malgré l'outil, les processus restent hybrides, avec une forte reliance sur des tableurs externes, source de désynchronisation.
- **Courbe d'apprentissage élevée** : Certains agents non formés peinent à maîtriser l'outil, limitant son adoption.

2.6 Solution proposée : Lumière Transport Management System (LUMTMS)

Dans le cadre de la modernisation des opérations de transport de Lumière Logistique, la solution Lumière Transport Management System (LUMTMS) est proposée comme un système de gestion de

transport complet, intelligent et évolutif. Ce système a pour objectif de remplacer les pratiques manuelles actuelles, souvent basées sur des fichiers Excel, un envoi manuel de SMS, et une planification basée sur l'expérience humaine, par une solution numérique performante, centralisée et automatisée. LUMTMS repose sur une architecture **microservices**, garantissant modularité, scalabilité et maintenabilité. Chaque service est déployé de manière indépendante, ce qui facilite l'évolution et l'intégration de nouvelles fonctionnalités. L'ensemble de la solution s'appuie sur les technologies suivantes :

- **Spring Boot** : framework pour la création de microservices REST, sécurisé via **Spring Security** et **JWT** afin d'assurer une gestion fiable des accès et des authentifications.
- **Angular** : développement d'une interface utilisateur moderne, responsive et dynamique, favorisant une expérience fluide et intuitive.
- **MySQL** : une base de données par microservice, garantissant l'indépendance et la cohérence des modules.
- **Docker & Docker Compose** : conteneurisation et orchestration, permettant un déploiement simplifié, reproductible et portable sur différents environnements.
- **Geode** : outil d'intégration pour l'import/export des ordres de transport.
- **OpenRouteService** : calcul d'itinéraires optimisés en tenant compte des contraintes réelles de transport.
- **OpenStreetMap** : visualisation cartographique interactive des trajets.
- **JasperReports** : génération automatisée de rapports et documents PDF personnalisés.
- **LangChain + Gemini** : intégration d'un chatbot intelligent, offrant assistance et support décisionnel avancé.

Le système sera déployé sur un serveur Linux distant, avec une infrastructure conteneurisée, assurant portabilité, performance, et facilité de maintenance.

LUMTMS couvre l'ensemble du cycle logistique transport :

- Importation et gestion des ordres de transport
- Planification automatique et manuelle des voyages
- Suivi en temps réel des chauffeurs, camions, clients et trajets
- Envoi automatique de notifications SMS/email
- Génération de rapports détaillés et visualisation des indicateurs clés (KPIs)
- Interaction intelligente avec les utilisateurs via un chatbot TMS

Le système LUMTMS se distingue par plusieurs atouts majeurs :

- **Architecture microservices** : chaque module (voyage, camion, chauffeur, utilisateur, etc.) fonctionne de manière indépendante, permettant une maintenance plus efficace et des déploiements ciblés

- **Intégration intelligente** : import/export des ordres via Geode, calcul d'itinéraires via OpenRouteService, génération de rapports via JasperReports
- **Visualisation géographique avancée** : cartes interactives pour les trajets et les ressources (camions, chauffeurs), facilitant la supervision et la planification
- **Chatbot embarqué** : aide immédiate aux agents de transport grâce à un assistant intelligent, capable de répondre à des questions spécifiques sur les voyages, camions, chauffeurs, etc.
- **Automatisation étendue** : création automatique de voyages à partir de tournées génériques, notifications automatisées, affectation intelligente des ressources
- **Interface utilisateur moderne** : Angular permet une expérience fluide et responsive, accessible depuis différents supports
- **Suivi et analyse en temps réel** : tableaux de bord dynamiques avec indicateurs de performance, facilitant la prise de décision
- **Traçabilité et sécurité** : authentification via JWT, gestion des rôles, journalisation des actions utilisateurs

En résumé, LUMTMS représente une avancée technologique et fonctionnelle majeure, qui répond aux limites de l'existant, en offrant une solution moderne, intelligente et évolutive pour la gestion des transports chez Lumière Logistique.

2.7 Méthode de travail et planification

2.7.1 Approches de gestion de projet

La réussite d'un projet dépend du choix d'une méthode de gestion adaptée à son contexte. Deux approches principales sont couramment utilisées : les méthodes traditionnelles (en cascade) et les méthodes agiles. Pour un système de gestion des transports (TMS), le choix de la méthode est crucial pour répondre aux besoins dynamiques du projet.

Les méthodes en cascade, basées sur une progression séquentielle des phases (expression des besoins, conception, développement, tests, déploiement), offrent une structure rigoureuse et une documentation détaillée. Elles conviennent aux projets aux exigences fixes, mais leur manque de flexibilité les rend inadaptées pour un TMS. En effet, un TMS nécessite une adaptation constante aux évolutions des besoins (nouvelles réglementations, ajustements des priorités) et une collaboration étroite avec les utilisateurs, ce que la rigidité des méthodes en cascade ne permet pas.

Les méthodes agiles, inspirées du Manifeste Agile [12], sont privilégiées pour ce projet en raison de leurs nombreux avantages :

- **Adaptabilité** : Les itérations courtes permettent d'intégrer rapidement les changements dans les besoins.
- **Livraison rapide** : Les fonctionnalités prioritaires sont développées et livrées tôt, offrant une valeur immédiate.
- **Collaboration** : Les échanges réguliers avec les utilisateurs garantissent un produit aligné sur leurs attentes.
- **Réduction des risques** : Les itérations fréquentes facilitent la détection précoce des problèmes, minimisant les erreurs coûteuses.

Ainsi, pour un TMS, les méthodes agiles sont idéales, car elles favorisent une approche flexible et collaborative, essentielle pour répondre aux exigences évolutives du secteur des transports.

2.7.2 Choix de la méthode : SCRUM

Le développement du système de gestion des transports (LUMTMS) doit répondre à une complexité fonctionnelle et technique liée à la diversité des acteurs (clients, transporteurs, planificateurs) et à l'évolution fréquente des besoins. SCRUM, une méthode agile décrite dans le Guide SCRUM [14], est choisie pour sa flexibilité et son cadre itératif, permettant des livraisons rapides, une adaptation aux changements et une collaboration étroite avec les utilisateurs.

SCRUM offre plusieurs avantages pour LUMTMS :

- **Priorisation** : Un Product Backlog dynamique classe les fonctionnalités par valeur métier.
- **Adaptabilité** : Les sprints (2 à 4 semaines) intègrent rapidement les évolutions des besoins.
- **Livraison continue** : Chaque sprint produit un incrément fonctionnel testé, potentiellement déployable.
- **Collaboration** : Les revues de sprint impliquent les utilisateurs pour aligner le produit sur leurs attentes.

Le cadre SCRUM s'appuie sur trois rôles clés : le **Product Owner**, qui définit et priorise les fonctionnalités ; le **Scrum Master**, qui facilite le processus et élimine les obstacles ; et l'**équipe de développement**, auto-organisée, qui livre les incréments. Les artefacts, tels que le **Product Backlog** (liste évolutive des besoins), le **Sprint Backlog** (tâches d'un sprint) et l'**Incrément** (version fonctionnelle), assurent la transparence. Les événements, comme le **Daily Scrum** (synchronisation quotidienne), la **Sprint Review** (retours utilisateurs) et la **Sprint Retrospective** (amélioration continue), structurent le projet.

Ainsi, SCRUM garantit une gestion efficace de LUMTMS, en répondant aux exigences évolutives du secteur des transports tout en maximisant la valeur livrée.

2.7.3 Modélisation fonctionnelle et technique avec UML

Le Unified Modeling Language (UML) est un langage standardisé pour modéliser graphiquement un système logiciel, facilitant la communication entre parties prenantes et la traçabilité des exigences [15]. Pour un système de gestion des transports (LUMTMS), UML est particulièrement adapté à la complexité fonctionnelle et technique, en soutenant le développement agile avec SCRUM.

Dans le cadre de LUMTMS, UML est utilisé de manière légère pour clarifier les besoins et structurer le développement sans alourdir le processus agile. Les diagrammes UML se divisent en deux catégories : les **diagrammes structurels** (ex. diagrammes de classes pour modéliser les entités comme Ordre ou Client) et les **diagrammes comportementaux** (ex. diagrammes de cas d'utilisation pour définir les user stories, diagrammes de séquence pour visualiser les interactions techniques). Ces diagrammes sont intégrés aux sprints : avant chaque sprint, les cas d'utilisation clarifient les besoins ; pendant le sprint, les diagrammes de séquence guident l'implémentation ; et lors de la planification, les diagrammes de classes structurent les entités.

Les avantages d'UML pour LUMTMS incluent :

- **Souplesse** : Modélisation progressive adaptée aux évolutions des besoins.
- **Communication** : Échanges clairs entre développeurs, utilisateurs et parties prenantes.
- **Réduction des erreurs** : Prévention des incohérences grâce à une visualisation ciblée.

Ainsi, UML renforce l'efficacité de SCRUM pour LUMTMS en assurant une modélisation agile, claire et adaptée aux exigences dynamiques du projet.

2.8 Conclusion

La gestion optimisée des opérations de transport chez Lumière Logistique nécessite une solution adaptée aux besoins spécifiques de l'entreprise. L'analyse a révélé les limites d'Akanea TMS, notamment en termes de fonctionnalités et de coûts, justifiant le développement de LUMTMS. En adoptant SCRUM et UML, ce projet garantit une approche agile, structurée et centrée sur la valeur métier, répondant aux enjeux de flexibilité et d'automatisation. Le chapitre suivant aborde le Sprint 0, consacré à l'étude préliminaire du projet.

Chapitre 3

Sprint 0 – Étude Préliminaire

3.1 Introduction

La Sprint 0 est une étape fondamentale dans la planification et l'organisation du développement du Système de Gestion de Transport (TMS). Ce sprint ne produit pas encore de fonctionnalités opérationnelles, mais sert à établir les bases du projet : identification des besoins, définition des fonctionnalités clés, choix technologiques, ainsi que la planification des itérations. Cette phase permet également de construire une vision globale du système à travers des modèles de cas d'utilisation et un product backlog bien structuré.

3.2 Analyse des besoins

L'analyse des besoins est essentielle pour assurer que le TMS réponde parfaitement aux attentes des utilisateurs finaux. Elle consiste à identifier les différents utilisateurs du système et à cerner leurs attentes fonctionnelles et non-fonctionnelles.

3.2.1 Identification des acteurs

Deux types d'utilisateurs principaux ont été identifiés dans le système :

— **Administrateur :**

- Dispose de tous les droits d'accès au système.
- Gère les utilisateurs, les configurations, les tournées, et supervise les opérations globales.
- Peut consulter des rapports détaillés et accéder aux KPIs.

— **Agent de Transport :**

- Gère les opérations quotidiennes : ordres de transport, voyages, chauffeurs, camions.
- Dispose d'un accès restreint (pas de gestion des utilisateurs ou de la configuration).
- Utilise le système pour le suivi des opérations et la planification.

3.2.2 Définition des besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités essentielles que le système LUMTMS doit offrir. Ils sont issus du Product Backlog et couvrent l'ensemble du cycle de vie des opérations de transport, depuis la création des ordres jusqu'à l'analyse des performances.

3.2.2.1 Gestion des Comptes et de l'Authentification

- **Authentification sécurisée** : Les utilisateurs doivent pouvoir s'authentifier via un système de sécurité basé sur Spring Security et JWT, garantissant une protection robuste des accès.
- **Gestion de profil** : Chaque utilisateur doit pouvoir consulter et modifier ses informations personnelles et ses privilèges.
- **Administration des utilisateurs** : L'administrateur doit pouvoir gérer l'ensemble des comptes utilisateurs (création, modification, suppression) et accéder à un journal d'audit retraçant les actions effectuées.

3.2.2.2 Gestion des Ordres de Transport (OT)

- **Import/Export via Geode** : Le système doit permettre l'échange d'ordres de transport avec des systèmes externes comme Geode, facilitant l'intégration des données logistiques.
- **CRUD des ordres** : Les utilisateurs doivent pouvoir créer, consulter, modifier et supprimer des ordres de transport.
- **Affectation aux voyages** : Les ordres doivent pouvoir être regroupés et affectés automatiquement ou manuellement à des voyages pour planifier les livraisons.

3.2.2.3 Gestion des Voyages

- **Création et planification** : Les voyages peuvent être créés à la main ou générés automatiquement à partir de tournées génériques prédéfinies.
- **Itinéraires optimisés** : Le système doit intégrer un calcul automatique des trajets via OpenRouteService, avec estimation des distances et durées.
- **Visualisation cartographique** : Les trajets doivent être affichés sur une carte interactive (OpenStreetMap), et chaque voyage doit générer un rapport PDF.

- **Notification des chauffeurs :** Un SMS automatique doit être envoyé au chauffeur à la création du voyage, avec les détails de sa mission.

3.2.2.4 Gestion des Ressources Matérielles

- **Gestion des camions :** L'administrateur doit pouvoir gérer les camions (caractéristiques techniques, type, capacité, état...).
- **Proposition intelligente :** Le système doit pouvoir suggérer automatiquement un camion adapté selon les caractéristiques du voyage.
- **Suivi GPS :** Il doit être possible de géolocaliser les camions en temps réel sur une carte interactive.

3.2.2.5 Gestion des Chauffeurs

- **CRUD des chauffeurs :** Le système permet la gestion complète des chauffeurs, avec leurs affectations et jours de repos.
- **Disponibilité :** Les utilisateurs peuvent suivre et mettre à jour l'état des chauffeurs (actif, en repos, non disponible).
- **Affectation intelligente :** Le système doit proposer automatiquement un chauffeur selon sa disponibilité et son historique d'affectation.

3.2.2.6 Gestion des Clients et Articles

- **Gestion des clients :** L'administrateur doit pouvoir gérer les informations des clients (coordonnées, historique, etc.).
- **Gestion des articles :** Le système permet de gérer les articles transportés et de les associer aux ordres de transport.

3.2.2.7 Outils de Consultation et d'Analyse

- **Planning journalier :** Un tableau de planification permet de visualiser les voyages, chauffeurs et camions avec des filtres et options de tri.
- **Tableau de bord analytique :** Le système propose un tableau de bord avec des indicateurs de performance (KPIs) interactifs (nombre de voyages, taux de remplissage, etc.).

3.2.2.8 Interaction intelligente via Chatbot

- **Assistance contextuelle** : Les utilisateurs peuvent interroger un chatbot intégré pour obtenir rapidement des informations sur les voyages, chauffeurs ou camions.

3.2.2.9 Déploiement, Automatisation et DevOps

- **Dockerisation** : Toute l'application doit être dockerisée pour simplifier le déploiement et assurer la portabilité.
- **Orchestration** : L'ensemble des services doit être orchestré via Docker Compose.
- **CI/CD** : Une pipeline Jenkins doit être mise en place pour automatiser les tests, le déploiement et l'envoi des images Docker sur Docker Hub.

3.2.3 Définition des besoins non-fonctionnels

Ces besoins garantissent la qualité du système au-delà de ses fonctions de base :

- **Sécurité** : Authentification via Spring Security + JWT, contrôle d'accès par rôle.
- **Performance** : Faible latence dans l'exécution des requêtes, optimisation du backend.
- **Évolutivité** : Architecture microservices permettant l'ajout ou la modification facile de modules.
- **Portabilité** : Conteneurisation via Docker pour faciliter le déploiement sur différents environnements.
- **Utilisabilité** : Interface responsive Angular, UX adaptée aux usages quotidiens en entreprise.

3.3 Diagramme de cas d'utilisation globale

Le diagramme suivant illustre les principales interactions entre les acteurs et le système TMS, mettant en évidence les différents cas d'utilisation.

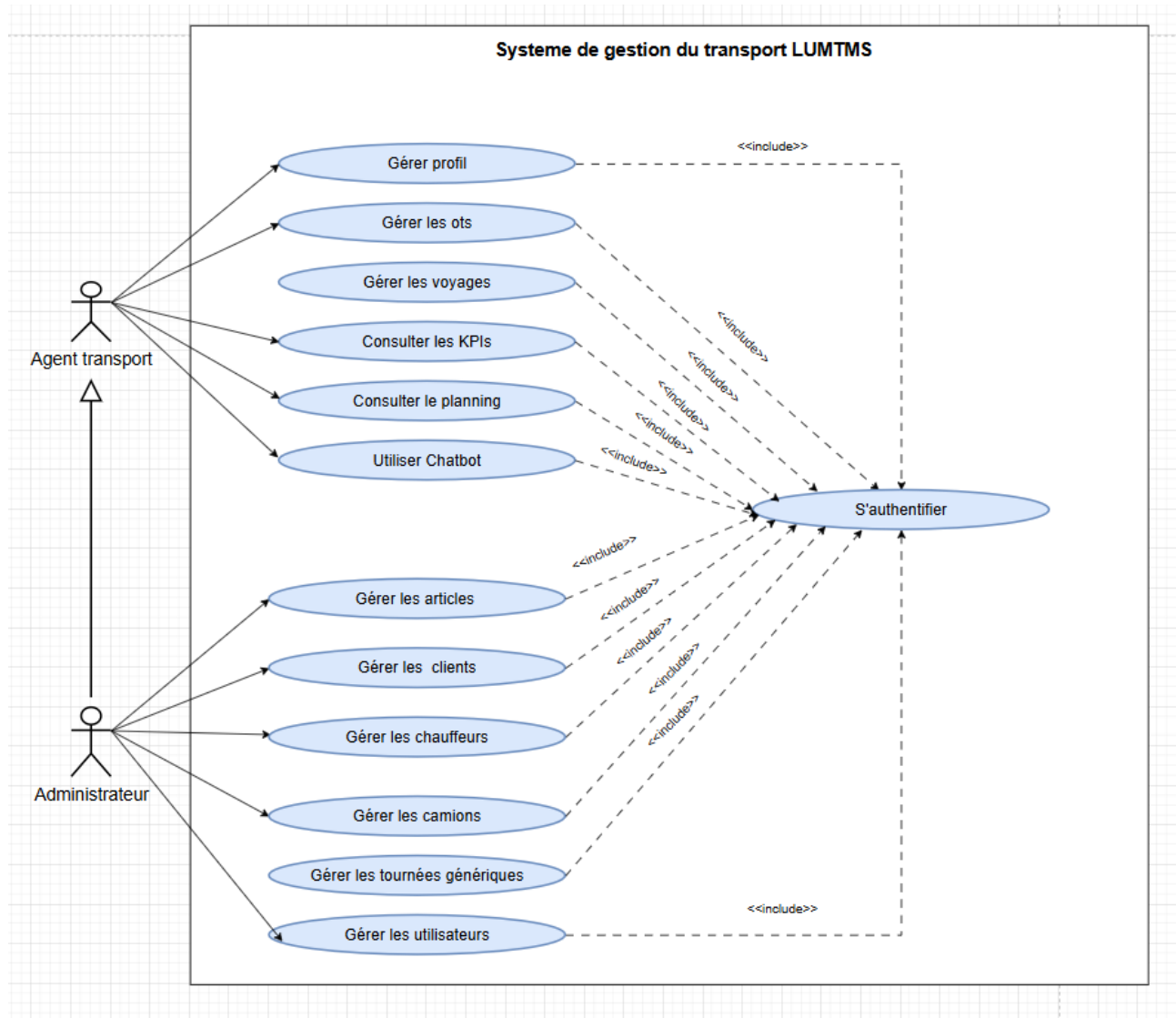


FIGURE 3.1 – Diagramme de cas d'utilisation globale du TMS.

3.4 Product Backlog

Le Product Backlog est une liste hiérarchisée de toutes les fonctionnalités à implémenter dans le système. Il est structuré autour des User Stories, chacune représentant un besoin exprimé par les utilisateurs. Les User Stories sont priorisées en fonction de leur criticité et de leur impact métier.

TABLE 3.1: Product Backlog du projet LUMTMS

Fonctionnalité	ID	Description	Priorité
Authentification et gestion des utilisateurs	US-1	En tant qu'utilisateur, je souhaite m'authentifier et consulter mon profil.	Haute

Product Backlog du projet LUMTMS (suite)

Fonctionnalité	ID	Description	Priorité
Authentification et gestion des utilisateurs	US-2	En tant qu'Administrateur, je souhaite gérer les utilisateurs (CRUD) et consulter un journal des actions.	Haute
Gestion des ordres de transport	US-3	En tant qu'Administrateur/Agent, je souhaite importer/exporter des ordres de transport depuis/vers Geode.	Haute
Gestion des ordres de transport	US-4	En tant qu'Administrateur/Agent, je souhaite gérer les ordres de transport (CRUD).	Haute
Gestion des voyages	US-5	En tant qu'Administrateur/Agent, je souhaite regrouper et planifier les ordres du transport pour créer des voyages.	Moyenne
Gestion des voyages	US-6	En tant qu'Administrateur/Agent, je souhaite créer automatiquement des voyages depuis des tournées génériques.	Moyenne
Gestion des voyages	US-7	En tant qu'Administrateur/Agent, je souhaite calculer des itinéraires optimisés via OpenRoute-Service lors du création du voyage.	Moyenne
Gestion des voyages	US-8	En tant qu'Administrateur/Agent, je souhaite visualiser les trajets sur une carte et générer des rapports PDF.	Moyenne
Gestion des voyages	US-9	En tant qu'Administrateur/Agent, je souhaite que le système doit envoyer automatiquement des SMS au chauffeur lors de la création d'un voyage.	Moyenne
Gestion des voyages	US-10	En tant qu'Administrateur/Agent, je souhaite déplanifier les voyages.	Moyenne
Gestion des camions	US-11	En tant qu'Administrateur, je souhaite gérer les camions (CRUD).	Haute
Gestion des camions	US-12	En tant qu'Administrateur/Agent, je souhaite consulter et modifier l'état des camions.	Haute
Gestion des camions	US-13	Le système doit proposer automatiquement des camions adaptés pour un voyage.	Haute
Gestion des camions	US-14	En tant qu'Administrateur/Agent, je souhaite visualiser la position des camions sur une carte.	Moyenne
Gestion des chauffeurs	US-15	En tant qu'Administrateur, je souhaite gérer les chauffeurs (CRUD).	Haute
Gestion des chauffeurs	US-16	En tant qu'Administrateur/Agent, je souhaite consulter et modifier l'état des chauffeurs.	Haute

Product Backlog du projet LUMTMS (suite)

Fonctionnalité	ID	Description	Priorité
Gestion des chauffeurs	US-17	Le système doit proposer automatiquement des chauffeurs adaptés pour un voyage.	Haute
Gestion des clients	US-18	En tant qu'Administrateur, je souhaite gérer les clients (CRUD).	Haute
Gestion des clients	US-19	En tant qu'Administrateur/Agent, je souhaite associer un client à un ordre de transport.	Haute
Gestion des articles	US-20	En tant qu'Administrateur, je souhaite gérer les articles (CRUD).	Haute
Gestion des articles	US-21	En tant qu'Administrateur/Agent, je souhaite associer un article à un ordre de transport.	Haute
Gestion des tournées génériques	US-22	En tant qu'Administrateur, je souhaite gérer les tournées génériques (CRUD).	Haute
Outils de consultation et d'analyse	US-23	En tant qu'Administrateur/Agent, je souhaite consulter un tableau de planification journalier.	Moyenne
Outils de consultation et d'analyse	US-24	En tant qu'Administrateur/Agent, je souhaite consulter un tableau de bord avec graphiques interactifs.	Moyenne
Chatbot	US-25	En tant qu'Administrateur/Agent, je souhaite interagir avec un chatbot pour obtenir de l'aide.	Moyenne
Déploiement	US-26	En tant que développeur, je souhaite dockeriser l'application.	Basse
Déploiement	US-27	En tant que développeur, je souhaite déployer les services via Docker Compose.	Basse
Déploiement	US-28	En tant que développeur, je souhaite héberger le système sur un serveur distant.	Basse
Automatisation	US-29	En tant que développeur, je souhaite mettre en place une pipeline CI/CD avec Jenkins.	Basse

3.5 Planification des itérations – Répartition en sprints

Le diagramme de Gantt présenté ci-dessous illustre la planification des sprints tout au long de la période de stage, s'étendant du 2 février au 1 août. Chaque sprint est défini selon une durée précise, variant entre deux et quatre semaines, en fonction des objectifs fixés et de la charge de travail estimée. Cette planification permet de structurer le déroulement du projet en différentes phases successives, allant de l'étude préliminaire et l'analyse des besoins à la mise en production et l'intégration continue (CI/CD). Ainsi, le diagramme offre une vue claire et chronologique de l'avancement du projet, garantissant un suivi rigoureux et une meilleure gestion du temps.

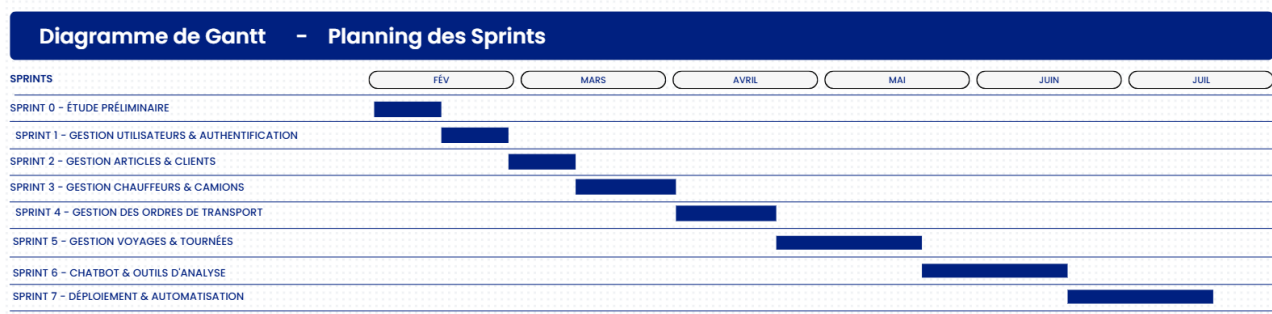


FIGURE 3.2 – Diagramme de Gantt - Planification des Sprints

3.6 Conception architecturale

Dans cette section, nous présentons l'architecture de l'application LUMTMS pour fournir une vue complète de ses composants fonctionnels. L'architecture générale adopte un modèle à trois niveaux[16], comprenant trois couches principales : le front-end, le back-end sous forme de microservices, et les bases de données, comme illustré dans la figure ci-dessous.

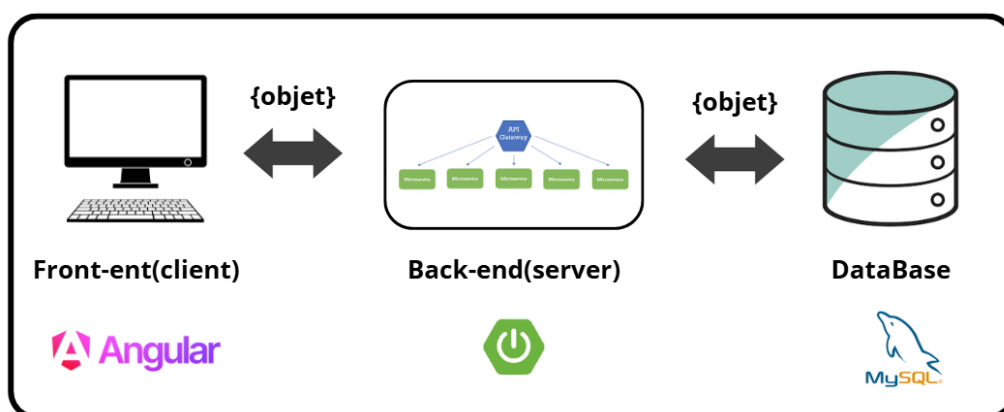


FIGURE 3.3 – Architecture générale à trois niveaux

L'architecture de l'application LUMTMS repose sur une organisation en trois couches principales afin d'assurer modularité, maintenabilité et évolutivité.

- Le front-end (client) : Conçu avec Angular, il gère l'interface utilisateur et permet aux usagers d'interagir avec le système via des formulaires, tableaux de bord et cartes. Cette couche se charge de l'envoi des requêtes vers les services du back-end et de l'affichage des réponses.
- Le back-end (serveur) : Développé sous forme de microservices, il centralise la logique métier de l'application. Chaque service est dédié à une fonctionnalité spécifique (gestion des utilisateurs, clients, chauffeurs, camions, articles, etc.), ce qui favorise la scalabilité et facilite le déploiement.

indépendant de chaque composant.

- La couche base de données : Elle permet le stockage persistant des informations nécessaires au fonctionnement du système, telles que les données des utilisateurs, commandes, véhicules ou itinéraires.

La communication entre ces couches se fait de manière synchrone, garantissant un échange structuré et direct entre les composants de l'application.

3.6.1 Conception et implémentation de l'architecture microservices

L'architecture microservices [17] repose sur une approche où l'application est décomposée en plusieurs services autonomes, chacun ayant une responsabilité métier spécifique. Cette conception permet une meilleure scalabilité, modularité, maintenabilité et facilité de déploiement indépendant de chaque composant.

Le projet LUMTMS est construit à l'aide de Spring Boot pour le back-end, organisé en microservices, et d'un frontend Angular pour la couche client.

Les microservices sont les suivants :

- **API Gateway** : Point d'entrée unique du système, il gère le routage des requêtes vers les microservices concernés, applique les politiques de sécurité (authentification, autorisation), journalisation, et limitation de débit.
- **Discovery Service (Eureka)** : Service de découverte qui permet aux microservices de s'enregistrer dynamiquement et de se découvrir mutuellement sans avoir besoin de configurer les adresses manuellement.
- **User Service** : Gère les opérations liées aux utilisateurs : inscription, authentification, autorisation, gestion des rôles et profils.
- **Client Service** : Responsable de la gestion des clients de l'application.
- **Article Service** : Gère les articles et leur classification pour les expéditions.
- **Chauffeur Service** : Service dédié à la gestion des chauffeurs, ainsi qu'au suivi GPS des camions.
- **Camion Service** : Gère le parc de camions (immatriculation, types, capacité, état, etc.).
- **OT-Voyage Service** : Gère les Ordres de Transport (OT), les voyages et les tournées génériques. Ce service communique avec l'API OpenRouteService pour récupérer les coordonnées géographiques (longitude et latitude) des destinations et optimiser les itinéraires. Ce microservice intègre également un chatbot intelligent qui exécute un script Python. Ce script utilise LangChain, Gemini (via l'API Google) et une connexion SQL pour interroger dynamiquement la base de données.

Chaque microservice dispose de sa propre base de données indépendante (MySQL) et les communications entre services s'effectuent via des requêtes HTTP RESTful.

Cette architecture microservices permet :

- Une évolutivité horizontale, chaque service pouvant être répliqué et déployé indépendamment.
- Une meilleure résilience, car une panne dans un service n’impacte pas l’ensemble du système.
- Une gestion granulaire des permissions, de la sécurité, et des ressources système.
- Un déploiement continu facilité par l’indépendance des composants.

3.6.2 Architecture logique

L’architecture logique du projet LUMTMS décrit les composants fonctionnels et leurs interactions. Elle est organisée en plusieurs couches logiques :

- **Client** : Interface utilisateur Angular, qui communique avec le backend via des appels HTTP. Il utilise également OpenStreetMap pour l’affichage cartographique des trajets.
- **API Gateway** : Sert de couche de médiation, assurant le routage, l’authentification, l’équilibrage de charge et la sécurité.
- **Microservices principaux** :
 - Auth Service : Gestion de l’authentification via JWT et gestion des sessions.
 - User Service : Gestion des profils utilisateurs et de leurs permissions.
 - OT/Voyage Service : Gestion des ordres de transport et des trajets.
 - Chauffeur Service : Gestion des chauffeurs.
 - Camion Service : Gestion des véhicules.
 - Client Service : Gestion des clients et articles.
- **Services transversaux** :
 - Discovery Service (Eureka) : Pour la localisation automatique des microservices.
- **Base de données** : Chaque microservice possède sa propre base MySQL, assurant l’isolation des données.

Les interactions se font principalement en REST synchrone, avec des découplages forts grâce à l’indépendance des services et à l’usage d’un service de découverte dynamique.

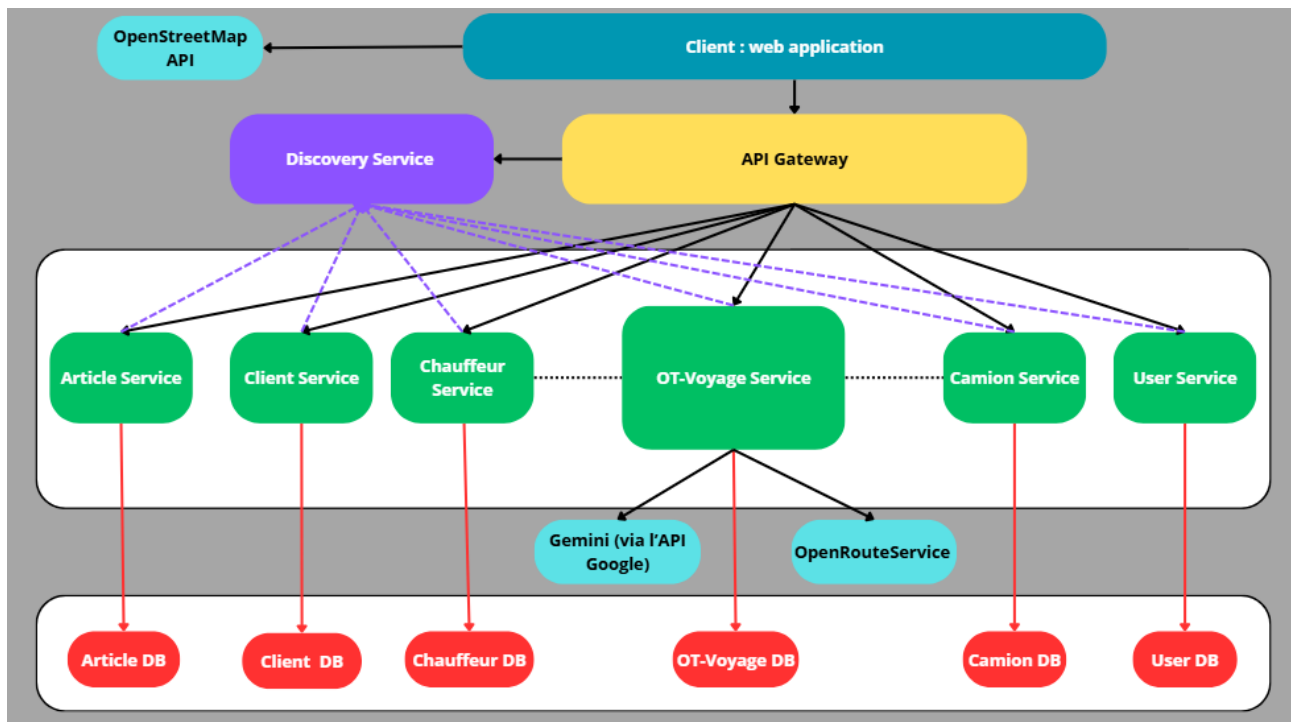


FIGURE 3.4 – Architecture logique de l’application LUMTMS

3.6.3 Architecture physique

L’architecture physique fait référence à l’infrastructure réelle qui supporte le système logiciel. Le projet LUMTMS est basé sur des technologies conteneurisées, principalement Docker, et comprend les éléments suivants :

- **Frontend Angular (port 4200)** : Déployé sur un serveur accessible via un navigateur, il consomme les APIs REST exposées par les microservices et intègre OpenStreetMap pour l’affichage dynamique des trajets.
- **API Gateway (port 8082)** : Sert de point d’entrée unique pour toutes les requêtes frontend. Il redirige les appels vers les microservices appropriés.
- **Eureka Server (port 8761)** : Composant central du système pour la découverte dynamique des services.
- **Microservices Spring Boot (communication synchrone)** : Chaque service est conteneurisé indépendamment et expose ses fonctionnalités via des APIs REST sur un port dédié :
 - User Service – port 8081
 - Client Service – port 8021
 - Article Service – port 8020
 - Chauffeur Service – port 8023

- Camion Service – port 8024
- OT-Voyage Service – port 8025
- **OpenRouteService** : Intégré via appel d’API dans le service OT-Voyage (8025), ce module permet la génération automatique d’itinéraires optimisés.
- **Chatbot Python** : Hébergé également dans le service OT-Voyage, ce composant utilise LangChain et l’API Gemini pour répondre dynamiquement à des requêtes SQL.
- **Bases de données MySQL (port 3306)** : Chaque microservice dispose de sa propre instance ou schéma MySQL, garantissant une séparation stricte des données.

Cette architecture assure :

- Une haute disponibilité, chaque composant pouvant être répliqué.
- Une scalabilité horizontale, les microservices étant extensibles indépendamment.
- Un déploiement isolé, permettant des mises à jour sans interruption du système global.
- Une résilience accrue, limitant l’impact des pannes à un seul service.

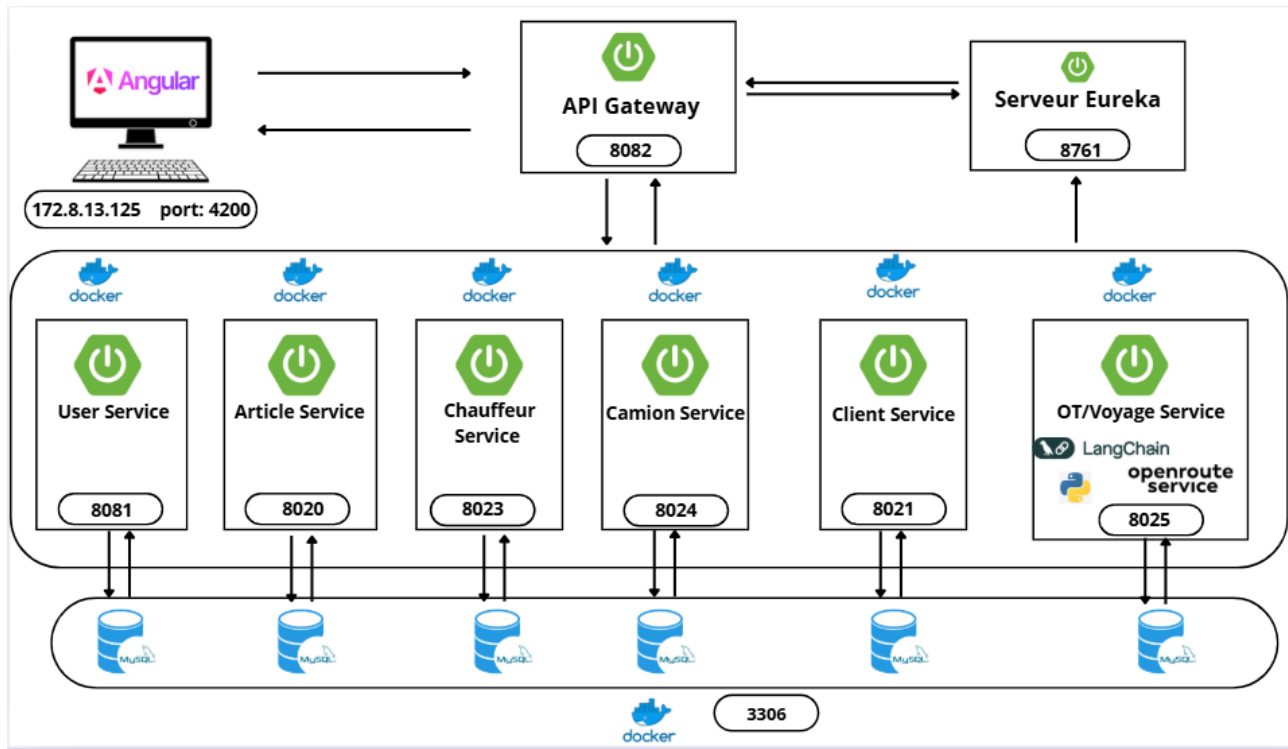


FIGURE 3.5 – Architecture physique de l’application LUMTMS

3.7 Choix et exigences technologiques

Dans cette partie, nous présentons les choix technologiques retenus pour la réalisation du système LUMTMS. Ces choix couvrent à la fois l’environnement matériel, les logiciels et outils de dévelop-

pement, les langages de programmation ainsi que les solutions de déploiement et d'hébergement. Ils ont été sélectionnés afin d'assurer une architecture robuste, performante et facilement maintenable, tout en facilitant le développement des différentes fonctionnalités liées à la gestion des transports et à la cartographie.

3.7.1 Environnement matériel

Mon poste de travail est équipé d'un PC HP Victus avec les caractéristiques suivantes :

- Processeur Intel Core i5 de 13ème génération
- Mémoire vive (RAM) : 32 Go
- Stockage : 512 Go SSD
- Système d'exploitation : Windows 10

3.7.2 Environnement logiciel

L'environnement logiciel du projet a été soigneusement sélectionné pour répondre aux exigences d'une application web moderne, distribuée et intelligente. Il s'articule autour de frameworks robustes pour le client et le serveur, d'outils de développement performants et de technologies de pointe pour l'intégration de l'IA et la visualisation de données.

3.7.2.1 Logiciels de développement

Les logiciels suivants constituent le cœur de la stack technique pour le développement des fonctionnalités de l'application.

Logiciel	Définition
Angular	Framework JavaScript/TypeScript pour développer des applications web dynamiques côté client [18].
Spring Boot	Framework Java pour construire des applications backend robustes et performantes [19].
MySQL Workbench	Outil graphique pour la conception, l'administration et la gestion de bases de données MySQL [20].
OpenRouteService	API et service pour la gestion et le calcul d'itinéraires et de géolocalisation [21].
OpenStreetMap	Projet collaboratif fournissant des cartes géographiques libres et modifiables [22].

Logiciel	Définition
LangChain	Framework facilitant la création d'applications basées sur les modèles de langage [23].
Gemini	API d'intelligence artificielle avancée pour le traitement du langage naturel [24].
Chart.js	Bibliothèque JavaScript pour la création de graphiques interactifs [25].
JasperReports	Outil de génération de rapports et documents PDF à partir de données [26].

3.7.2.2 Outils de développement

Ces outils ont été essentiels pour optimiser le cycle de développement, de la conception à la collaboration et aux tests.

Outil	Définition
IntelliJ IDEA	Environnement de développement intégré (IDE) performant pour Java et d'autres langages [27].
Visual Studio Code	Éditeur de code source léger, extensible, utilisé pour le développement web et multi-langages [28].
Postman	Application pour tester et documenter des APIs REST [29].
GitHub	Plateforme de gestion et de partage de code avec contrôle de version Git [30].
draw.io	Outil en ligne pour la création de diagrammes et schémas techniques [31].
LaTeX	Système de composition de documents scientifique et technique de haute qualité [32].

3.7.2.3 Langages de programmation

Le choix des langages a été guidé par leur adéquation avec les frameworks sélectionnés et les besoins spécifiques de chaque couche de l'application.

Langage	Définition
Java	Langage orienté objet principalement utilisé pour le développement backend et les applications d'entreprise [33].
TypeScript	Surensemble typé de JavaScript, utilisé surtout pour le développement frontend avec Angular [34].

Langage	Définition
HTML	Langage de balisage permettant de structurer le contenu des pages web [35].
CSS	Langage de style permettant de définir la présentation et la mise en forme des pages web [36].
Python	Langage polyvalent, souvent utilisé pour les scripts, l'automatisation et l'intégration d'IA [37].

3.7.2.4 Outils de déploiement et d'hébergement

Pour garantir un déploiement fiable, automatisé et scalable, les outils suivants ont été adoptés.

Outil	Définition
Docker	Plateforme de conteneurisation pour automatiser le déploiement des applications dans des environnements isolés [38].
Jenkins	Serveur d'intégration continue et de livraison continue (CI/CD) automatisant les builds et déploiements [39].
Nginx	Serveur web léger utilisé comme proxy inverse, load balancer, et serveur d'hébergement d'applications web [40].

3.8 Conclusion

L'étude préliminaire, a été une étape fondamentale pour jeter les bases solides du projet LUMTMS. Au cours de cette phase, une analyse approfondie des besoins a permis d'identifier les acteurs clés et de définir avec précision les exigences fonctionnelles et non fonctionnelles.

Le Product Backlog qui en résulte constitue une feuille de route claire et priorisée pour le développement futur. Parallèlement, la conception d'une architecture à trois niveaux, orientée microservices, et la sélection rigoureuse des technologies et outils de développement garantissent que le projet repose sur des fondations techniques modernes, robustes et évolutives.

Enfin, la planification initiale des itérations fournit un cadre structuré pour les sprints de développement à venir. En somme, ce Sprint 0 a permis de transformer une vision initiale en un plan de projet concret et réalisable, prêt à être mis en œuvre dans les phases de développement successives.

Chapitre 4

Sprint 1 – Gestion des Utilisateurs et Profils, Authentification

4.1 Introduction

Ce premier sprint, d’une durée de deux semaines, marque le lancement du développement de l’application LUMTMS. Il se concentre sur les aspects fondamentaux liés à la gestion des utilisateurs et à l’authentification. Durant cette période, les fonctionnalités de sécurité, de gestion des profils, ainsi que les premières interfaces utilisateur ont été mises en place, accompagnées de tests unitaires pour assurer la qualité du code.

4.2 Objectifs du Sprint

Les principaux objectifs de ce sprint sont :

- Mettre en œuvre l’authentification des utilisateurs via Spring Security et JWT.
- Créer les composants Angular nécessaires pour l’inscription, la connexion et l’affichage du profil.
- Permettre à un administrateur de gérer les utilisateurs (CRUD).
- Concevoir les interfaces frontend pour chaque fonctionnalité.
- Écrire et exécuter des tests unitaires pour garantir la robustesse du code dès le départ.

4.3 Spécification et Analyse des Besoins

Cette section détaille les exigences fonctionnelles et les scénarios d'utilisation qui ont guidé le développement durant ce sprint. Le Sprint Backlog ci-dessous décompose les User Stories en tâches techniques concrètes, tandis que le diagramme de cas d'utilisation illustre les interactions clés des acteurs avec le système.

4.3.1 Sprint Backlog

Le tableau suivant présente le Sprint Backlog du sprint 1, détaillant les fonctionnalités, les scénarios utilisateurs, les tâches associées ainsi que leurs estimations en termes de charge de travail.

TABLE 4.1: Sprint Backlog du Sprint 1

Fonctionnalité	ID	Scénario Utilisateur	ID Tâche	Tâches	Estimation
Authentification et gestion des utilisateurs	US-1	En tant qu'utilisateur (Admin ou Agent), je souhaite m'authentifier pour accéder à mon espace personnel.	T1	Développement backend	3 jours
			T2	Création des interfaces de connexion et de profil	2 jours
			T3	Écriture des tests unitaires pour les endpoints d'authentification	1 jour
Gestion des utilisateurs	US-2	tant qu'administrateur, je souhaite gérer les utilisateurs (ajouter, modifier, supprimer) .	T4	Développement backend (CRUD utilisateurs, logs d'actions)	3 jours
			T5	Création des interfaces de gestion des utilisateurs (liste, formulaire, etc.)	2 jours

Suite à la page suivante

Table 4.1 – suite de la page précédente

Fonctionnalité	ID	Scénario Utilisateur	ID Tâche	Tâches	Estimation
			T6	Écriture des tests unitaires pour les opérations CRUD	1 jour

4.3.2 Diagramme de cas d'utilisation du Sprint

Le diagramme de cas d'utilisation ci-dessous illustre les interactions principales entre les utilisateurs (administrateur, agent) et le système au cours de ce sprint :

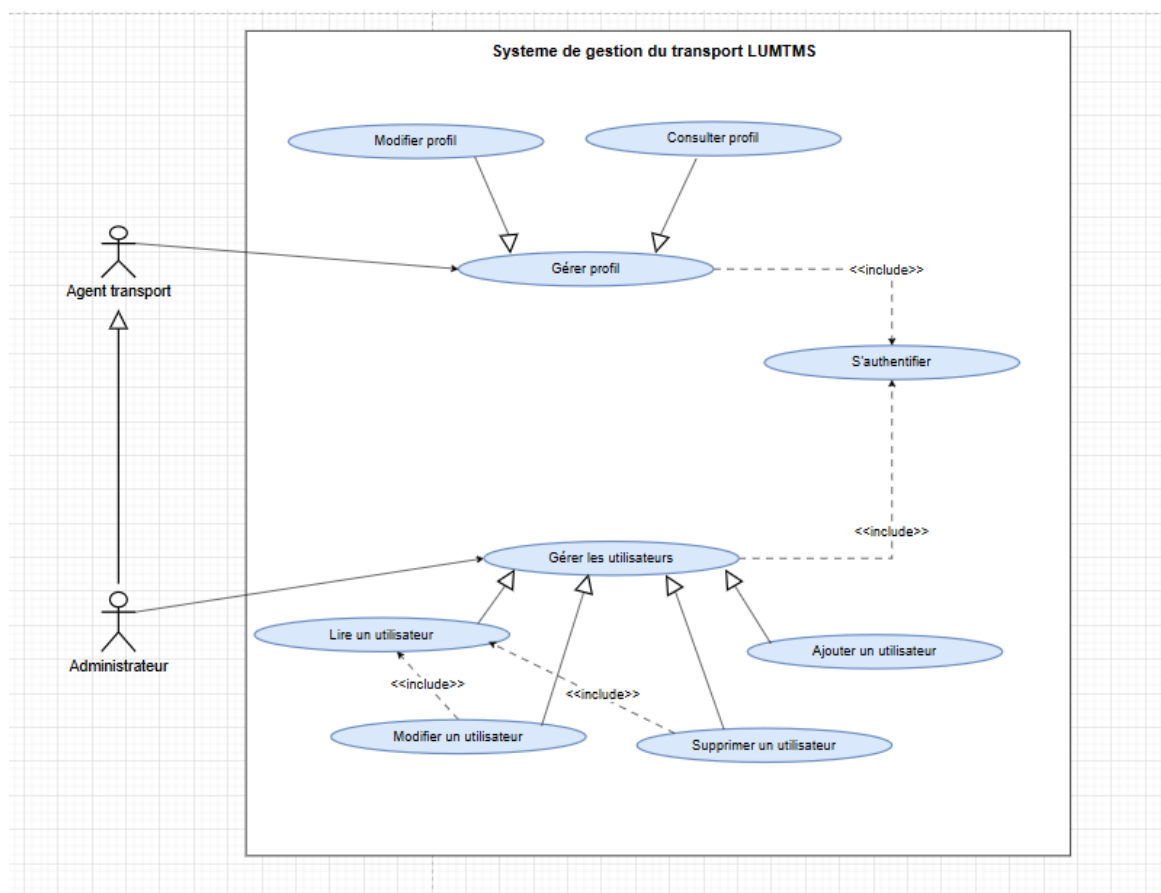


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint 1

4.4 Description textuelle des cas d'utilisation

4.4.1 Description textuelle du cas d'utilisation "S'authentifier"

Ce cas d'utilisation décrit le processus par lequel un utilisateur accède au système de manière sécurisée. Il couvre la validation des identifiants et la gestion des accès via un token JWT.

TABLE 4.2: Description textuelle du cas d'utilisation "S'authentifier"

Titre	S'authentifier
Acteur	Utilisateur (Administrateur ou Agent de Transport)
Précondition	L'utilisateur possède un compte valide dans le système.
Postcondition	L'utilisateur est authentifié et reçoit un token JWT pour accéder au système.
Scénario nominal	1. L'utilisateur accède à la page de connexion. 2. Le système affiche le formulaire de connexion (email, mot de passe). 3. L'utilisateur saisit ses identifiants et soumet le formulaire. 4. Le système transmet la requête d'authentification à l'API Gateway. 5. L'API Gateway transmet la requête au service d'authentification. 6. Le service d'authentification vérifie les identifiants auprès de la base de données. 7. Si les identifiants sont valides, le service génère un token JWT. 8. Le token JWT est retourné à l'utilisateur. 9. L'utilisateur est redirigé vers la page principale avec le token stocké.

Scénarios alternatifs	— A1 : Identifiants invalides : 1. Le système affiche un message d’erreur indiquant que les identifiants sont incorrects. 2. Retour à l’étape 2. — A2 : Erreur système : 1. Le système informe l’utilisateur qu’une erreur est survenue. 2. Retour à l’étape 1.
------------------------------	---

4.4.2 Description textuelle du cas d’utilisation "Modifier profil"

Ce cas d’utilisation permet à un utilisateur authentifié de mettre à jour ses propres informations de profil. Le système garantit l’unicité de l’adresse e-mail pour maintenir la cohérence des données.

TABLE 4.3: Description textuelle du cas d’utilisation “Modifier profil”

Titre	Modifier son profil
Acteur	Utilisateur (Administrateur ou Agent de Transport)
Précondition	L’utilisateur est authentifié et se trouve sur la page de son profil.
Postcondition	Les informations du profil de l’utilisateur sont mises à jour dans la base de données.
Scénario nominal	1. L’utilisateur accède à la page de son profil. 2. Le système affiche les informations actuelles de l’utilisateur. 3. L’utilisateur modifie les champs souhaités et soumet le formulaire. 4. Le système envoie une requête PUT à l’API Gateway avec les nouvelles données. 5. L’API Gateway transmet la requête au service utilisateur. 6. Le service vérifie si l’email (s’il a été modifié) est unique. 7. Le service met à jour les informations dans la base de données. 8. Le système affiche un message de succès.

Scénarios alternatifs	— A1 : Email déjà utilisé : 1. Le système affiche un message d’erreur indiquant que l’email est déjà pris. 2. Retour à l’étape 3. — A2 : Données invalides : 1. Le système affiche un message d’erreur de validation. 2. Retour à l’étape 3.
------------------------------	--

4.4.3 Description textuelle du cas d’utilisation ”Ajouter un compte utilisateur”

Ce cas d’utilisation, réservé à l’administrateur, détaille la procédure de création d’un nouveau compte utilisateur. Il inclut la validation des données et l’assignation d’un rôle.

TABLE 4.4: Description textuelle du cas d’utilisation “Ajouter un compte utilisateur”

Titre	Ajouter un compte utilisateur
Acteur	Administrateur
Précondition	L’administrateur est authentifié et dispose des droits nécessaires.
Postcondition	Un nouveau compte utilisateur est créé et enregistré dans le système.

Scénario nominal	1. L'administrateur accède à la page de gestion des utilisateurs. 2. Il clique sur le bouton "Ajouter un utilisateur". 3. Le système affiche un formulaire de création (nom, email, rôle, etc.). 4. L'administrateur remplit les informations et soumet le formulaire. 5. Le système envoie une requête POST à l'API Gateway. 6. L'API Gateway transmet la requête au service utilisateur. 7. Le service valide les données et vérifie l'unicité de l'email. 8. Le service enregistre le nouvel utilisateur dans la base de données. 9. Le système affiche un message de succès et met à jour la liste des utilisateurs.
Scénarios alternatifs	— A1 : Email déjà existant : 1. Le système affiche un message d'erreur indiquant que l'email est déjà utilisé. 2. Retour à l'étape 3. — A2 : Données de formulaire invalides : 1. Le système affiche un ou plusieurs messages d'erreur de validation. 2. Retour à l'étape 3.

4.4.4 Gestion des autres actions liées aux comptes utilisateurs

Les opérations de modification et de suppression d'un compte utilisateur suivent une démarche comparable à celle de l'ajout d'un compte. Elles se déroulent selon les étapes suivantes :

- Accès à l'interface de gestion des utilisateurs,
- Sélection du compte concerné (à modifier ou à supprimer),
- Validation des modifications apportées ou confirmation de la suppression.

4.5 Conception

4.5.1 Diagramme de classes

Le diagramme de classes ci-dessous modélise la structure statique du système, en présentant les classes principales, leurs attributs, méthodes et les relations qui les unissent.

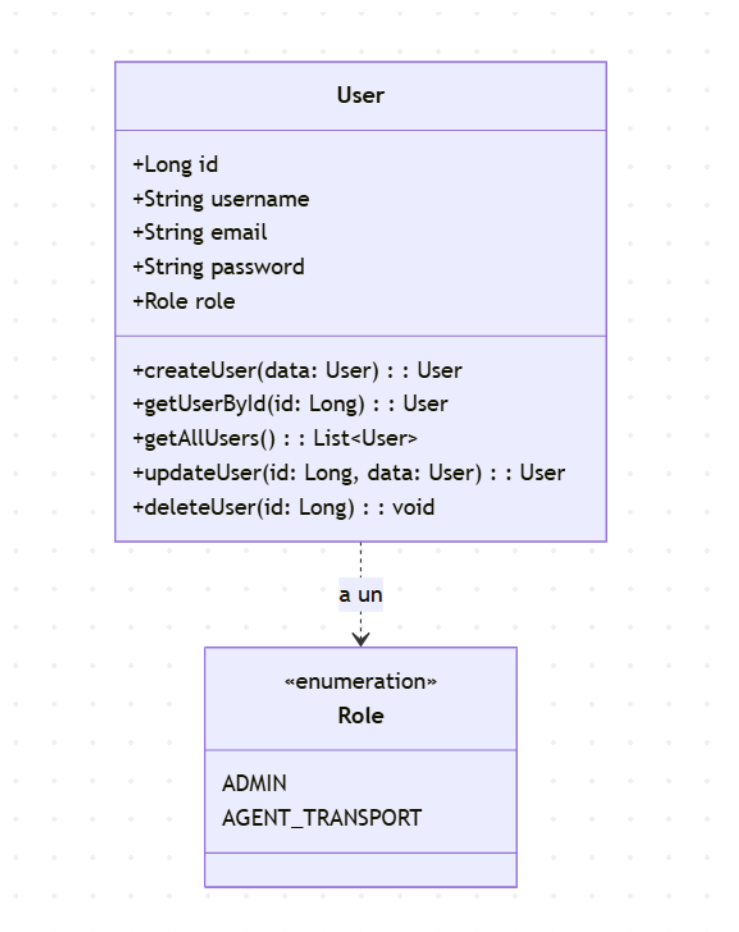


FIGURE 4.2 – Diagramme de classes du Sprint 1

4.5.2 Diagrammes de séquence

Les diagrammes de séquence suivants illustrent les interactions dynamiques entre les objets pour des scénarios spécifiques.

4.5.2.1 Diagramme de séquence "S'authentifier"

Ce diagramme détaille les étapes du processus d'authentification d'un utilisateur.

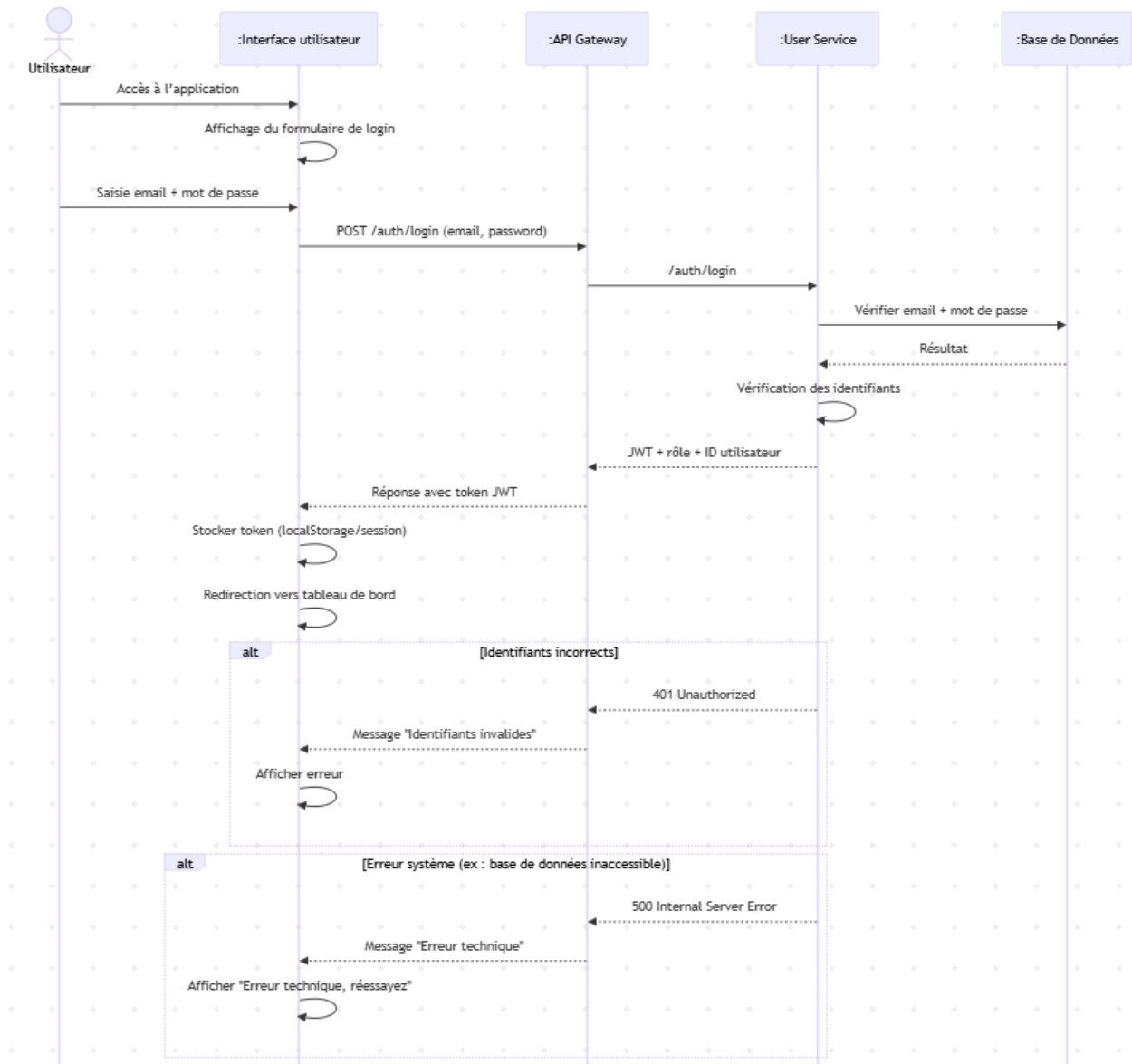


FIGURE 4.3 – Diagramme de séquence - Authentification

4.5.2.2 Diagramme de séquence " modifier profil "

Ce diagramme montre comment un utilisateur peut modifier ses informations de profil.

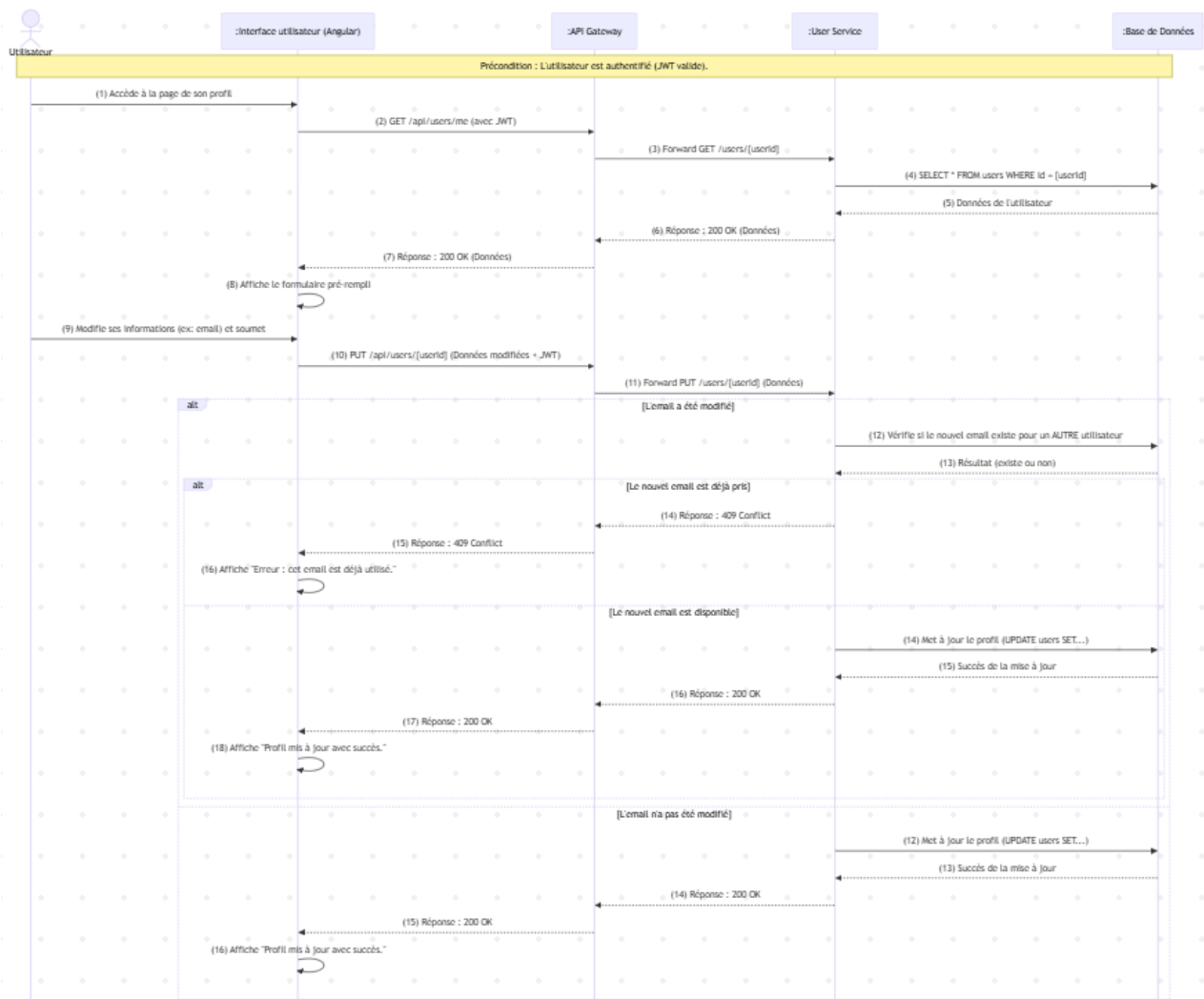


FIGURE 4.4 – Diagramme de séquence -"Modifier profil"

4.5.2.3 Diagramme de séquence "Ajouter un compte utilisateur"

Ce diagramme illustre le processus d'ajout d'un nouvel utilisateur par un administrateur.

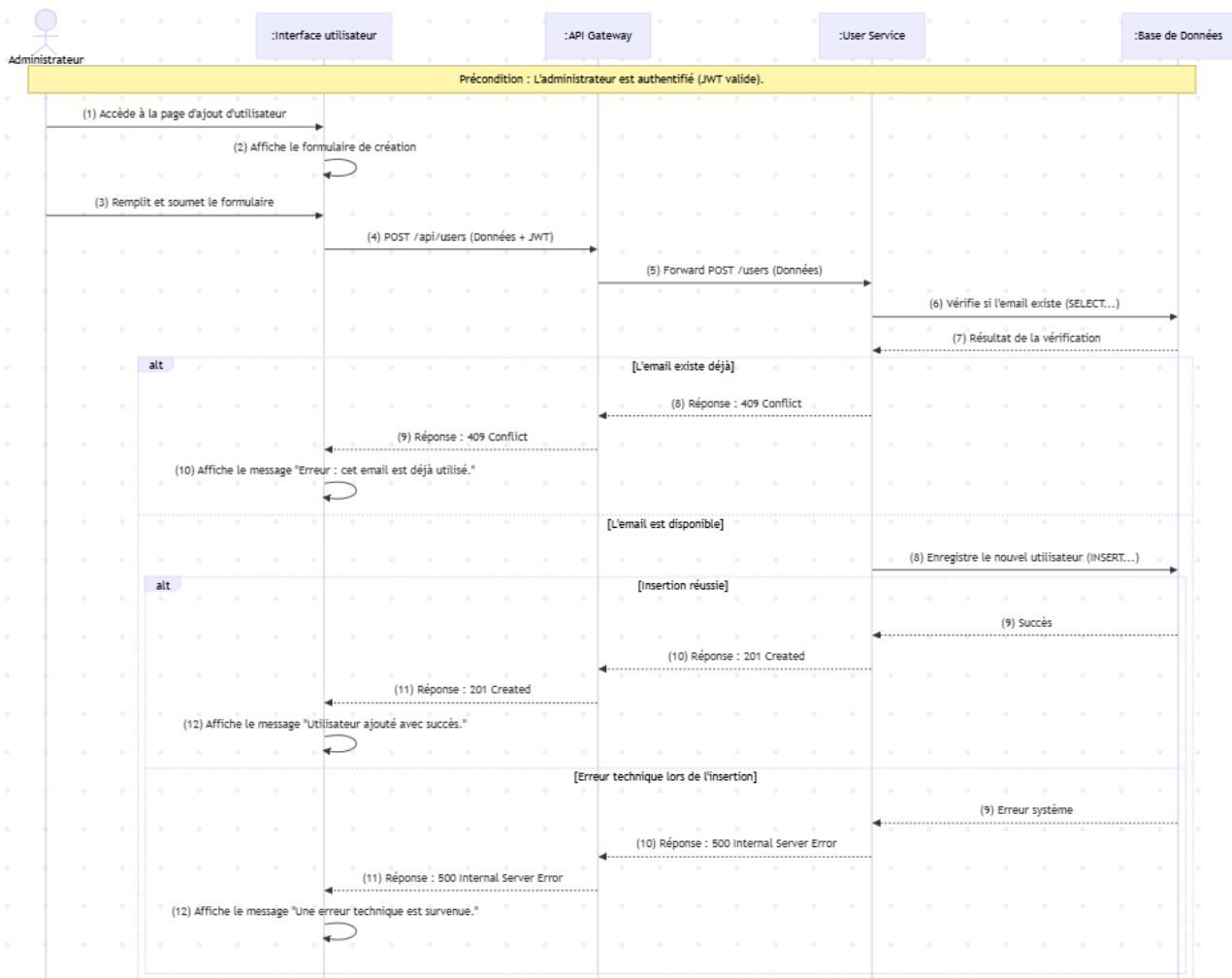


FIGURE 4.5 – Diagramme de séquence - "Ajouter un compte utilisateur"

4.6 Réalisation

4.6.1 Interface d'authentification (login)

L'interface de connexion est la porte d'entrée de l'application. Elle a été conçue pour être simple et intuitive, permettant à l'utilisateur de s'identifier rapidement et en toute sécurité.

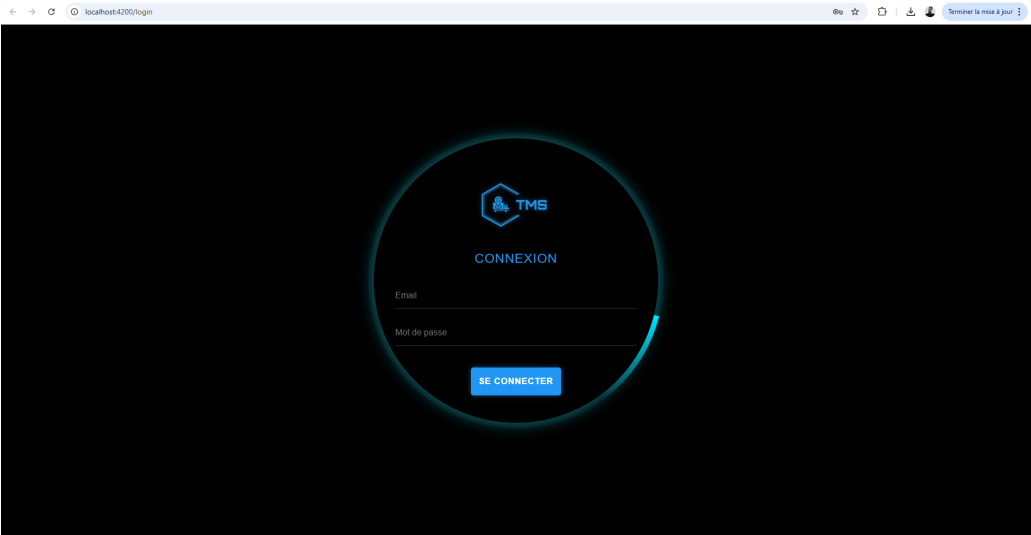


FIGURE 4.6 – Interface de connexion de l’application LUMTMS

4.6.2 Interface de gestion des utilisateurs

L’interface de gestion des utilisateurs, accessible aux administrateurs, offre une vue d’ensemble de tous les comptes. Elle permet d’effectuer les opérations CRUD (créer, lire, mettre à jour, supprimer) de manière centralisée.

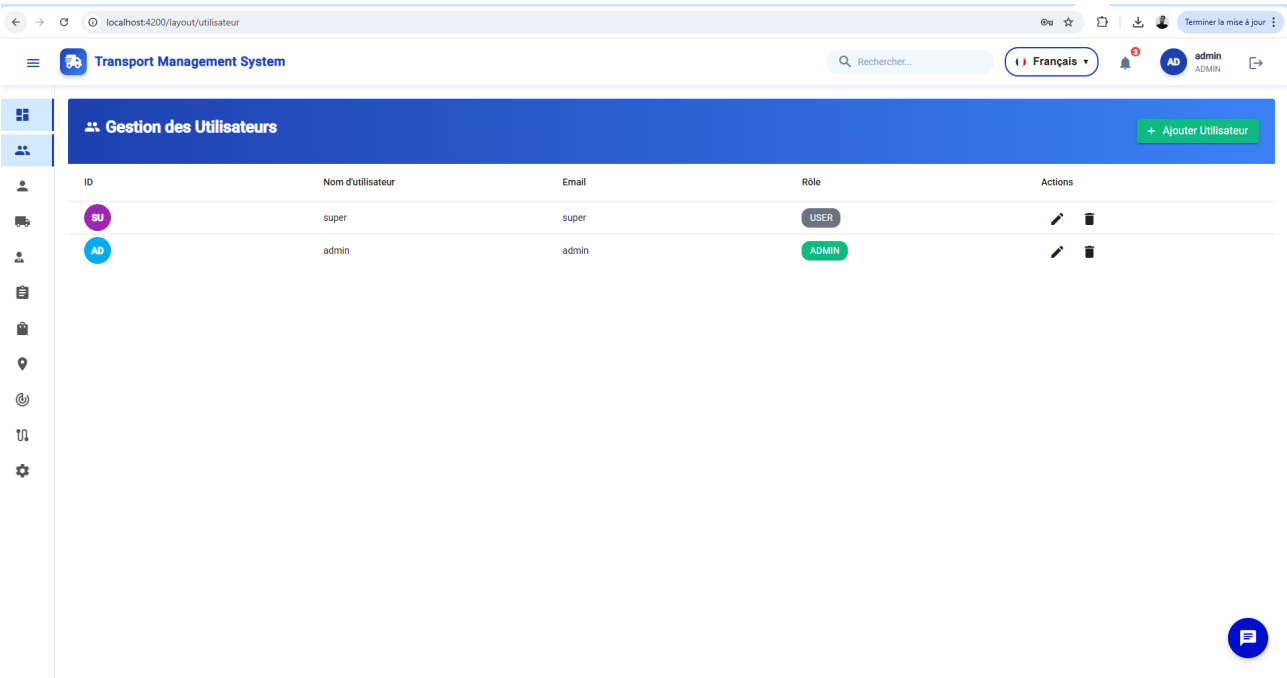


FIGURE 4.7 – Interface de gestion des utilisateurs

4.6.3 Interface du profil utilisateur

Cette interface permet à chaque utilisateur de consulter et de mettre à jour ses informations personnelles. La modification de l'adresse e-mail est soumise à une vérification d'unicité pour garantir l'intégrité des données.

The screenshot displays the 'Transport Management System' user profile interface. The browser address bar shows 'localhost:4200/layout/profil'. The page title is 'Transport Management System'. The user is logged in as 'admin' (ADMIN). The interface is in French ('Français').

Bienvenue, admin

Informations Personnelles

Informations	Valeur
Nom d'utilisateur	admin
Email	admin
Rôle	ADMIN

Modifier le Profil

Nom d'utilisateur*
admin

Email*
admin

Mot de passe
.....
Laissez vide pour ne pas modifier

Annuler Enregistrer

FIGURE 4.8 – Interface du profil utilisateur

4.7 Conclusion

Ce premier sprint a permis de jeter les bases solides de l'application LUMTMS. En se concentrant sur la gestion des utilisateurs et l'authentification, nous avons mis en place le cœur sécuritaire du système. Les objectifs fixés ont été atteints, avec la livraison de fonctionnalités essentielles et la mise en place de tests unitaires garantissant la qualité du code. Ce sprint initial marque une étape cruciale et réussie, ouvrant la voie au développement des fonctionnalités métier plus complexes dans les itérations suivantes.

Chapitre 5

Sprint 2 – Gestion des Articles et des Clients

5.1 Introduction

Le second sprint, d'une durée de deux semaines, est dédié à la mise en place des fonctionnalités de gestion des clients et des articles dans l'application LUMTMS. Ce sprint se concentre sur le développement des opérations CRUD pour les administrateurs, la consultation des données pour les agents, ainsi que la création des interfaces et l'exécution des tests unitaires pour garantir la fiabilité du système.

5.2 Objectifs du Sprint

Les principaux objectifs sont :

- Mettre en place les API backend pour la gestion des clients et des articles.
- Permettre aux administrateurs d'ajouter, modifier, supprimer et consulter les clients et les articles.
- Permettre aux agents de consulter uniquement les informations relatives aux clients et articles.
- Concevoir et intégrer les interfaces frontend correspondantes.
- Assurer la qualité du code avec des tests unitaires sur les fonctionnalités développées.

5.3 Spécification et Analyse des Besoins

5.3.1 Sprint Backlog

Le tableau suivant présente le Sprint Backlog correspondant à ce sprint, en détaillant les fonctionnalités à implémenter, les scénarios utilisateurs associés, les tâches identifiées ainsi que leur estimation de charge de travail.

Fonctionnalité	ID	Scénario Utilisateur	ID Tâche	Tâches	Estimation
Gestion des clients (CRUD)	US-17	En tant qu'Administrateur, je souhaite gérer les clients (ajouter, modifier, supprimer et consulter) pour maintenir les données à jour.	T1	Développement backend CRUD Clients	3 jours
			T2	Création des interfaces de gestion des clients	2 jours
			T3	Écriture et exécution des tests unitaires pour la gestion des clients	1 jour
Consultation des clients	US-18	En tant qu'Agent, je souhaite consulter la liste des clients pour accéder aux informations nécessaires.	T4	Développement backend (endpoint de consultation)	1 jour
			T5	Interface de consultation des clients (Angular)	1 jour
Gestion des articles (CRUD)	US-19	En tant qu'Administrateur, je souhaite gérer les articles (ajouter, modifier, supprimer et consulter) pour maintenir les données à jour.	T6	Développement backend CRUD Articles	3 jours
			T7	Création des interfaces de gestion des articles	2 jours

Suite à la page suivante

Sprint Backlog (suite)

Fonctionnalité	ID	Scénario Utilisateur	ID Tâche	Tâches	Estimation
			T8	Écriture et exécution des tests unitaires pour la gestion des articles	1 jour
Consultation des articles	US-20	En tant qu'Agent, je souhaite consulter la liste des articles pour accéder aux informations nécessaires.	T9	Développement backend (endpoint de consultation)	1 jour
			T10	Interface de consultation des articles	1 jour

5.4 Diagramme de cas d'utilisation du Sprint

Le diagramme de cas d'utilisation suivant illustre les interactions entre les acteurs (Administrateur, Agent) et le système pour la gestion des clients et des articles :

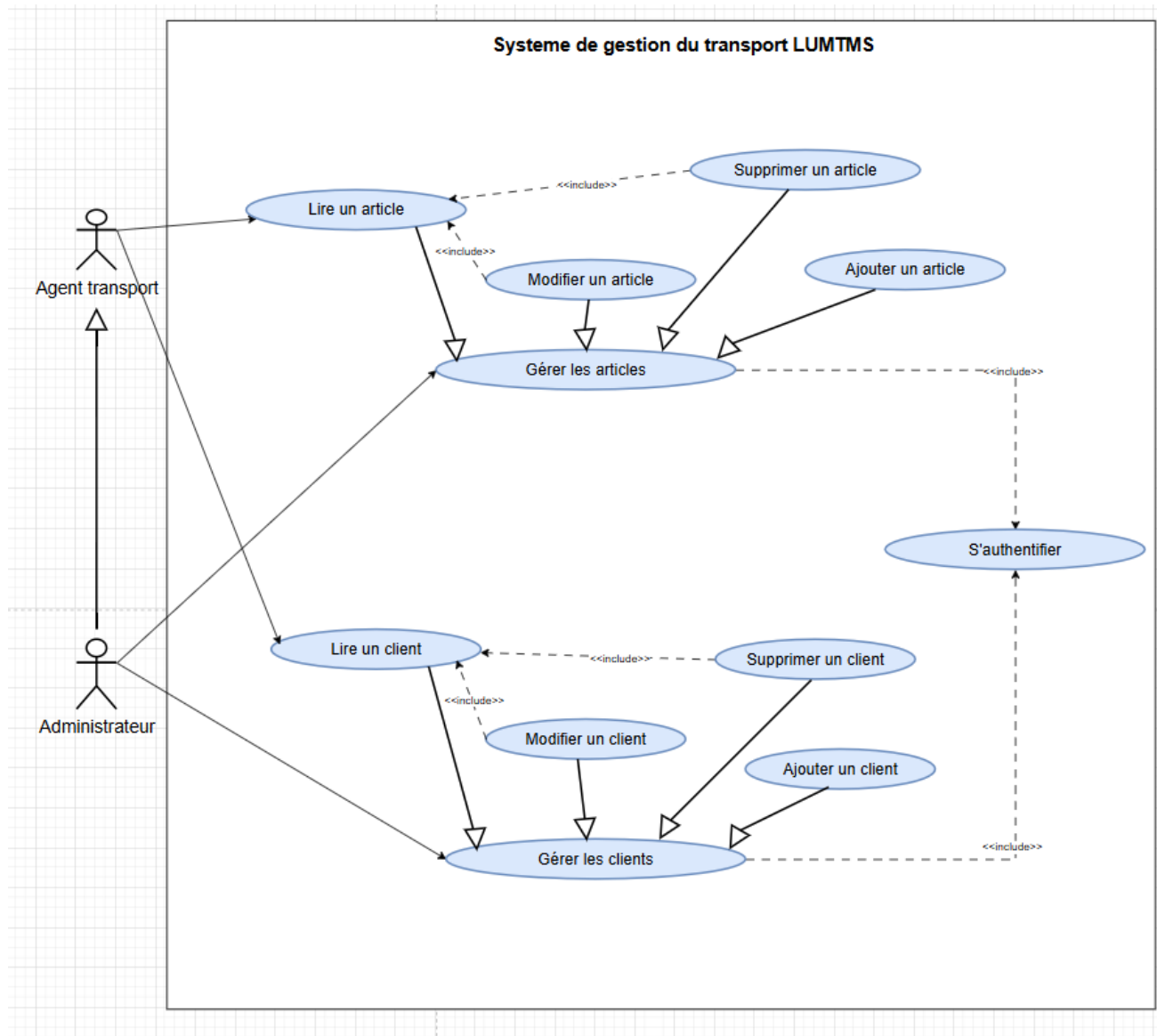


FIGURE 5.1 – Diagramme de cas d'utilisation du Sprint 2

5.5 Description textuelle des cas d'utilisation

5.5.1 Cas d'utilisation "Ajouter un article"

Ce cas d'utilisation, réservé à l'administrateur, décrit comment ajouter un nouvel article au système.

TABLE 5.2: Description textuelle du cas d'utilisation "Ajouter un article"

Titre	Ajouter un article
Acteur	Administrateur
Précondition	L'administrateur est authentifié et dispose des droits nécessaires.
Postcondition	Un nouvel article est créé et enregistré dans la base de données.
Scénario nominal	1. L'administrateur ouvre l'écran "Ajouter article". 2. L'interface utilisateur affiche un formulaire de saisie. 3. L'administrateur remplit et soumet le formulaire. 4. Le frontend envoie une requête POST à l'API Gateway. 5. L'API Gateway transmet la requête au service Article. 6. Le service vérifie l'unicité du code article dans la base de données. 7. Le service insère le nouvel article dans la base de données. 8. Le système retourne une confirmation et l'interface est mise à jour.
Scénarios alternatifs	— A1 : Code article déjà existant : 1. Le système retourne une erreur 409 Conflict. 2. L'interface affiche un message d'erreur indiquant que le code est déjà utilisé. — A2 : Erreur système : 1. Le système retourne une erreur 500 Internal Server Error. 2. L'interface affiche un message d'erreur générique.

5.5.2 Autres cas d'utilisation de gestion des articles

La modification et la suppression d'un article suivent un processus similaire à l'ajout. L'administrateur sélectionne un article existant pour le modifier ou le supprimer via des interfaces dédiées, et le système

met à jour la base de données en conséquence.

5.5.3 Cas d'utilisation "Ajouter un client"

Ce cas d'utilisation, réservé à l'administrateur, détaille la procédure de création d'un nouveau client.

TABLE 5.3: Description textuelle du cas d'utilisation "Ajouter un client"

Titre	Ajouter un client
Acteur	Administrateur
Précondition	L'administrateur est authentifié et dispose des droits nécessaires.
Postcondition	Un nouveau client est créé et enregistré dans la base de données.
Scénario nominal	1. L'administrateur ouvre l'écran "Ajouter client". 2. L'interface utilisateur affiche un formulaire de saisie. 3. L'administrateur remplit et soumet le formulaire. 4. Le frontend envoie une requête POST à l'API Gateway. 5. L'API Gateway transmet la requête au service Client. 6. Le service vérifie l'unicité du code client dans la base de données. 7. Le service insère le nouveau client dans la base de données. 8. Le système retourne une confirmation et l'interface est mise à jour.
Scénarios alternatifs	— A1 : Code client déjà existant : 1. Le système retourne une erreur 409 Conflict. 2. L'interface affiche un message d'erreur indiquant que le code est déjà utilisé. — A2 : Erreur système : 1. Le système retourne une erreur 500 Internal Server Error. 2. L'interface affiche un message d'erreur générique.

5.5.4 Autres cas d'utilisation de gestion des clients

La gestion des clients inclut également la modification et la suppression. Ces opérations permettent à l'administrateur de mettre à jour les informations d'un client ou de le retirer du système, en suivant

des étapes similaires à celles de l'ajout.

5.6 Conception

5.6.1 Diagramme de classes

Le diagramme de classes suivant pour le Sprint 2 illustre les nouvelles entités 'Article' et 'Client'.

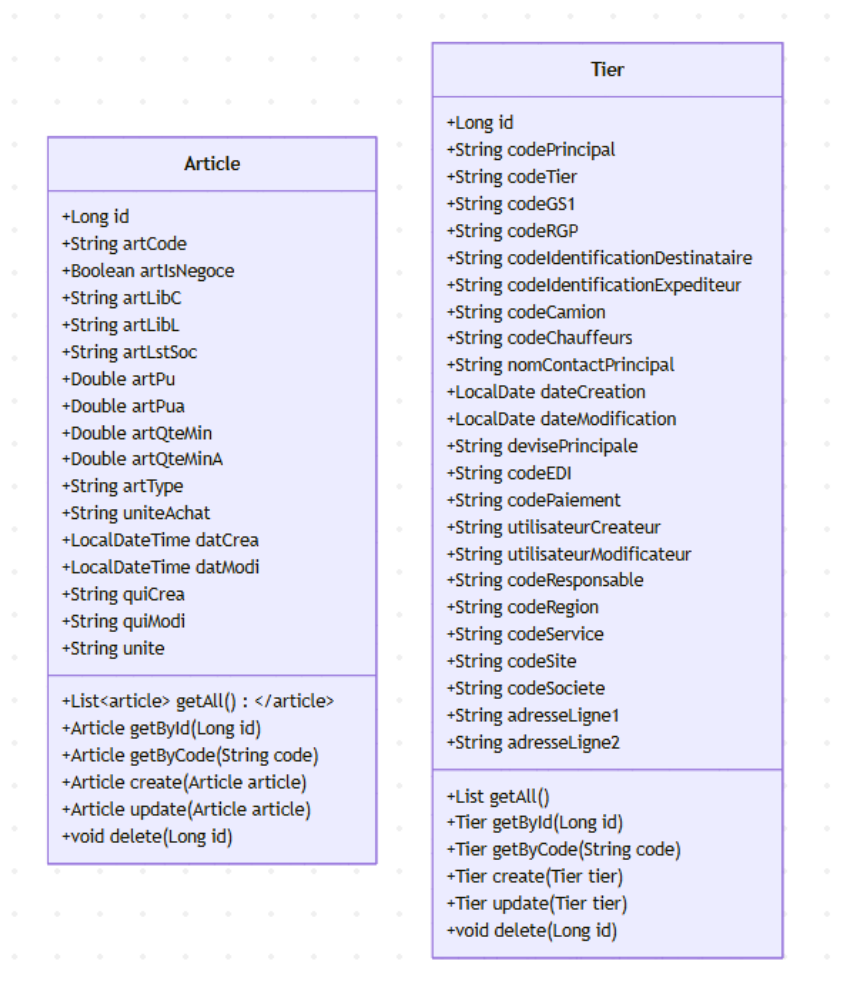


FIGURE 5.2 – Diagramme de classes du Sprint 2

5.6.2 Diagrammes de séquence

5.6.2.1 Diagramme de séquence "Ajouter un article"

Ce diagramme montre les interactions lors de l'ajout d'un nouvel article par un administrateur.

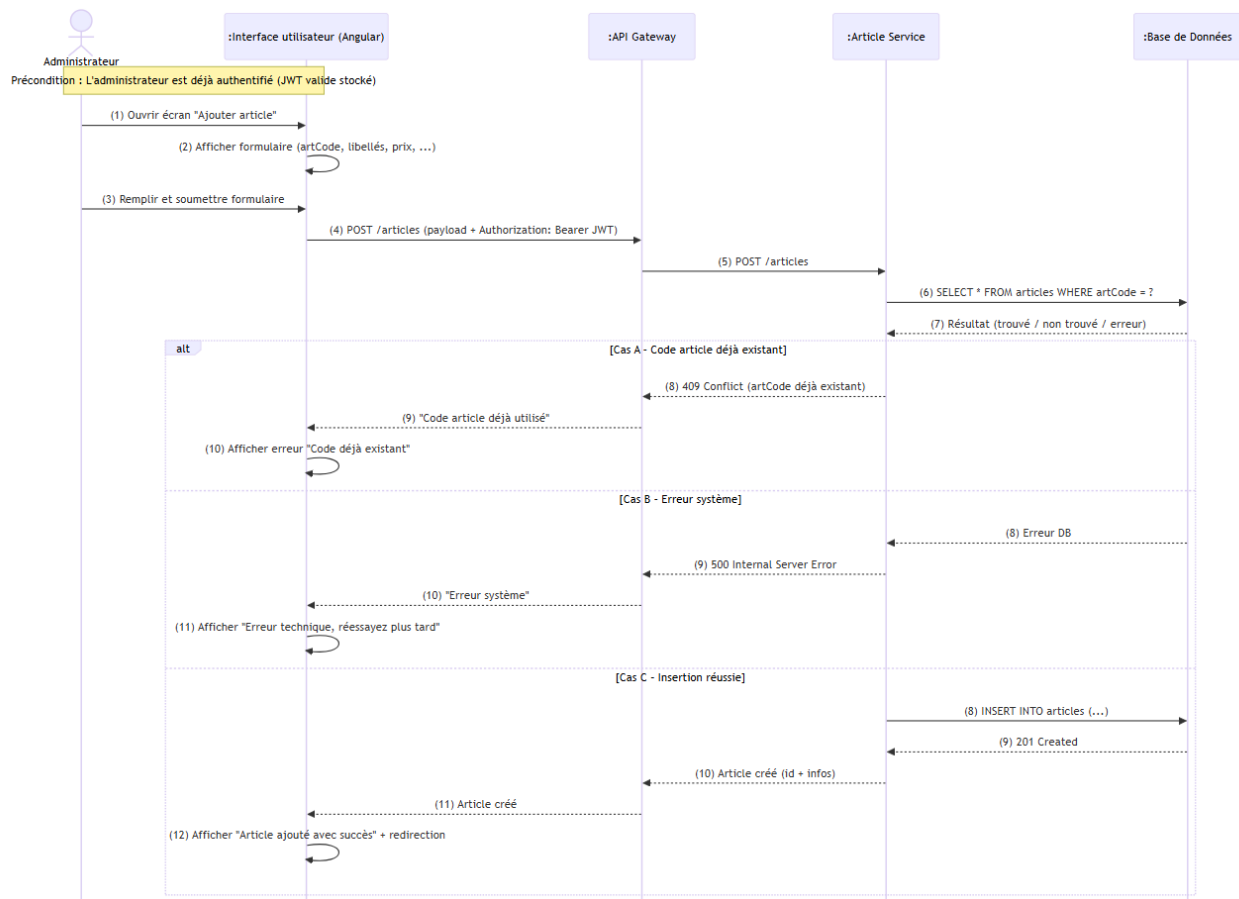


FIGURE 5.3 – Diagramme de séquence - "Ajouter un article"

5.6.2.2 Diagramme de séquence de "ajouter un client"

Ce diagramme illustre le flux d'interactions pour l'ajout d'un nouveau client dans le système.

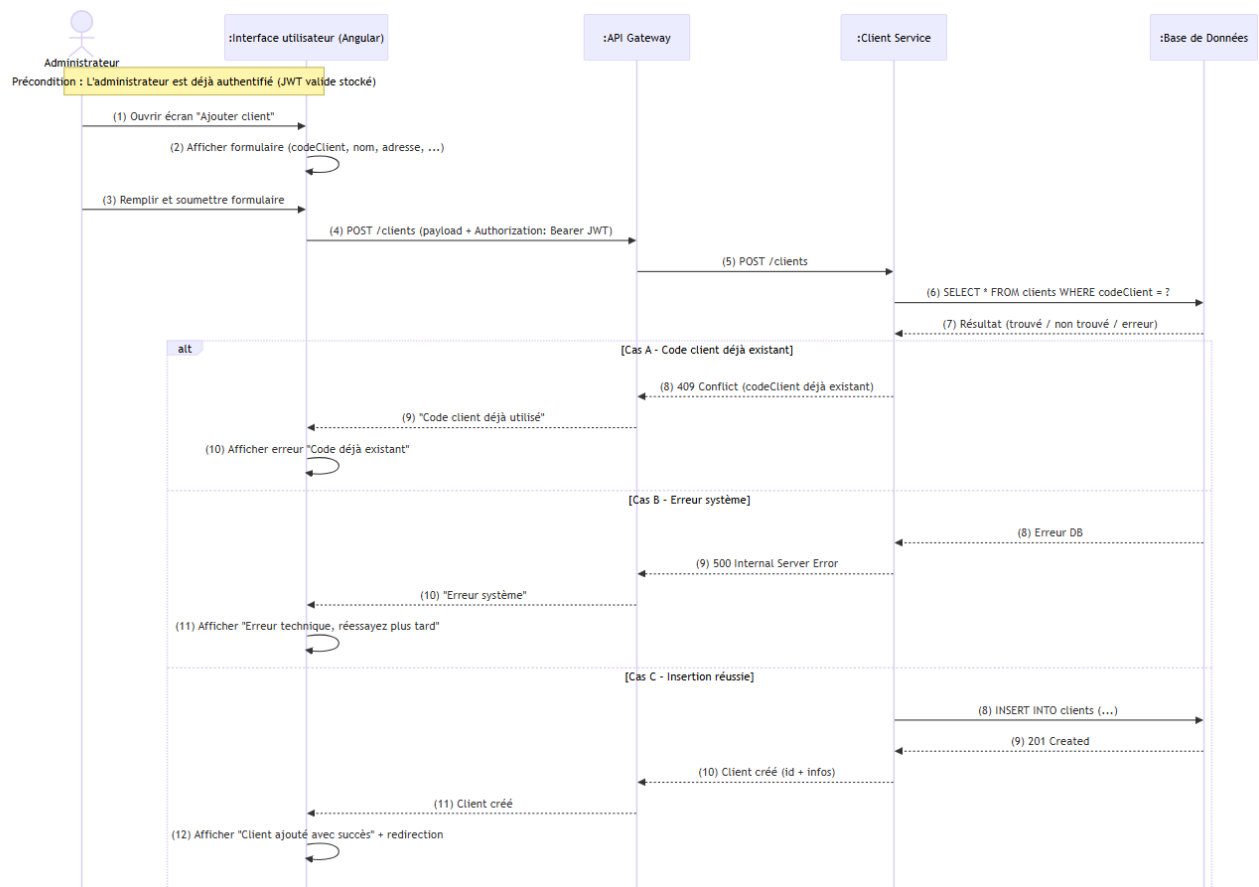


FIGURE 5.4 – Diagramme de séquence - "ajouter un client"

5.7 Réalisation

5.7.1 Interface de gestion des articles

L'interface de gestion des articles permet aux administrateurs de gérer l'ensemble du catalogue. Elle offre des fonctionnalités pour ajouter, consulter, modifier et supprimer des articles de manière efficace.

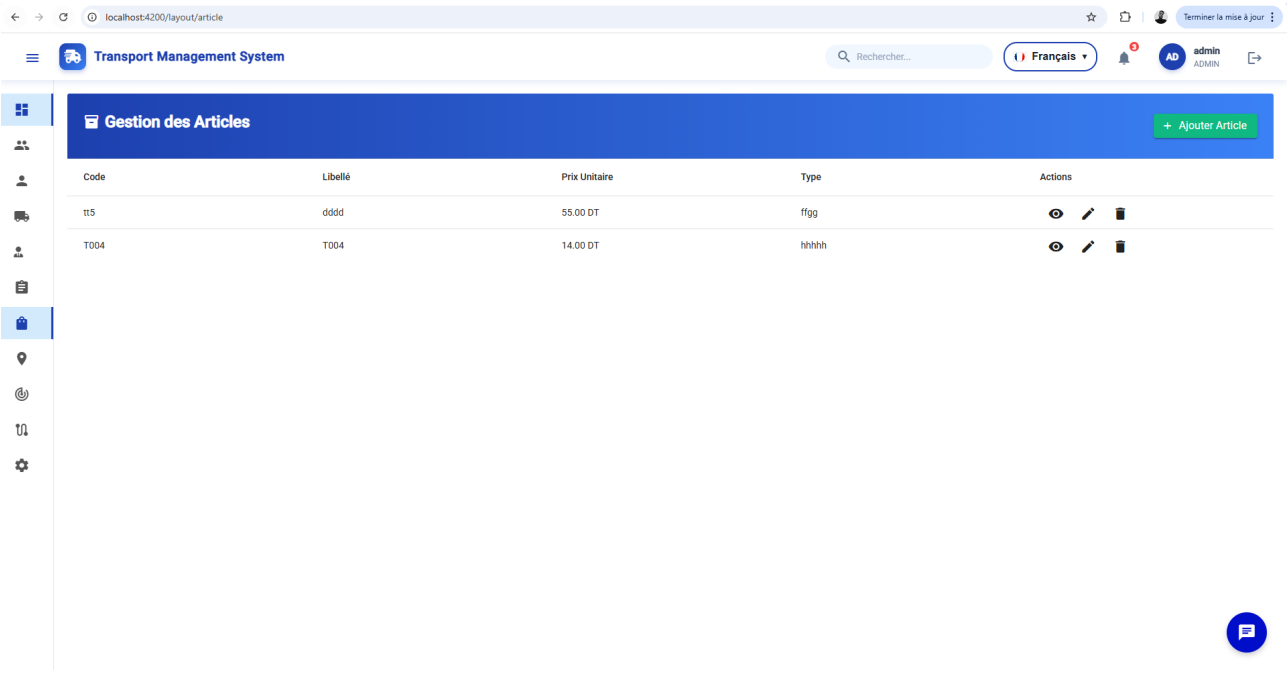


FIGURE 5.5 – Interface de gestion des articles

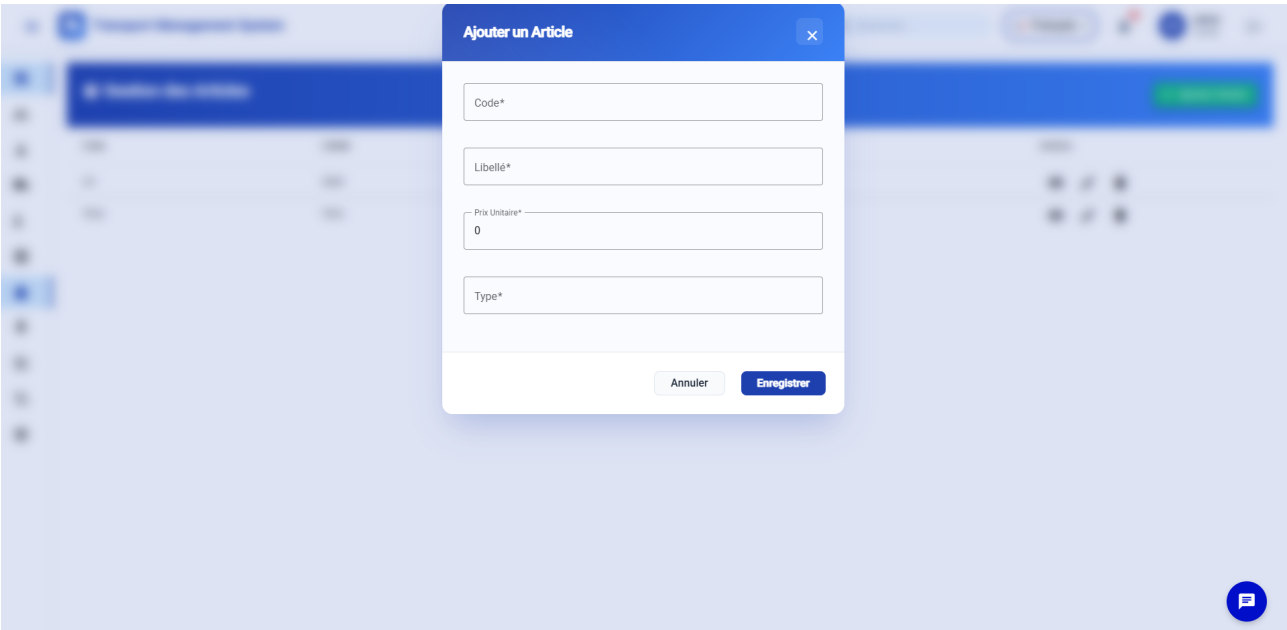


FIGURE 5.6 – Interface ajouter article

5.7.2 Interface de gestion des clients

Cette interface centralise la gestion des clients. Les administrateurs peuvent y créer de nouveaux clients, mettre à jour leurs informations, et consulter la liste complète des clients enregistrés.

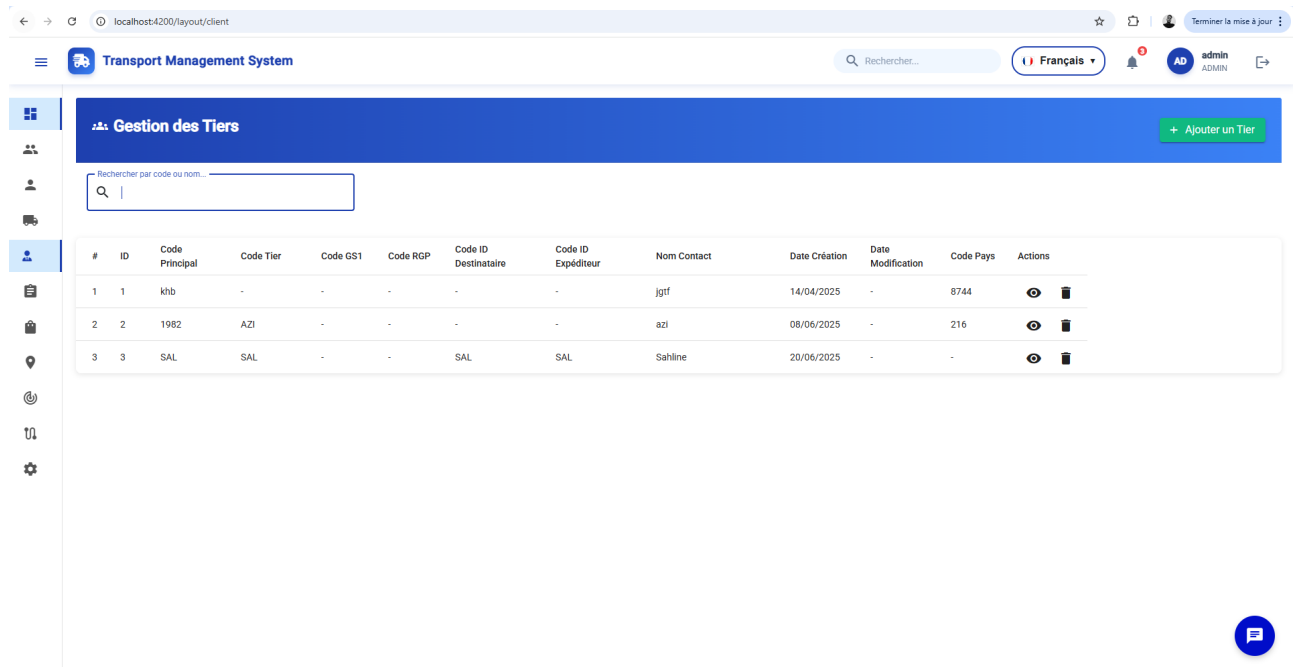


FIGURE 5.7 – Interface de gestion des clients

Ajouter un Tier

Code Principal **	Code Tier
Code GS1	Code RGP
Adresse (Ligne 1)	Code ID Destinataire
Code ID Expéditeur	Nom Contact Principal **
Code Pays	

Annuler Créer

FIGURE 5.8 – Interface ajouter des clients

Détails du Tier

ID : 1	CODE PRINCIPAL : khb	CODE TIER : -
CODE GS1 : -	CODE RGP : -	ADRESSE (LIGNE 1) : -
CODE ID DESTINATAIRE : -	CODE ID EXPÉDITEUR : -	NOM CONTACT PRINCIPAL : jgtf
DATE CRÉATION : 14/04/2025	DATE MODIFICATION : -	CODE PAYS : 8744

Fermer

FIGURE 5.9 – Interface consulter client

5.8 Conclusion

Le Sprint 2 a permis d'enrichir considérablement les fonctionnalités de l'application LUMTMS en introduisant la gestion complète des articles et des clients. Les objectifs de ce sprint ont été atteints, avec la mise en place des opérations CRUD nécessaires pour les administrateurs et des droits de consultation pour les agents. Ces nouvelles fonctionnalités sont essentielles pour la gestion des données de base du système de transport. Cette étape consolide l'architecture de l'application et prépare le terrain pour les sprints suivants, qui se concentreront sur la gestion des ordres de transport.

Chapitre 6

Sprint 3 – Gestion des Chauffeurs et des Camions

6.1 Introduction

Le troisième sprint du projet, est consacré à la gestion des chauffeurs et des camions. Ce sprint a pour but de mettre en place les fonctionnalités permettant aux administrateurs et agents de gérer efficacement le parc de camions et le personnel de conduite, tout en intégrant des mécanismes d'optimisation et de suivi en temps réel. Il constitue une étape clé dans la mise en œuvre du système, car il relie les aspects logistiques (véhicules) et opérationnels (chauffeurs) afin de préparer les phases ultérieures de planification et de suivi des voyages.

6.2 Objectifs du Sprint

Les objectifs principaux de ce sprint sont :

- Implémenter les fonctionnalités CRUD (Create, Read, Update, Delete) pour les entités Camion et Chauffeur.
- Permettre la mise à jour des statuts (actif, en maintenance, en congé, etc.).
- Mettre en place un système d'attribution automatique des camions et des chauffeurs adaptés pour un voyage donné.
- Intégrer une visualisation cartographique des camions pour un suivi en temps réel.
- Assurer une interface intuitive et réactive pour les utilisateurs administrateurs et agents.

6.3 Spécification et Analyse des Besoins

6.3.1 Sprint Backlog

Le tableau suivant présente le Sprint Backlog du sprint 3. Il détaille les fonctionnalités prévues, les scénarios utilisateurs correspondants, ainsi que les différentes tâches techniques identifiées avec leur estimation en termes de charge de travail.

TABLE 6.1: Sprint Backlog du Sprint 3

ID Fonctionnalité	Scénario Utilisateur	ID Tâche	Tâches	Estimation
US-10	En tant qu'Administrateur, je souhaite gérer les camions (CRUD)	T1	Développement API REST pour gestion des camions	2 j
		T2	Création interface front-end pour ajout, modification, suppression	3 j
		T3	Tests unitaires du service Camion	1,5 j
US-11	En tant qu'Administrateur/Agent, je souhaite consulter et modifier l'état des camions	T4	API de mise à jour de l'état	1,5 j
		T5	Affichage du statut en interface	1 j
US-12	Le système doit proposer automatiquement des camions adaptés pour un voyage	T6	Développement algorithme de filtrage selon critères (poids, volume, état)	3 j
US-13	En tant qu'Administrateur/Agent, je souhaite visualiser la position des camions sur une carte	T7	Intégration API cartographique	4 j
US-14	En tant qu'Administrateur, je souhaite gérer les chauffeurs (CRUD)	T8	API REST pour gestion des chauffeurs	2 j

Suite à la page suivante

Table 6.1 – suite de la page précédente

ID Fonction-nalité	Scénario Utilisateur	ID Tâche	Tâches	Estimation
		T9	Interface front-end CRUD chauffeurs	3 j
		T10	Tests unitaires du service Chauffeur	1,5 j
US-15	En tant qu'Administrateur/Agent, je souhaite consulter et modifier l'état des chauffeurs	T11	API de mise à jour du statut chauffeur	1,5 j
US-16	Le système doit proposer automatiquement des chauffeurs adaptés pour un voyage	T12	Développement algorithme de filtrage (disponibilité, compétences)	3 j

6.3.2 Diagramme de cas d'utilisation du Sprint

Le diagramme de cas d'utilisation suivant illustre les interactions entre les acteurs (Administrateur, Agent) et le système pour la gestion des chauffeurs et des camions :

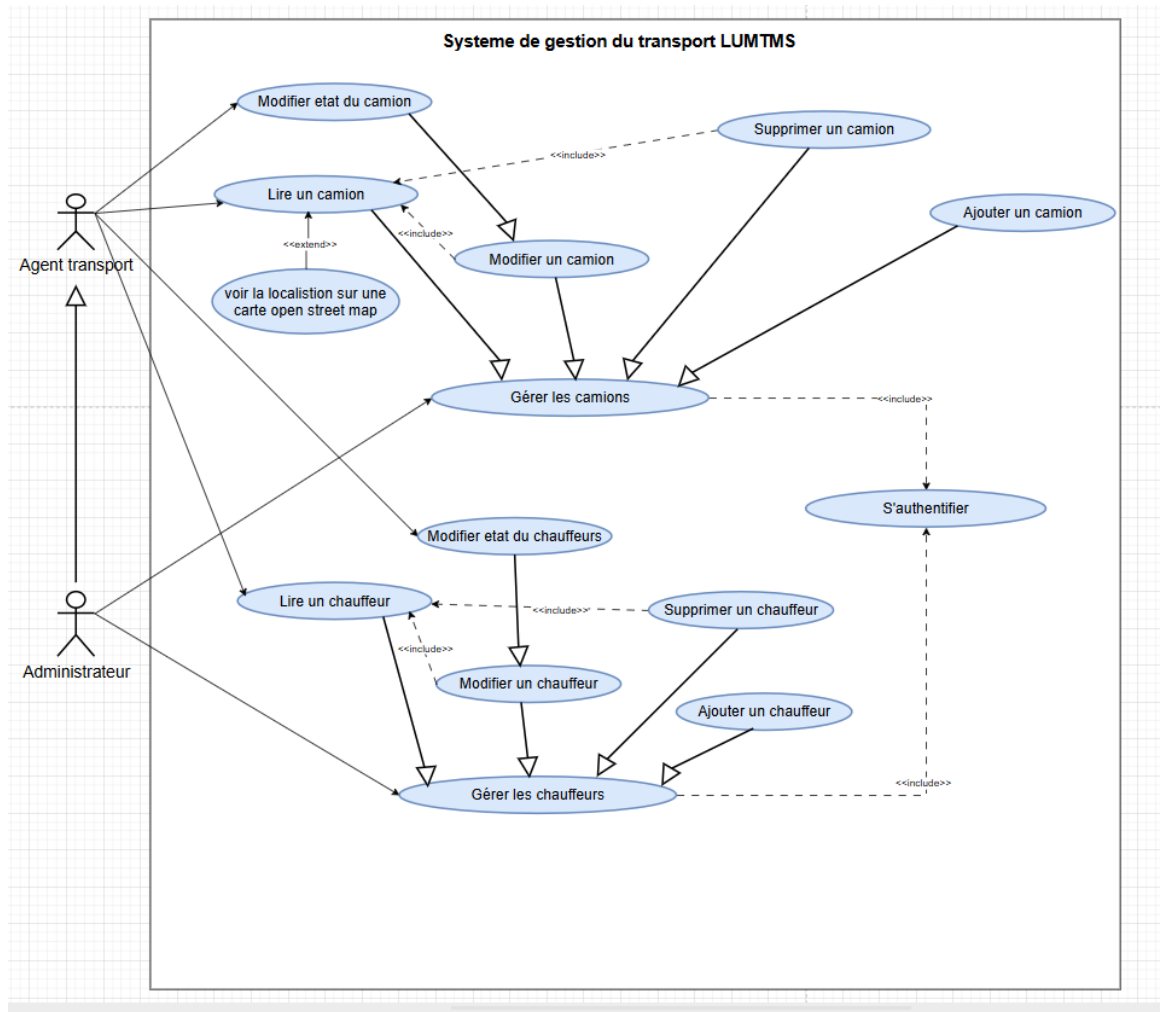


FIGURE 6.1 – Diagramme de cas d'utilisation du Sprint 3

6.4 Description textuelle des cas d'utilisation

6.4.1 Description textuelle du cas d'utilisation "Ajouter un camion"

Ce cas d'utilisation, réservé à l'administrateur, détaille la procédure de création d'un nouveau camion. Il inclut la validation des données et la vérification de l'unicité de l'immatriculation.

TABLE 6.2: Description textuelle du cas d'utilisation "Ajouter un camion"

Titre	Ajouter un camion
Acteur	Administrateur
Précondition	L'administrateur est authentifié et dispose des droits nécessaires.
Postcondition	Un nouveau camion est créé et enregistré dans le système.
Scénario nominal	1. L'administrateur accède à la page "Ajouter camion". 2. Le système affiche un formulaire de création (immatriculation, capacité, type, état, etc.). 3. L'administrateur remplit les informations et soumet le formulaire. 4. Le système envoie une requête POST à l'API Gateway. 5. L'API Gateway transmet la requête au service Camion. 6. Le service valide les données et vérifie l'unicité de l'immatriculation. 7. Le service enregistre le nouveau camion dans la base de données. 8. Le système affiche un message de succès et met à jour la liste des camions.
Scénarios alternatifs	— A1 : Immatriculation déjà existante : 1. Le système affiche un message d'erreur indiquant que l'immatriculation est déjà utilisée. 2. Retour à l'étape 2. — A2 : Erreur système : 1. Le système informe l'utilisateur qu'une erreur est survenue. 2. Retour à l'étape 1.

6.4.2 Description textuelle du cas d'utilisation "Consulter la localisation d'un camion"

Ce cas d'utilisation décrit comment un utilisateur peut visualiser la position des camions en temps réel et rechercher un véhicule spécifique.

TABLE 6.3: Description textuelle du cas d'utilisation
 “Consulter la localisation d’un camion”

Titre	Consulter la localisation d’un camion
Acteur	Utilisateur (Administrateur ou Agent)
Précondition	L'utilisateur est authentifié et dispose des droits nécessaires.
Postcondition	La position des camions est affichée sur une carte interactive.
Scénario nominal	1. L'utilisateur ouvre l'interface de localisation. 2. L'interface envoie une requête GET à l'API Gateway pour obtenir la position des camions. 3. L'API Gateway transmet la requête au service Camion. 4. Le service Camion exécute un script Python pour récupérer les données. 5. Le script se connecte à la base de données et récupère les positions. 6. La base de données retourne les positions au format JSON. 7. Le script transmet le JSON au service Camion, qui le relaie à l'API Gateway. 8. L'API Gateway retourne les données à l'interface utilisateur. 9. L'interface affiche les camions sur la carte OpenStreetMap. 10. L'utilisateur saisit une matricule pour rechercher un camion. 11. La carte zoome sur le camion trouvé et affiche ses détails.
Scénarios alternatifs	— A1 : Erreur d'exécution du script Python : 1. Le système affiche un message "Erreur technique – veuillez réessayer". — A2 : Base de données inaccessible : 1. Le système affiche un message "Impossible d'accéder aux données". — A3 : API OpenStreetMap indisponible : 1. Le système affiche un message "Carte indisponible, réessayez plus tard". — A4 : Matricule introuvable : 1. Le système affiche un message "Aucun camion trouvé".

6.4.3 Autres cas d'utilisation de gestion des camions

La gestion des camions inclut également la modification et la suppression. Ces opérations permettent à l'administrateur de mettre à jour les informations d'un camion ou de le retirer du système, en suivant des étapes similaires à celles de l'ajout.

6.4.4 Description textuelle du cas d'utilisation "Ajouter un chauffeur"

Ce cas d'utilisation, réservé à l'administrateur, détaille la procédure de création d'un nouveau compte chauffeur. Il inclut la validation des données et la vérification de l'unicité de la CIN.

TABLE 6.4: Description textuelle du cas d'utilisation "Ajouter un chauffeur"

Titre	Ajouter un chauffeur
Acteur	Administrateur
Précondition	L'administrateur est authentifié et dispose des droits nécessaires.
Postcondition	Un nouveau chauffeur est créé et enregistré dans le système.
Scénario nominal	1. L'administrateur accède à la page "Ajouter chauffeur". 2. Le système affiche un formulaire de création (CIN, nom, prénom, etc.). 3. L'administrateur remplit les informations et soumet le formulaire. 4. Le système envoie une requête POST à l'API Gateway. 5. L'API Gateway transmet la requête au service Chauffeur. 6. Le service valide les données et vérifie l'unicité de la CIN. 7. Le service enregistre le nouveau chauffeur dans la base de données. 8. Le système affiche un message de succès et met à jour la liste des chauffeurs.

Scénarios alternatifs	— A1 : CIN déjà existante : 1. Le système affiche un message d’erreur indiquant que la CIN est déjà utilisée. 2. Retour à l’étape 2. — A2 : Erreur système : 1. Le système informe l’utilisateur qu’une erreur est survenue. 2. Retour à l’étape 1.
------------------------------	---

6.4.5 Description textuelle du cas d’utilisation ” Modifier l’état d’un chauffeur (mettre en repos)”

Ce cas d’utilisation décrit comment un utilisateur peut changer l’état d’un chauffeur pour le mettre en congé. L’opération n’est autorisée que si le chauffeur n’est pas déjà actif sur une mission.

TABLE 6.5: Description textuelle du cas d’utilisation “ Modifier l’état d’un chauffeur (mettre en repos)”

Titre	Mettre un chauffeur en repos
Acteur	Utilisateur (Administrateur ou Agent)
Précondition	L’utilisateur est authentifié et se trouve sur la page de gestion des chauffeurs.
Postcondition	L’état du chauffeur est mis à jour à "EN_CONGE" dans la base de données.
Scénario nominal	1. L’utilisateur sélectionne un chauffeur dans la liste. 2. Il choisit l’option pour modifier l’état en "EN_CONGE". 3. Le système envoie une requête PUT à l’API Gateway. 4. L’API Gateway transmet la requête au service Chauffeur. 5. Le service vérifie que l’état actuel du chauffeur n’est pas "ACTIF". 6. Le service met à jour l’état du chauffeur à "EN_CONGE" dans la base de données. 7. Le système affiche un message de succès et met à jour l’interface.

Scénarios alternatifs	— A1 : Chauffeur actif : 1. Si le chauffeur est actif, le système retourne une erreur. 2. L'interface affiche un message indiquant que l'opération est impossible. 3. Retour à l'étape 1.
------------------------------	---

6.4.6 Autres cas d'utilisation de gestion des chauffeurs

La gestion des chauffeurs inclut également la modification et la suppression. Ces opérations permettent à l'administrateur de mettre à jour les informations d'un chauffeur ou de le retirer du système, en suivant des étapes similaires à celles de l'ajout.

6.5 Conception

6.5.1 Diagramme de classes

Le diagramme de classes suivant pour le Sprint 3 illustre les nouvelles entités 'Chauffeur' et 'Camion' ainsi que leurs relations avec les autres composants du système.

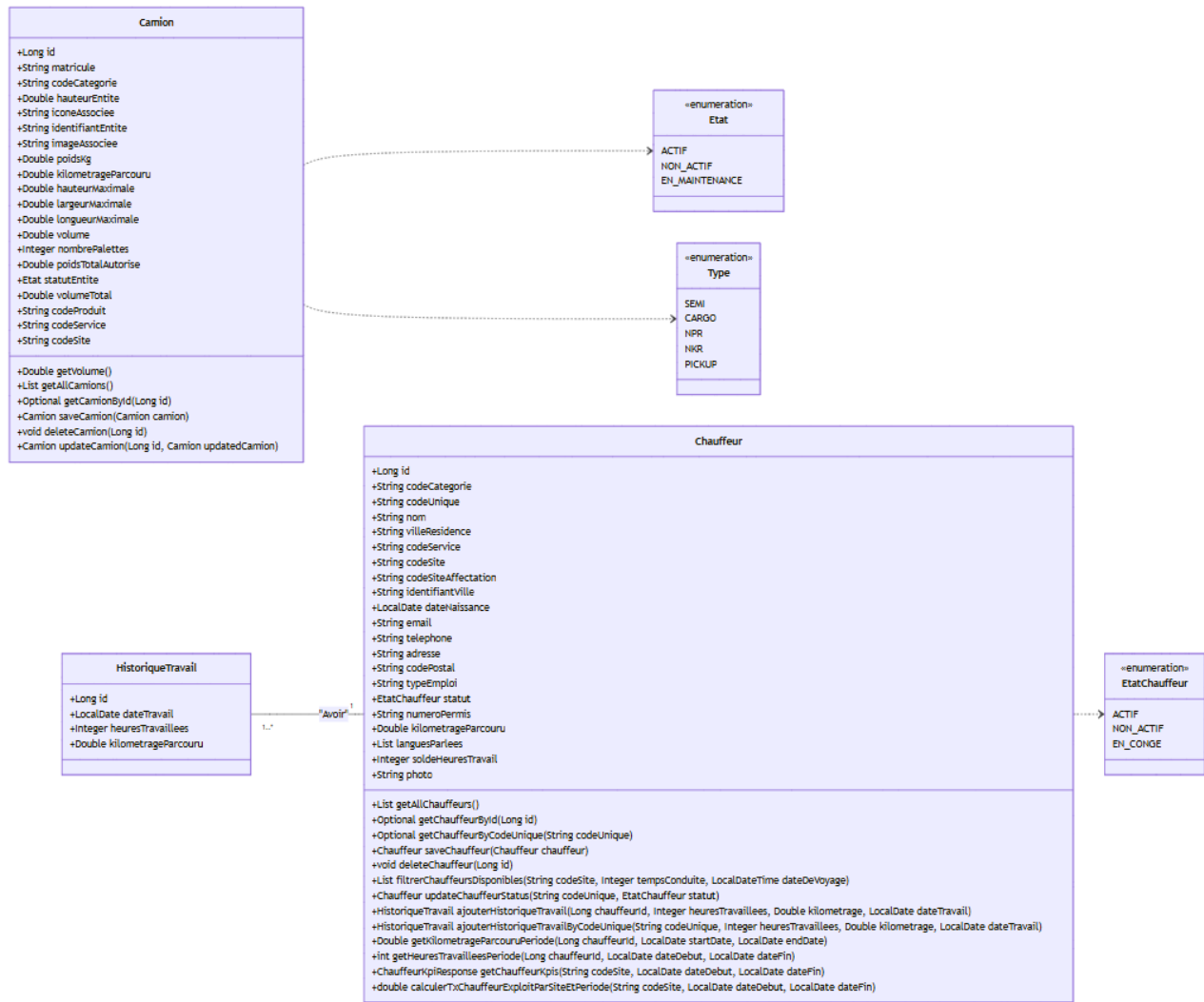


FIGURE 6.2 – Diagramme de classes du Sprint 3

6.5.2 Diagrammes de séquence

Les diagrammes de séquence suivants illustrent les interactions entre les acteurs et le système lors de l'exécution de certaines fonctionnalités clés de ce sprint.

6.5.2.1 Diagramme de séquence "Ajouter un camion"

Le diagramme ci-dessous présente les échanges entre l'utilisateur (Administrateur) et le système lors de l'ajout d'un nouveau camion dans la base de données.

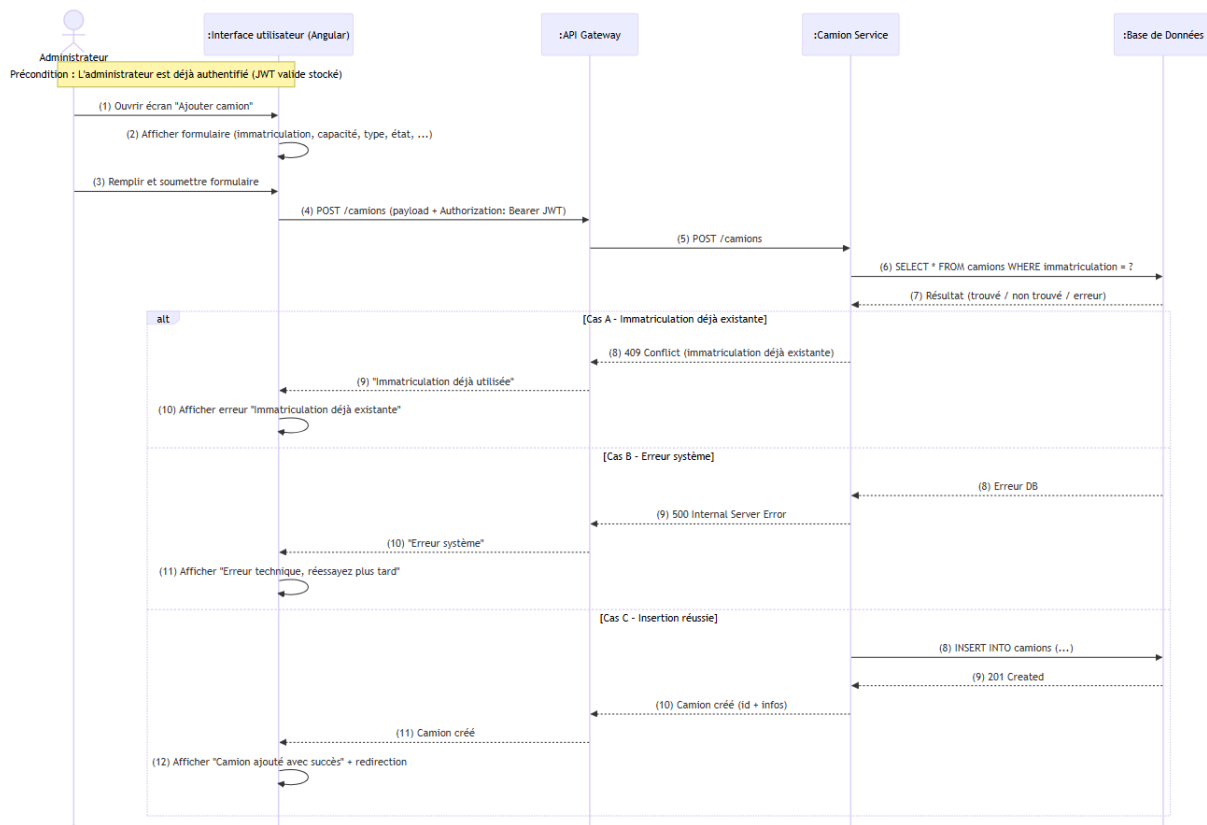


FIGURE 6.3 – Diagramme de séquence – "Ajouter un camion"

6.5.2.2 Diagramme de séquence "Consulter la localisation d'un camion"

Ce diagramme montre comment un utilisateur (Administrateur ou Agent) consulte la position d'un camion sur la carte via le système.

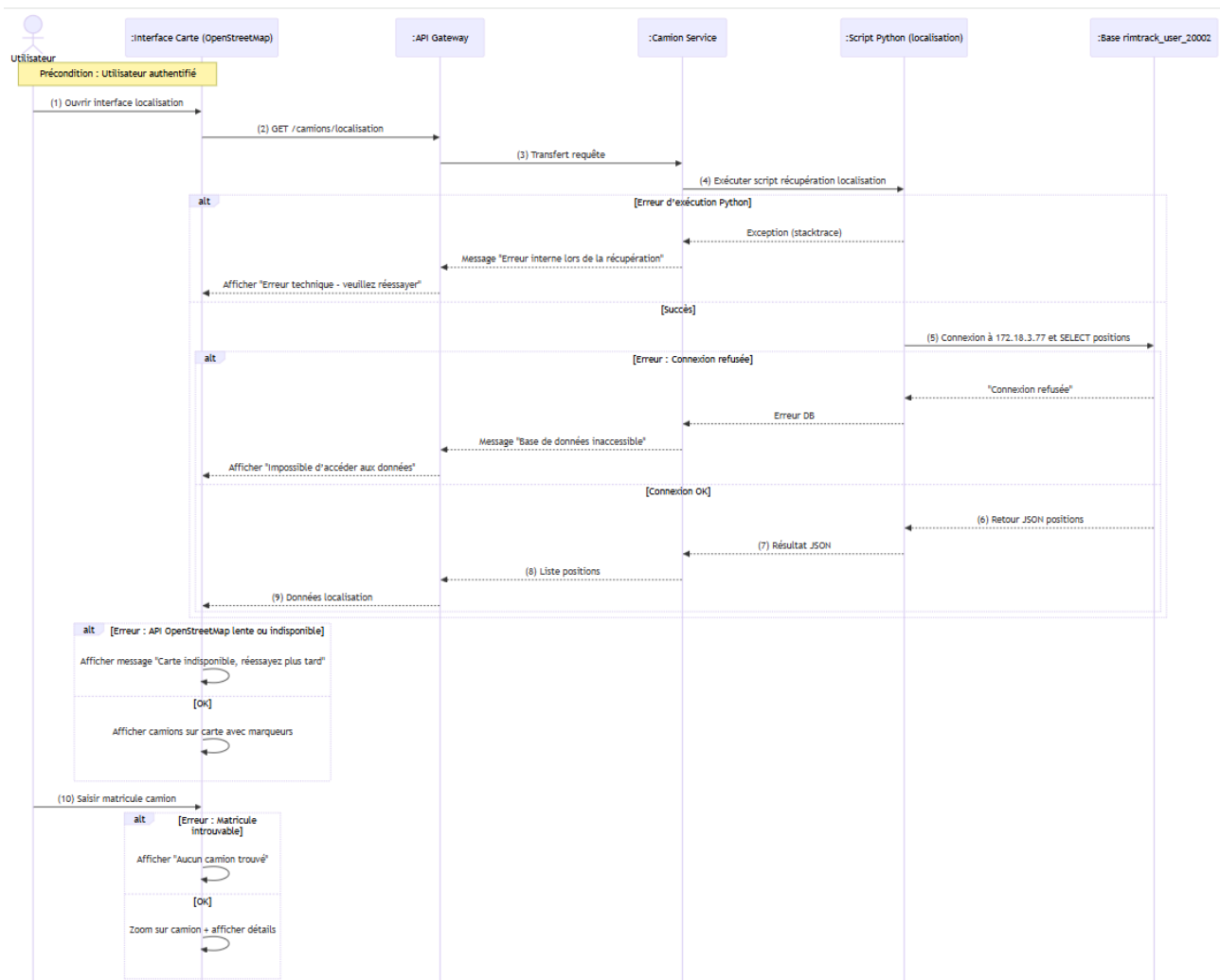


FIGURE 6.4 – Diagramme de séquence – "Consulter la localisation d'un camion"

6.5.2.3 Diagramme de séquence "Ajouter un chauffeur"

Le diagramme suivant illustre le processus d'ajout d'un nouveau chauffeur par l'Administrateur, incluant les interactions avec le backend et la validation des données.

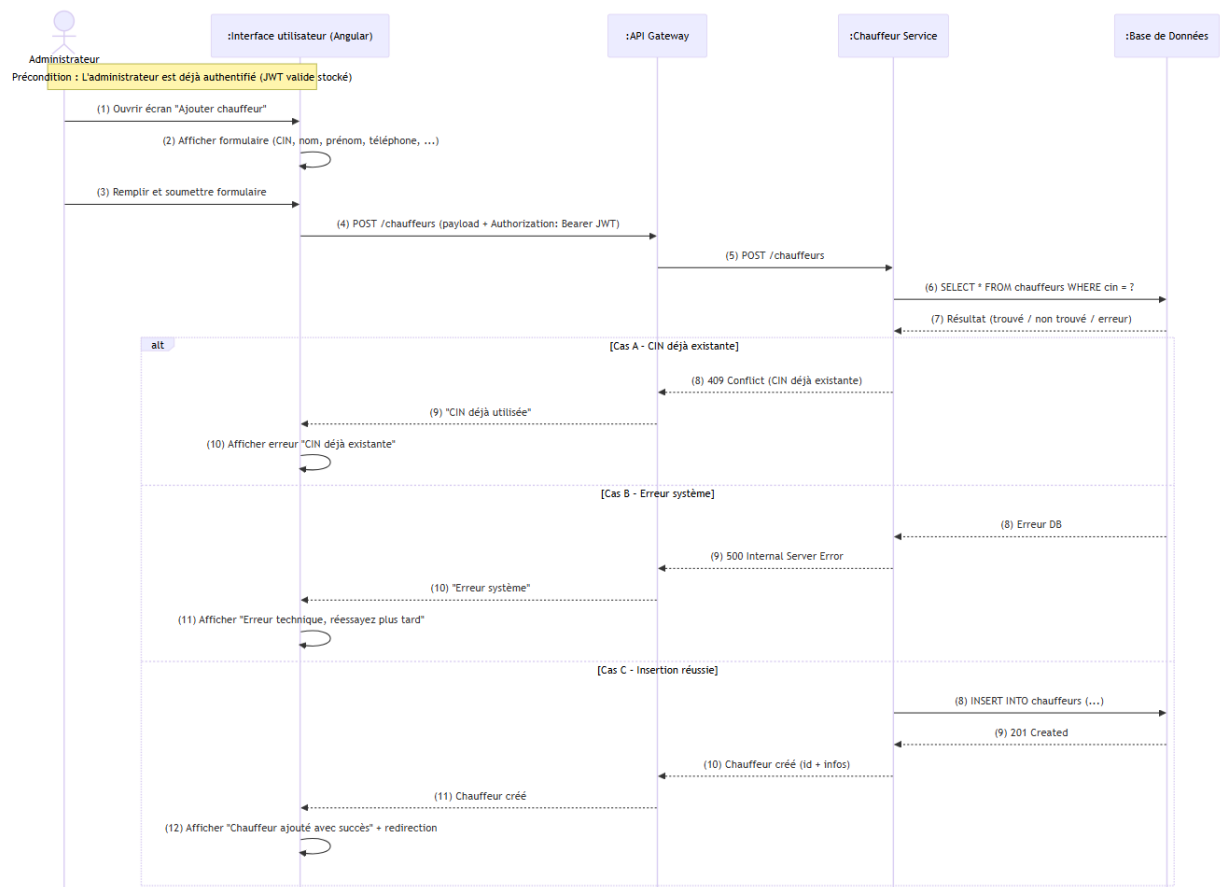


FIGURE 6.5 – Diagramme de séquence – "Ajouter un chauffeur"

6.5.2.4 Diagramme de séquence "Modifier l'état d'un chauffeur (mettre en repos)"

Ce diagramme décrit les interactions nécessaires pour modifier le statut d'un chauffeur, par exemple pour le mettre en repos, et la manière dont le système met à jour ces informations.

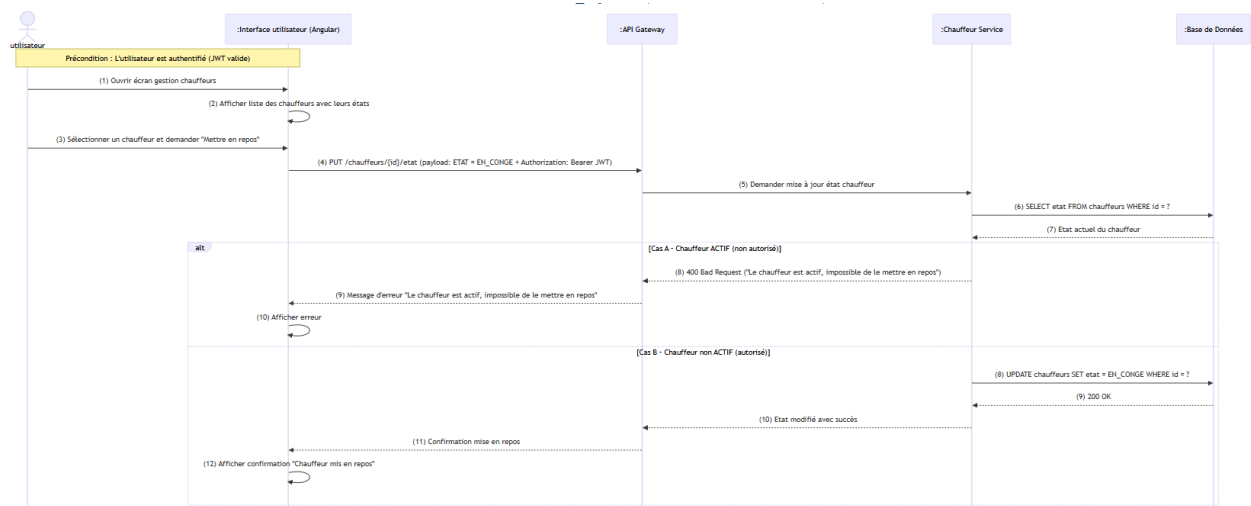


FIGURE 6.6 – Diagramme de séquence – ”Modifier l’état d’un chauffeur (mettre en repos)”

6.6 Réalisation

6.6.1 Interface de gestion des camions

L’interface de gestion des camions offre une vue centralisée pour administrer la flotte. Elle permet aux utilisateurs autorisés de consulter la liste des véhicules, d’ajouter de nouveaux camions et de visualiser les détails de chaque véhicule.

La figure suivante présente l’interface principale de gestion de la flotte, où l’on peut visualiser la liste complète des camions et accéder aux différentes options de gestion.

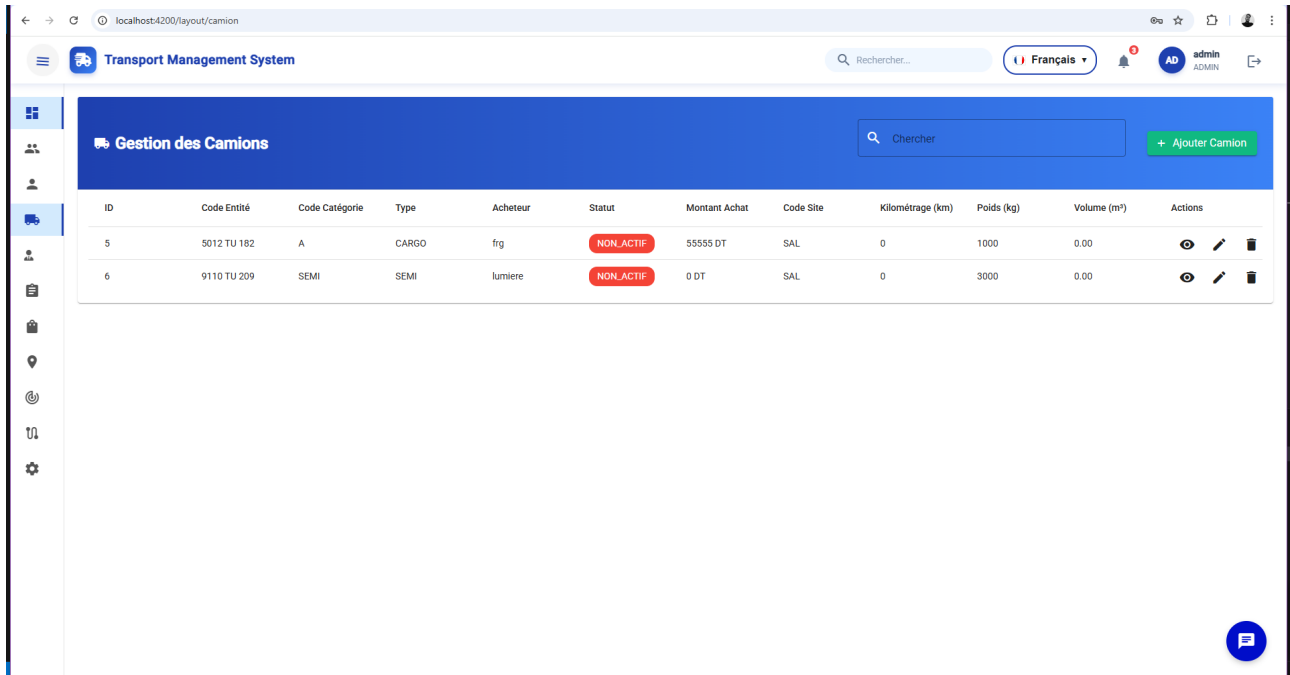


FIGURE 6.7 – Interface de gestion des camions

Le diagramme suivant illustre le formulaire permettant d'ajouter un nouveau camion dans le système, avec saisie des informations nécessaires.

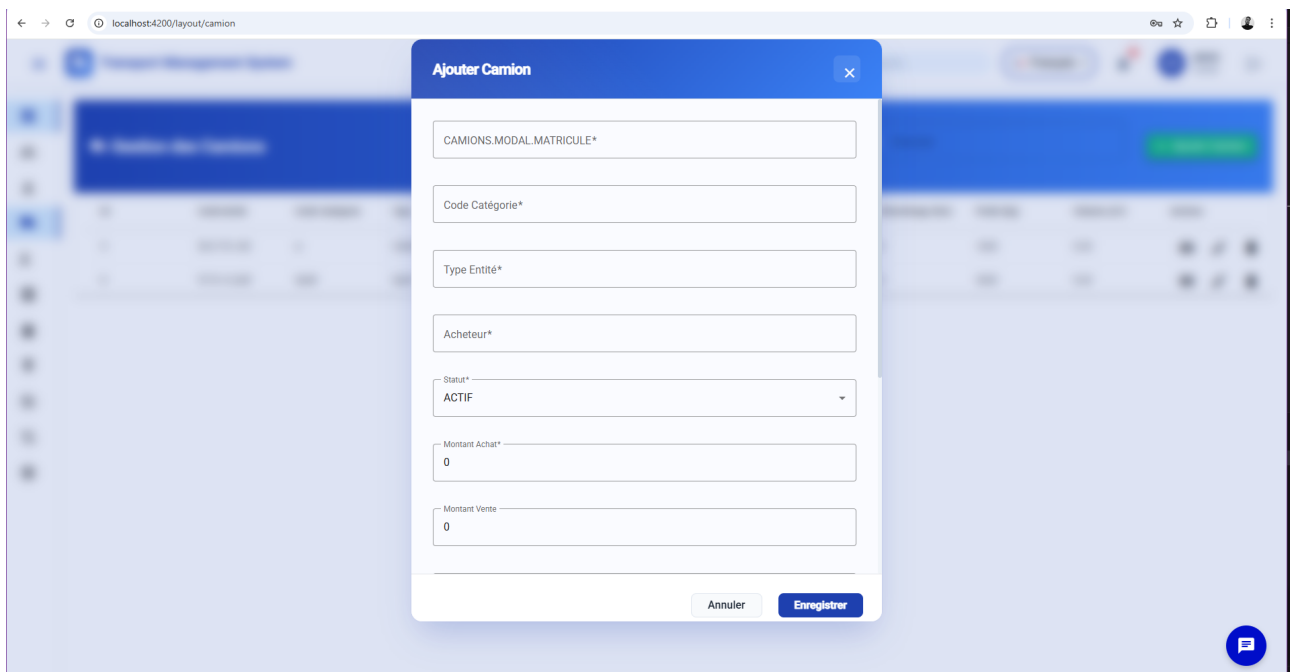


FIGURE 6.8 – Interface d'ajout d'un camion

L'interface ci-dessous permet à l'utilisateur de consulter les détails d'un camion spécifique, incluant ses caractéristiques et son état actuel.

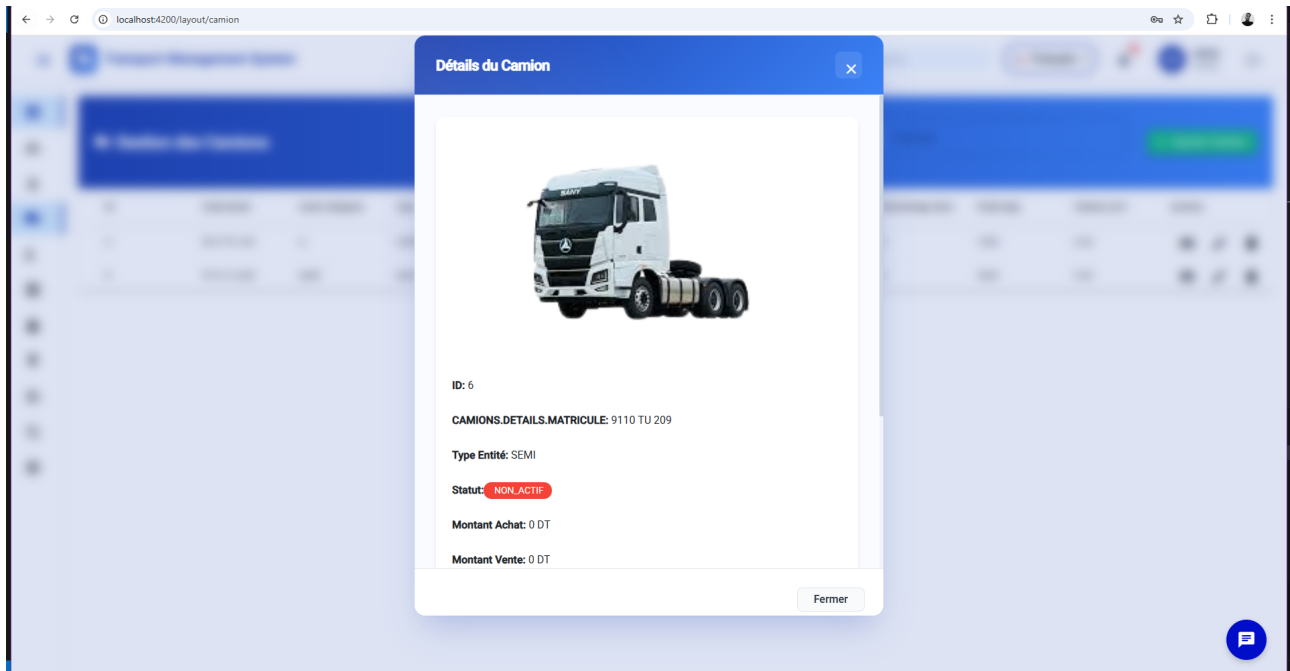


FIGURE 6.9 – Interface de consultation d'un camion

La figure suivante montre la vue générale de la localisation de tous les camions sur la carte, pour un suivi en temps réel.

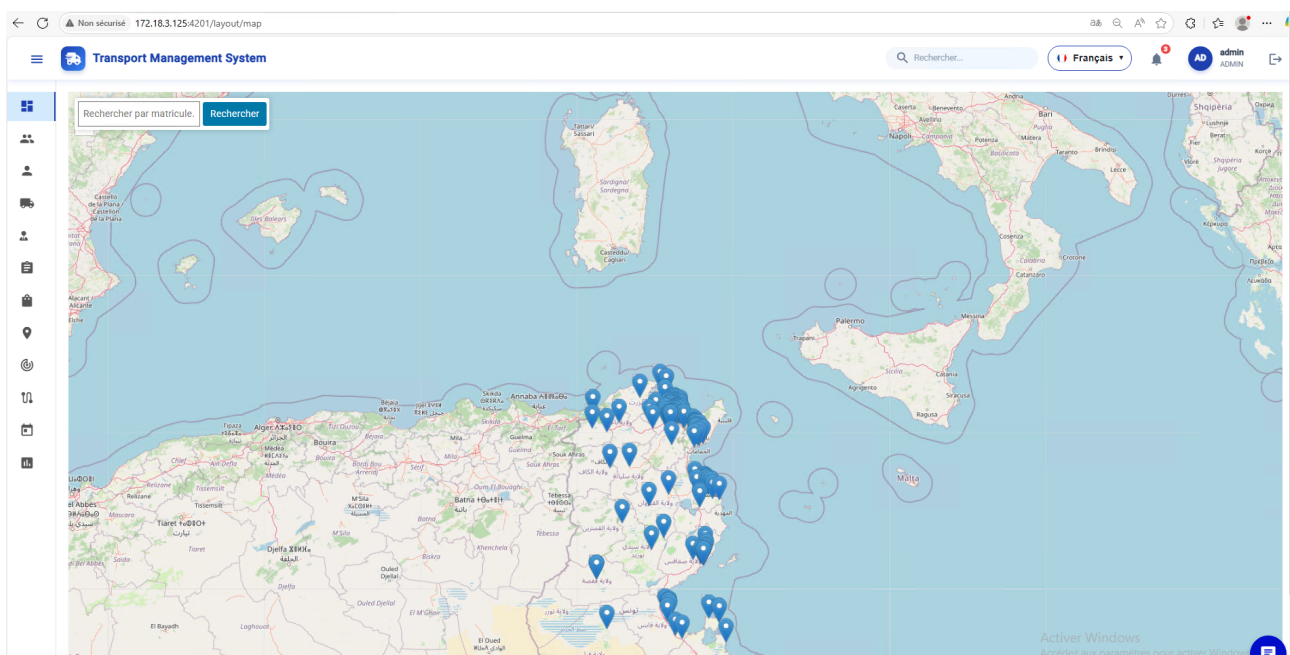


FIGURE 6.10 – Interface de consultation de la localisation des camions

Cette interface permet de visualiser la position exacte d'un camion sélectionné sur la carte.

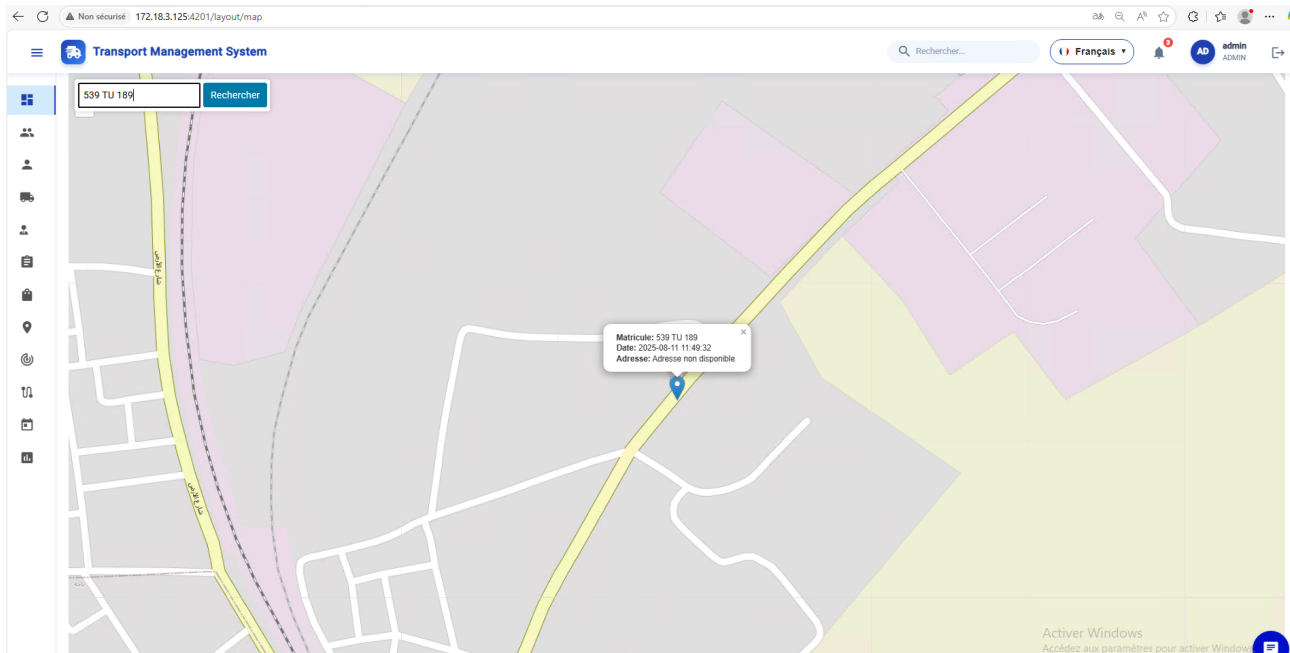


FIGURE 6.11 – Interface de consultation de la localisation d'un camion

6.6.2 Interface de gestion des chauffeurs

Cette section présente les interfaces dédiées à la gestion des chauffeurs. Elles permettent d'ajouter de nouveaux chauffeurs au système et de gérer leurs informations et leur statut.

L'interface suivante permet de visualiser la liste complète des chauffeurs et d'accéder aux fonctionnalités de gestion (ajout, modification, suppression).

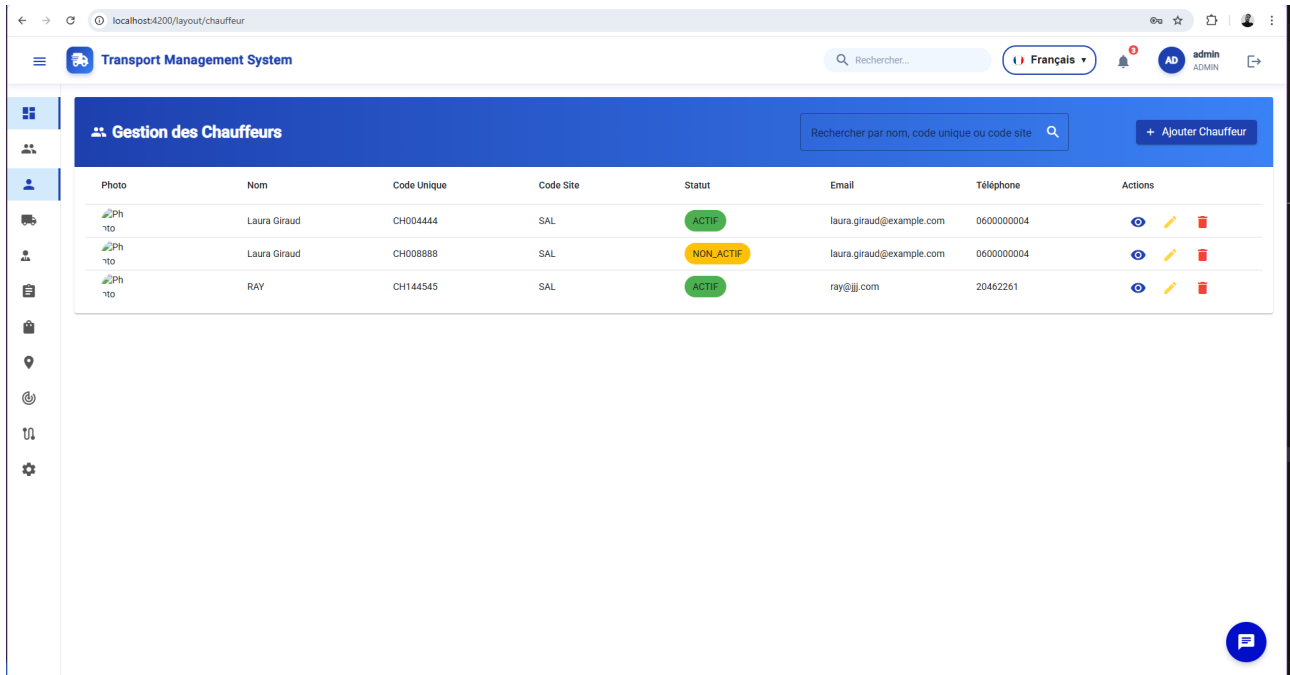


FIGURE 6.12 – Interface de gestion des chauffeurs

Le formulaire présenté ci-dessous permet d’ajouter un nouveau chauffeur, en renseignant toutes les informations nécessaires et en définissant son statut initial.

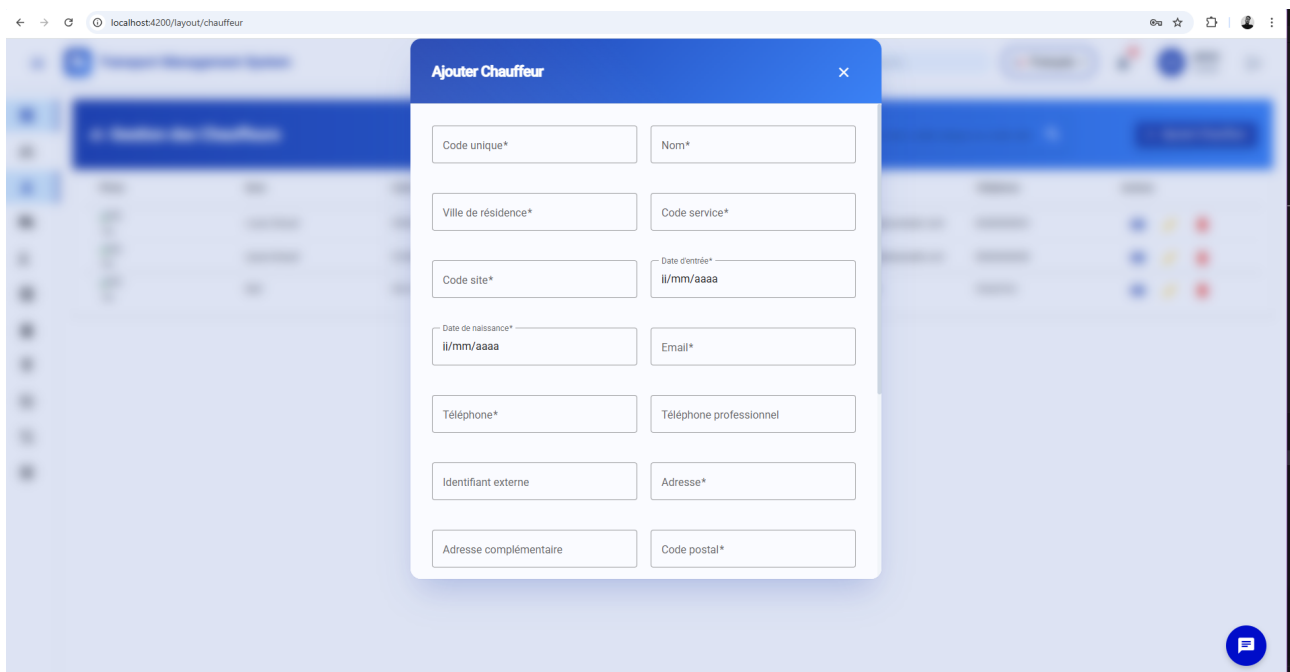


FIGURE 6.13 – Interface d’ajout d’un chauffeur

6.7 Conclusion

Le troisième sprint a été une étape majeure dans le développement de LUMTMS, en dotant l'application des fonctionnalités essentielles de gestion des chauffeurs et des camions. Les opérations CRUD complètes, la gestion des statuts et les bases pour l'assignation automatique sont désormais en place. Ces réalisations constituent le socle opérationnel nécessaire pour aborder la prochaine phase du projet : la gestion des ordres de transport et la planification des voyages, qui utiliseront directement les entités et les logiques métier implémentées au cours de ce sprint.

Chapitre 7

Sprint 4 : Gestion des Ordres de Transport (OT)

7.1 Introduction

Le quatrième sprint, se concentre sur la gestion des ordres de transport (OT), un élément central pour le bon fonctionnement du système logistique. Ce sprint vise à implémenter les fonctionnalités d'import/export des OT avec le système externe Geode, ainsi que les opérations classiques de création, modification, consultation et suppression des ordres de transport. Cette étape permet d'assurer une meilleure intégration des données et une gestion fluide des ordres dans la chaîne logistique.

7.2 Objectifs du Sprint

- Permettre aux utilisateurs (Administrateurs et Agents) d'importer et exporter les ordres de transport depuis/vers Geode, facilitant ainsi l'interopérabilité des systèmes.
- Implémenter les fonctionnalités de gestion complète des ordres de transport (CRUD).
- Assurer la qualité et la robustesse des opérations à travers des validations, et la gestion des erreurs.
- Offrir une interface utilisateur claire et intuitive pour faciliter la gestion des OT.

7.3 Spécification et Analyse des Besoins

7.3.1 Sprint Backlog

Le tableau suivant présente le Sprint Backlog du quatrième sprint, détaillant les fonctionnalités à implémenter, les scénarios utilisateurs correspondants, les tâches associées et leur estimation en termes de charge de travail.

TABLE 7.1: Sprint Backlog du Sprint 4

Fonctionnalité	ID Fonctionnalité	Scénario Utilisateur	ID Tâche	Tâches	Estimation
Gestion des OTs	US-3	En tant qu'Administrateur/Agent, je souhaite importer/exporter des ordres de transport depuis/vers Geode	T1	Développement API d'import/export avec Geode	4 j
			T2	Intégration front-end pour importer/exporter les OT	3 j
			T3	Tests unitaires et d'intégration pour import/export	2 j
Gestion des OTs	US-4	En tant qu'Administrateur/Agent, je souhaite gérer les ordres de transport (CRUD)	T4	API REST pour gestion CRUD des ordres de transport	3 j
			T5	Interface utilisateur CRUD pour les ordres de transport	4 j
			T6	Tests unitaires du service Ordre	2 j

7.3.2 Diagramme de cas d'utilisation du Sprint

Le diagramme de cas d'utilisation suivant illustre les interactions des utilisateurs avec le système pour la gestion des ordres de transport durant le Sprint 4.

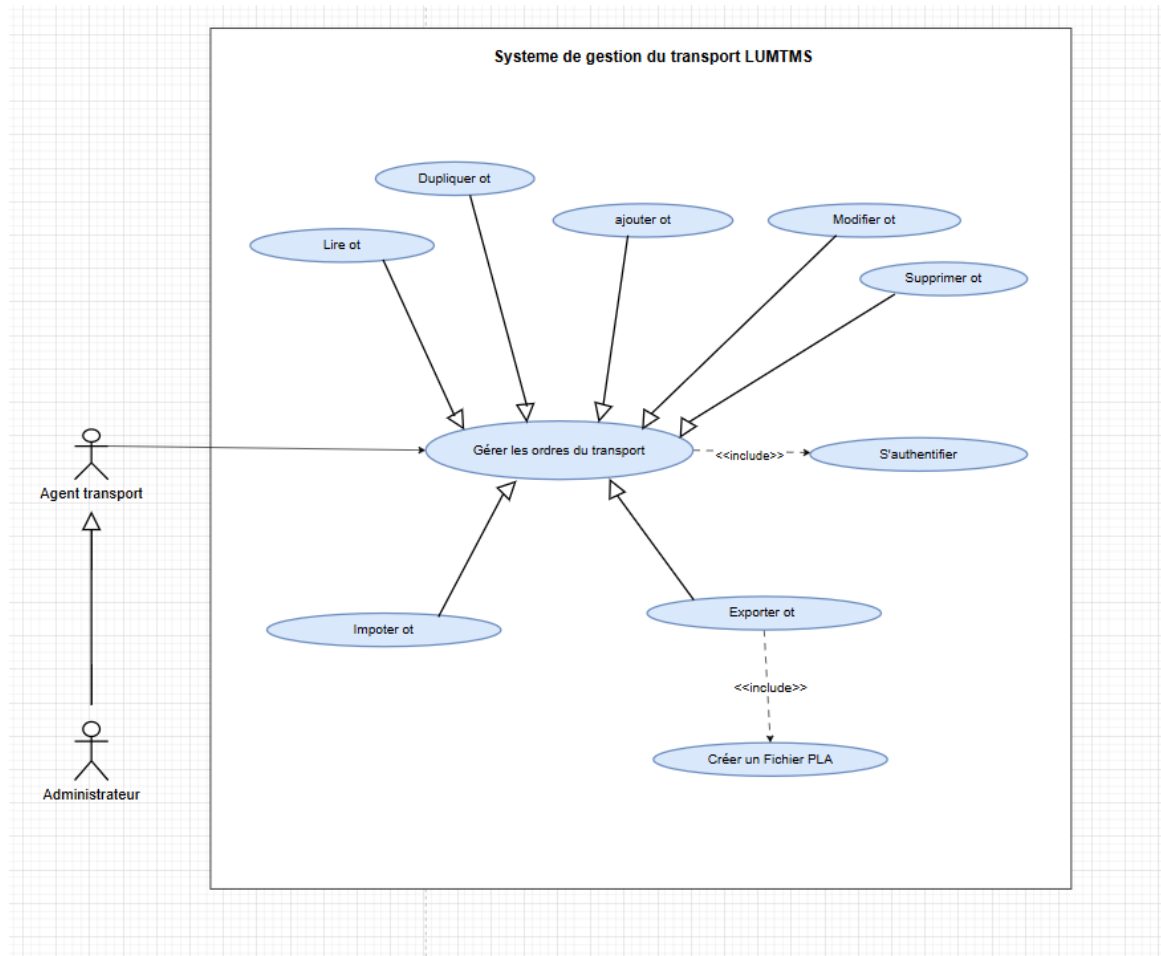


FIGURE 7.1 – Diagramme de cas d'utilisation du Sprint 4

7.4 Description textuelle des cas d'utilisation

7.4.1 Description textuelle du cas d'utilisation "Importer des Ordres de Transport (OT) depuis Geode"

Le tableau suivant décrit le cas d'utilisation pour l'importation des OT depuis Geode, en détaillant l'acteur, les préconditions, le scénario nominal et les scénarios alternatifs.

TABLE 7.2: Description textuelle du cas d'utilisation "Importation des OT"

Titre	Importation des Ordres de Transport (OT) depuis Geode
Acteur(s)	Utilisateur
Précondition(s)	L'utilisateur est authentifié et se trouve sur l'interface de gestion des OT.
Scénario nominal	1. L'utilisateur clique sur le bouton « Importer les OT ». 2. Le système vérifie la présence de fichiers d'importation (format PLA) dans le répertoire source. 3. Pour chaque fichier trouvé, le système lit son contenu. 4. Le système envoie une requête à un service de géocodage (OpenRouteService) pour obtenir les coordonnées GPS des adresses de départ et d'arrivée. 5. Le système construit et enregistre le nouvel OT dans la base de données avec le statut « NON_PLANIFIE » et les géocodes. 6. Le système déplace le fichier traité vers un répertoire d'archives pour éviter les doublons. 7. Le système notifie l'utilisateur du succès de l'importation et met à jour la liste des OT affichée.
Scénarios alternatifs/d'erreur	- Si aucun fichier d'importation n'est trouvé, le système affiche un message « Aucun nouvel OT à importer ».

7.4.2 Description textuelle du cas d'utilisation "Exporter des Ordres de Transport (OT) vers Geode"

Le tableau suivant décrit le cas d'utilisation pour l'exportation des OT vers Geode, détaillant les étapes normales et les scénarios d'erreur possibles.

TABLE 7.3: Description textuelle du cas d'utilisation "Exportation des OT"

Titre	Exportation des Ordres de Transport (OT) vers Geode
Acteur(s)	Utilisateur
Précondition(s)	L'utilisateur est authentifié et a sélectionné un ou plusieurs OT à exporter.
Scénario nominal	1. L'utilisateur clique sur le bouton « Exporter OT ». 2. Le système met à jour le statut des OT sélectionnés à « LIVRE » dans la base de données. 3. Le système génère les fichiers d'exportation (format PLA) contenant les données des OT. 4. Le système enregistre les fichiers générés dans le répertoire d'exportation dédié. 5. Le système affiche un message « Export réussi » à l'utilisateur.
Scénarios alternatifs/d'erreur	- Si une erreur survient lors de l'écriture des fichiers d'exportation, le système affiche un message « Erreur système, réessayez plus tard ».

7.4.3 Description textuelle du cas d'utilisation "Créer un Ordre de Transport (OT)"

Le tableau suivant présente les étapes pour créer un nouvel OT, avec le scénario nominal et les scénarios alternatifs possibles.

TABLE 7.4: Description textuelle du cas d'utilisation "Créer un Ordre de Transport (OT)"

Titre	Création d'un Ordre de Transport (OT)
Acteur(s)	Utilisateur
Précondition(s)	L'utilisateur est authentifié et se trouve sur l'interface de création d'OT.
Scénario nominal	1. L'utilisateur remplit le formulaire de création d'OT avec toutes les informations requises (adresses, dates, etc.). 2. L'utilisateur soumet le formulaire. 3. Le système génère un numéro d'OT unique pour l'identifier. 4. Le système initialise le nouvel OT avec le statut « NON_PLANIFIE ». 5. Le système enregistre le nouvel OT dans la base de données. 6. Le système affiche un message « OT créé avec succès ».
Scénarios alternatifs/d'erreur	- Si une erreur se produit lors de l'enregistrement en base de données, le système affiche un message « Erreur système, veuillez réessayer ».

7.4.4 Description textuelle du cas d'utilisation "Dupliquer un Ordre de Transport (OT)"

Le tableau suivant décrit le processus de duplication d'un OT existant, précisant le scénario nominal et les erreurs possibles.

TABLE 7.5: Description textuelle du cas d'utilisation "Dupliquer un OT"

Titre	Duplication d'un Ordre de Transport (OT)
Acteur(s)	Utilisateur
Précondition(s)	L'utilisateur est authentifié et a sélectionné un OT existant dans la liste.
Scénario nominal	1. L'utilisateur clique sur le bouton « Dupliquer ». 2. Le système récupère toutes les données de l'OT original sélectionné. 3. Le système génère un nouveau numéro d'OT unique pour la copie. 4. Le système réinitialise le statut de la copie à « NON_PLANIFIE ». 5. Le système enregistre la nouvelle OT (la copie) dans la base de données. 6. Le système affiche un message « Duplication réussie » et met à jour la liste des OT.
Scénarios alternatifs/d'erreur	- Si une erreur survient durant le processus (ex : lecture de l'original, écriture de la copie), le système affiche un message « Erreur lors de la duplication ».

7.4.5 Autres cas d'utilisation de gestion des OT

La gestion des OT inclut également la modification et la suppression. Ces opérations permettent à l'administrateur de mettre à jour les informations d'un OT ou de le retirer du système.

7.5 Conception

7.5.1 Diagramme de classe de sprint

Le diagramme de classes ci-dessous illustre les entités principales impliquées dans la gestion des ordres de transport et leurs relations.

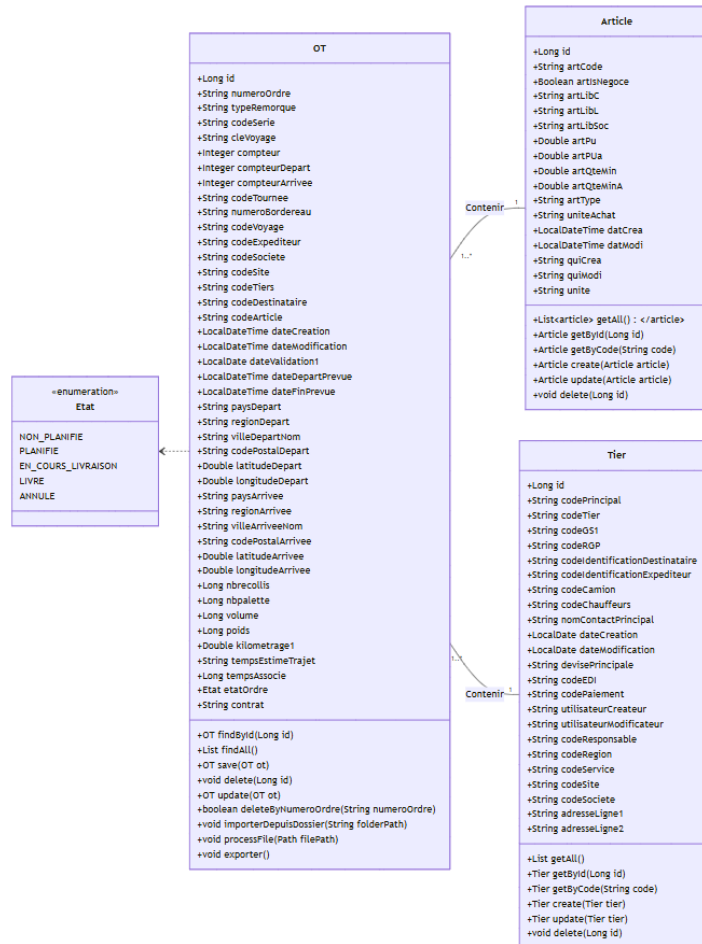


FIGURE 7.2 – Diagramme de classe du Sprint 4

7.5.2 Diagrammes de séquence

Les diagrammes suivants détaillent les interactions pour chaque cas d'utilisation de ce sprint.

7.5.2.1 Diagramme de séquence "Importer des OTs"

Ce diagramme montre le processus d'importation d'un ordre de transport à partir d'un fichier externe.

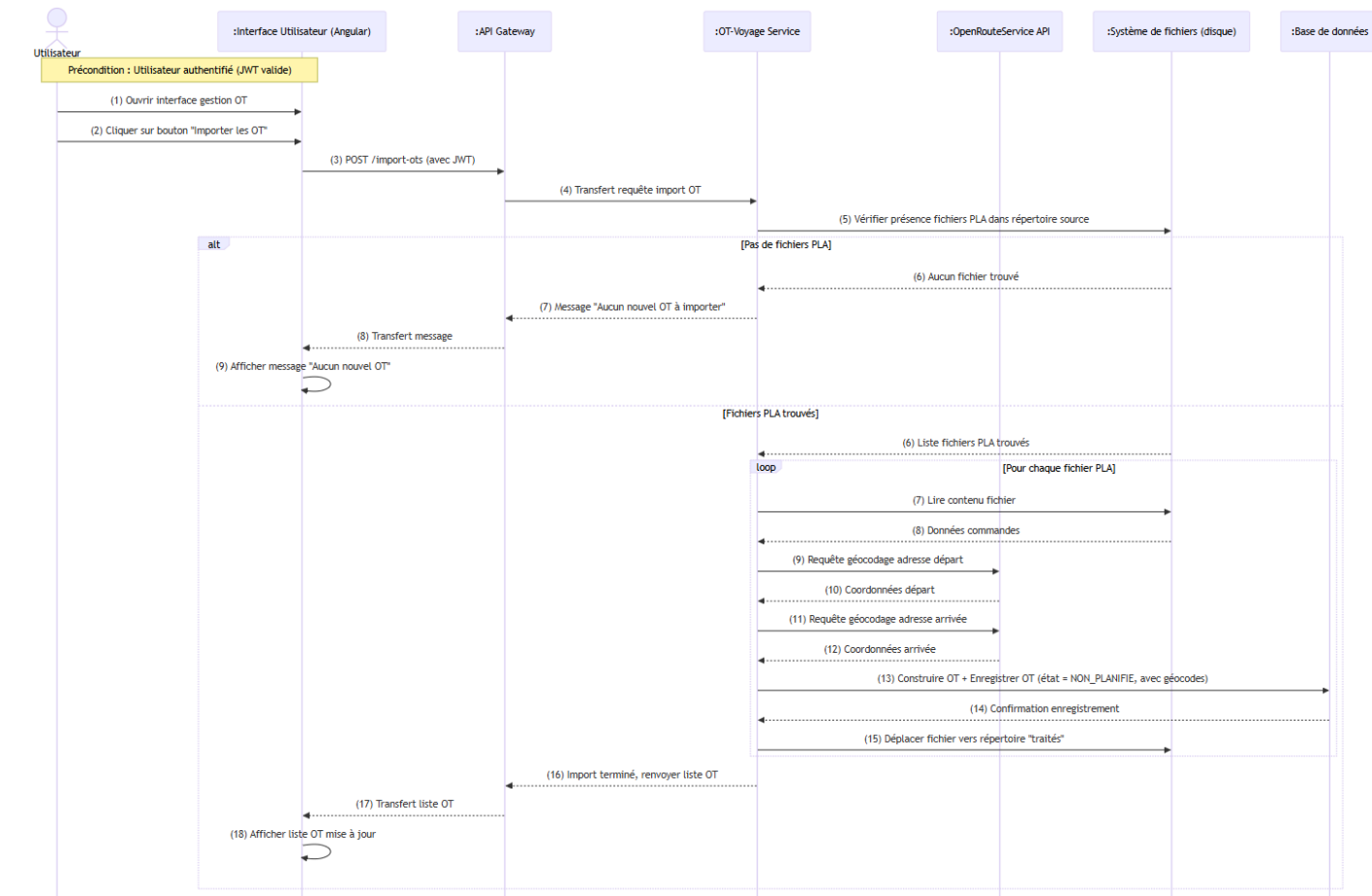


FIGURE 7.3 – Diagramme de séquence - Importation des OT

7.5.3 Diagramme de séquence de "Exporter des OTs"

Ce diagramme illustre comment un ordre de transport est exporté vers un système externe.

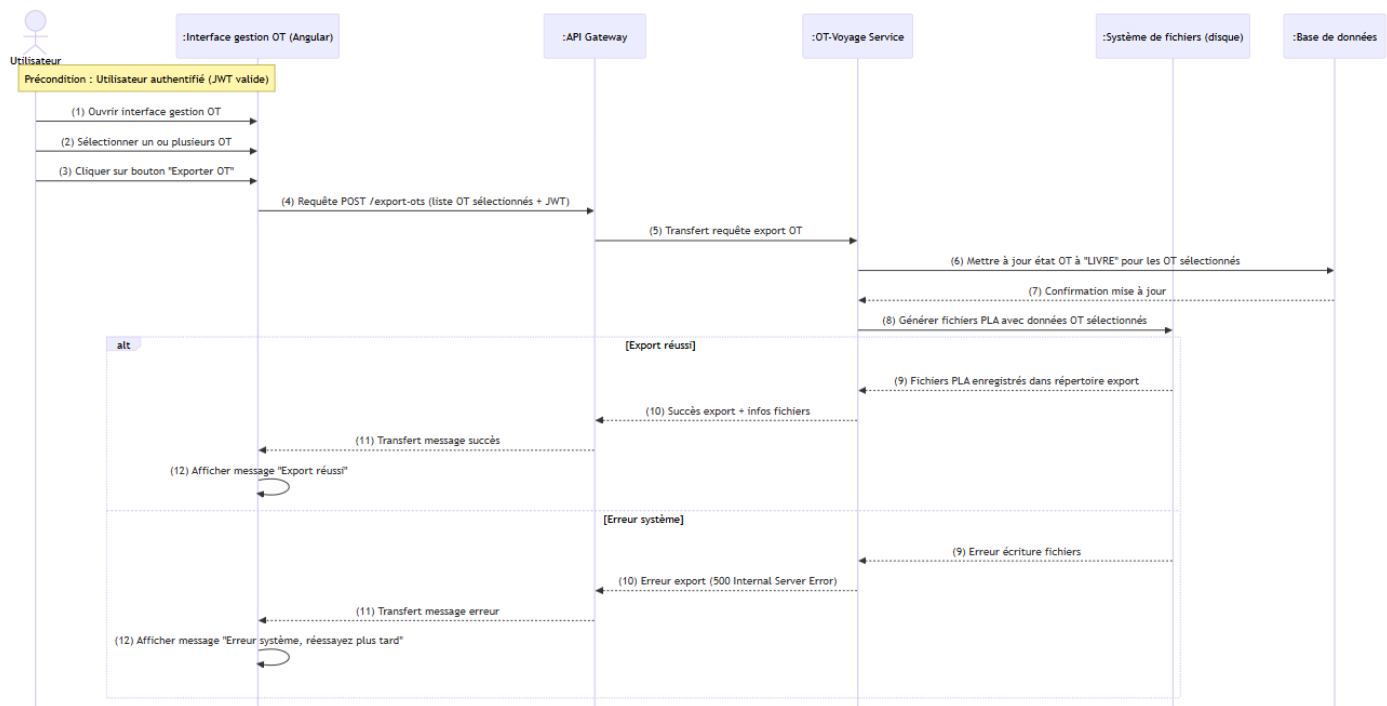


FIGURE 7.4 – Diagramme de séquence - Exportation des OT

7.5.4 Diagramme de séquence "Créer un OT"

Ce diagramme détaille les étapes de la création manuelle d'un nouvel ordre de transport.

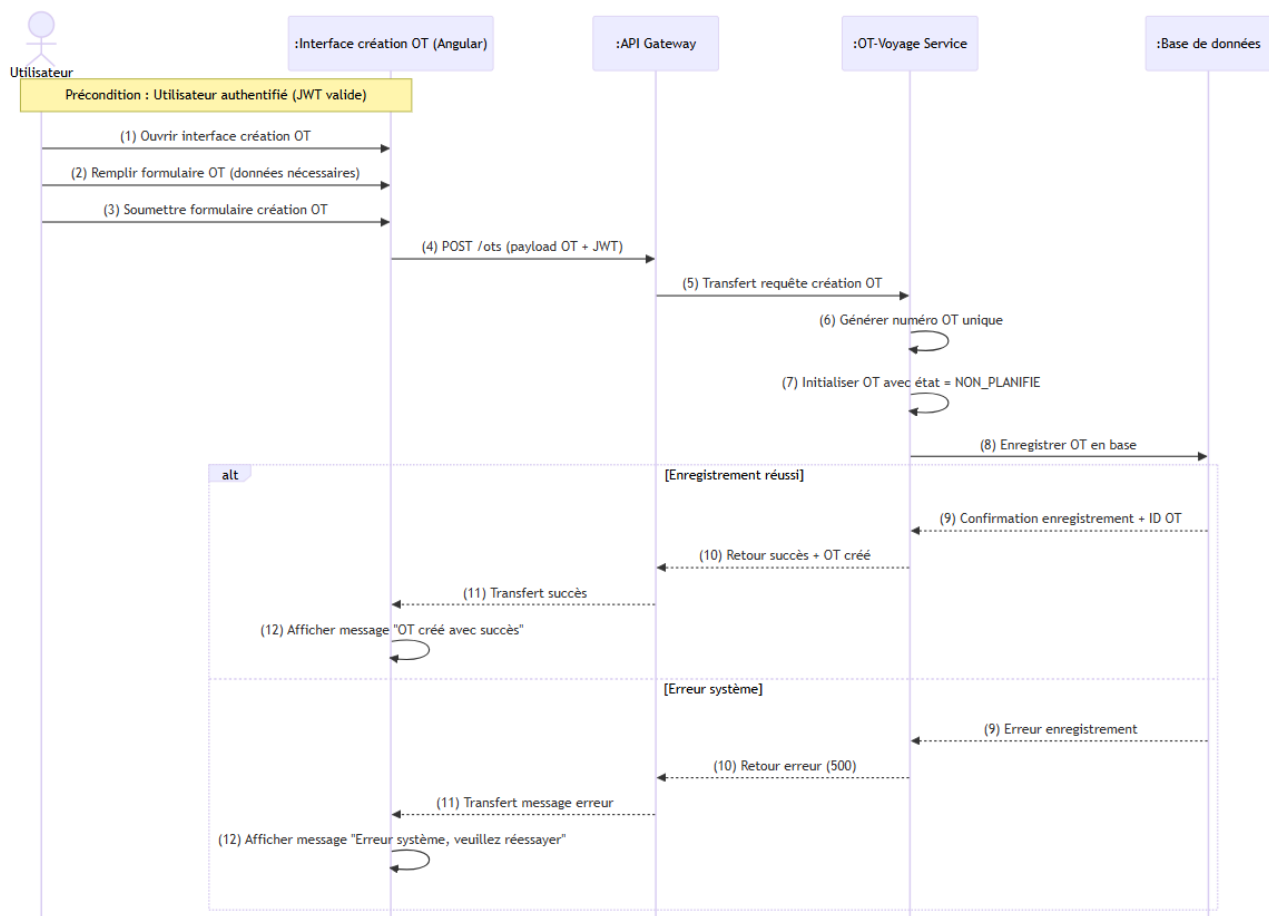


FIGURE 7.5 – Diagramme de séquence - Création d'un OT

7.5.5 Diagramme de séquence "dupliquer un OT"

Ce diagramme montre le processus de duplication d'un ordre de transport existant pour en créer un nouveau.

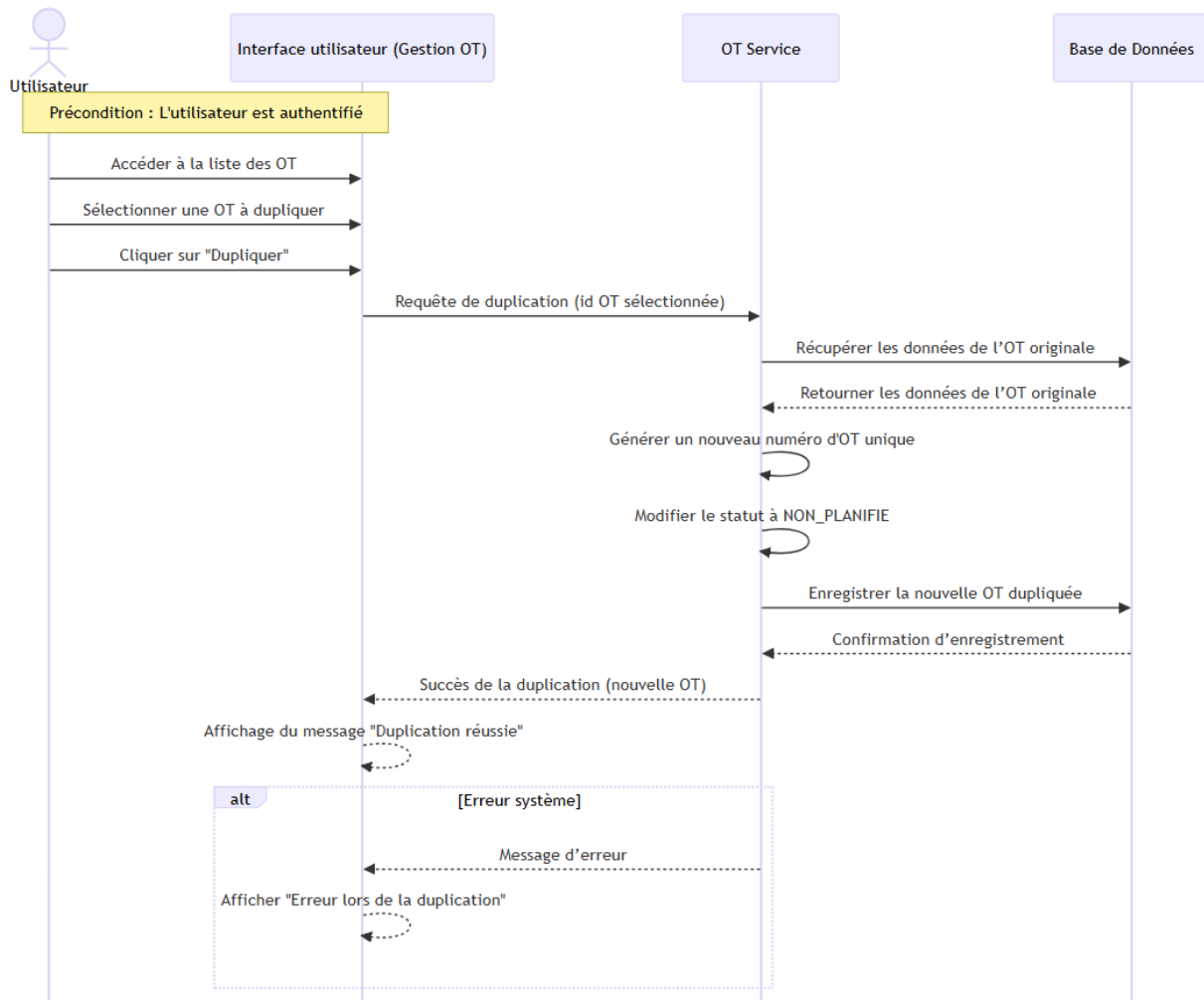


FIGURE 7.6 – Diagramme de séquence - Duplication d'un OT

7.6 Réalisation

L'implémentation des fonctionnalités de gestion des ordres de transport (OT) a abouti à la création de plusieurs interfaces utilisateur clés, conçues pour simplifier les opérations quotidiennes des administrateurs et des agents.

7.6.1 Interface de gestion des OT

L’interface principale de gestion des OT offre une vue centralisée de tous les ordres de transport. Elle permet aux utilisateurs de consulter, rechercher, et trier les OT, ainsi que d’accéder aux fonctionnalités d’import/export, de création, et de duplication.

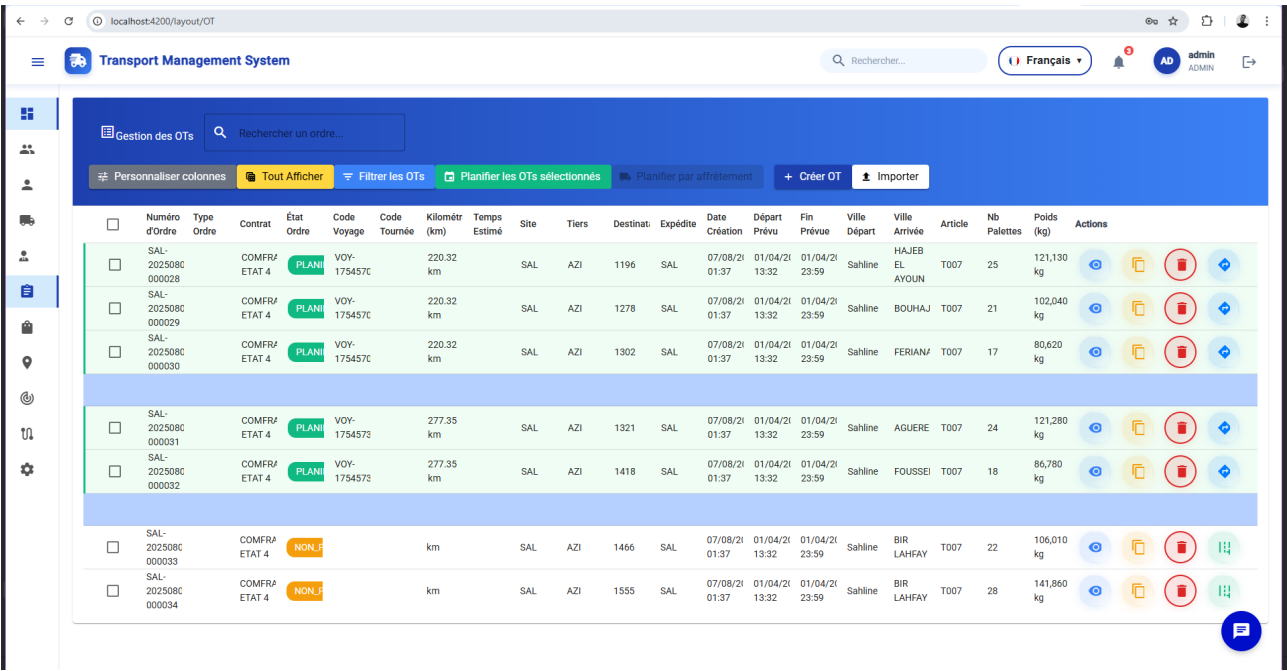


FIGURE 7.7 – Interface de gestion des Ordres de Transport

7.6.2 Interface de création d’un OT

Le formulaire de création d’un OT a été conçu pour être à la fois complet et intuitif, guidant l’utilisateur dans la saisie de toutes les informations nécessaires pour un ordre de transport, telles que les adresses, les dates, et les détails de la marchandise.

← → localhost:4200/layout/OT

Créer un nouvel Ordre de Transport

Informations de base

Número d'Ordre **

OT-1755258163366-538

Type d'Ordre

Code Site **

Code Tournée

Transport

Type Transport

Type Remorque

Parties impliquées

Code Expéditeur

Code Destinataire **

Annuler

Créer OT

FIGURE 7.8 – Interface de création d'un Ordre de Transport

7.6.3 Structure d'un fichier PLA

L'intégration avec le système externe Geode repose sur l'échange de fichiers au format PLA. La figure ci-dessous montre un exemple de la structure de ces fichiers, qui contiennent les informations essentielles des ordres de transport pour l'import et l'export.

[illegible]

FIGURE 7.9 – Exemple de structure d'un fichier d'importation PLA

7.7 Conclusion

Le quatrième sprint a été une étape fondamentale dans le développement de LUMTMS, en introduisant la gestion complète des ordres de transport. L'intégration réussie avec le système Geode via les fichiers PLA, ainsi que la mise en place des opérations CRUD, dotent l'application d'une fonctionnalité métier essentielle. Ces avancées permettent désormais de centraliser la gestion des transports et préparent le terrain pour le Sprint 5, qui se concentrera sur la planification de ces ordres en voyages et tournées optimisées.

Chapitre 8

Sprint 5 – Gestion des voyages et des tournées génériques

8.1 Introduction

Le cinquième sprint du projet, couvrant la période du 26 avril au 23 mai (soit quatre semaines), est consacré à la gestion des voyages et des tournées génériques. Ce sprint vise à enrichir le système en intégrant la planification avancée des ordres de transport, l’optimisation des itinéraires et la communication proactive avec les chauffeurs. Il constitue une étape essentielle pour améliorer l’efficacité logistique, réduire les coûts de transport et automatiser certaines tâches administratives.

8.2 Objectifs du sprint

Les objectifs principaux de ce sprint sont :

- Mettre en place la création de voyages à partir des ordres de transport regroupés.
- Intégrer OpenRouteService (ORS) pour calculer des itinéraires optimisés.
- Permettre la visualisation des trajets sur une carte et la génération de rapports PDF.
- Automatiser l’envoi de notifications SMS aux chauffeurs lors de la création d’un voyage.
- Développer la gestion des tournées génériques (CRUD), permettant la génération automatique de voyages à partir de modèles prédéfinis.

8.3 Analyse et spécification

Le sprint introduit des fonctionnalités liées à la planification et au suivi des voyages, en s'appuyant sur les OT (ordres de transport). Les tournées génériques permettent de définir des trajets répétitifs qui serviront de base pour créer des voyages réels, réduisant ainsi la charge de planification manuelle.

8.3.1 Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog du Sprint 5, détaillant les fonctionnalités prévues, les scénarios utilisateur, les tâches associées et leur estimation.

TABLE 8.1: Sprint Backlog du Sprint 5

ID Fonctionnalité	Scénario Utilisateur	ID Tâche	Tâches	Estimation
US-5	En tant qu'Administrateur/Agent, je souhaite regrouper et planifier les OT pour créer des voyages	T1	Développement API de regroupement des OT en voyages	3 j
		T2	Interface front-end pour création manuelle d'un voyage	3 j
		T3	Tests unitaires service Voyage	2 j
US-7	En tant qu'Administrateur/Agent, je souhaite calculer des itinéraires optimisés via ORS	T4	Intégration API OpenRouteService pour calculer les itinéraires	4 j
		T5	Développement service optimisation itinéraires	3 j
US-8	En tant qu'Administrateur/Agent, je souhaite visualiser les trajets sur une carte et générer des rapports PDF	T6	Intégration front-end avec carte (OpenStreetMap/Leaflet)	4 j

Suite à la page suivante

Tableaux 8.1 suite

ID Fonction-nalité	Scénario Utilisateur	ID Tâche	Tâches	Estimation
		T7	Génération de rapports PDF (itinéraires et détails des voyages)	3 j
US-9	En tant qu'Administrateur/Agent, je souhaite que le système envoie automatiquement des SMS au chauffeur lors de la création d'un voyage	T8	Intégration API SMS (Twilio ou autre)	2 j
		T9	Déclenchement automatique envoi SMS à la création d'un voyage	2 j
US-6	En tant qu'Administrateur/Agent, je souhaite créer automatiquement des voyages depuis des tournées génériques	T10	Développement API de génération automatique de voyages	3 j
		T11	Tests unitaires et d'intégration génération voyages	2 j
US-22	En tant qu'Administrateur, je souhaite gérer les tournées génériques (CRUD)	T12	API REST CRUD tournées génériques	3 j
		T13	Interface front-end CRUD tournées génériques	4 j

8.3.2 Diagramme de cas d'utilisation de sprint 5

Le diagramme ci-dessous présente les cas d'utilisation couverts par le Sprint 5, axés sur la gestion des voyages et des tournées.

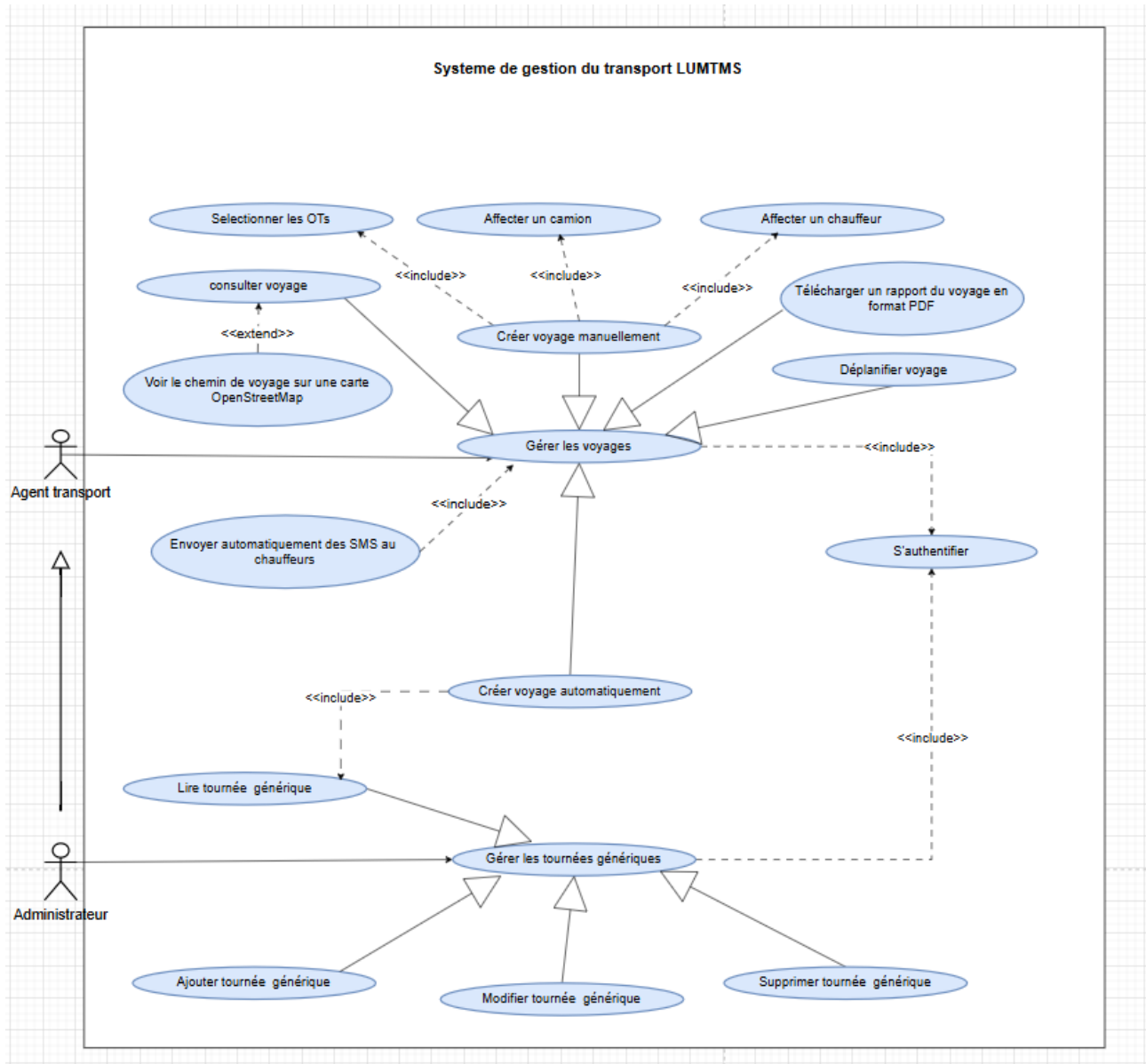


FIGURE 8.1 – Diagramme des cas d'utilisation du Sprint 5

8.4 Description textuelle des cas d'utilisation

Cette section présente la description détaillée de trois cas d'utilisation principaux du système de des voyages et destournées génériques, choisis pour leur importance dans le fonctionnement de l'application.

8.4.1 Description textuelle du cas d'utilisation "Créer manuellement un voyage"

Le tableau ci-dessous décrit le processus complet de création manuelle d'un voyage, incluant les acteurs, les préconditions, le scénario nominal et les scénarios alternatifs.

TABLE 8.2: Description textuelle du cas d'utilisation "Créer manuellement un voyage"

Titre	Créer manuellement un voyage
Acteur(s)	Utilisateur(Administrateur ou Agent transport)
Précondition(s)	L'utilisateur est authentifié et des ordres de transport (OT) sont disponibles.
Postcondition	Le voyage est créé et enregistré dans le système.
Scénario nominal	1. L'utilisateur sélectionne un ou plusieurs OT à planifier. 2. Le système propose une liste de chauffeurs et de camions compatibles (disponibilité, capacité, etc.). 3. L'utilisateur sélectionne un chauffeur et un camion parmi les suggestions. 4. Le système calcule un itinéraire optimisé via OpenRoute-Service, incluant la distance et le temps estimé. 5. Le système met à jour le statut du chauffeur et du camion. 6. Le système enregistre le nouveau voyage et met à jour le statut des OT associés à « PLANIFIE ». 7. Le système envoie automatiquement une notification SMS au chauffeur avec les détails du voyage. 8. L'interface est mise à jour pour afficher le voyage nouvellement créé.
Scénarios alternatifs/d'erreur	- Si aucun chauffeur ou camion n'est compatible, un message d'erreur est affiché.

8.4.2 Description textuelle du cas d'utilisation "Déplanifier un voyage"

Le tableau suivant détaille le cas d'utilisation de déplanification d'un voyage, indiquant les étapes, les acteurs et les conditions d'erreur possibles.

TABLE 8.3: Description textuelle du cas d'utilisation "Déplanifier un voyage"

Titre	Déplanifier un voyage
Acteur(s)	Utilisateur(Administrateur ou Agent transport)
Précondition(s)	L'utilisateur est authentifié et un voyage est à l'état « PLANIFIE ».
Scénario nominal	1. L'utilisateur sélectionne un voyage planifié et clique sur « Déplanifier ». 2. Le système remet le statut du chauffeur et du camion à « DISPONIBLE ». 3. Le système remet le statut des OT associés au voyage à « NON-PLANIFIE ». 4. Le système annule ou supprime le voyage de la base de données. 5. L'interface est mise à jour pour refléter l'annulation.
Scénarios alternatifs/d'erreur	- Si le voyage n'est plus à l'état « PLANIFIE », l'action est refusée.

8.4.3 Description textuelle du cas d'utilisation "Créer une tournée générique"

Le tableau ci-dessous présente le processus de création d'une tournée générique, avec ses étapes principales et les vérifications nécessaires pour assurer l'unicité du code de la tournée.

TABLE 8.4: Description textuelle du cas d'utilisation "Créer une tournée générique"

Titre	Créer une tournée générique
Acteur(s)	Administrateur
Précondition(s)	L'administrateur est authentifié.

Scénario nominal	1. L'administrateur accède à l'interface de gestion des tournées génériques. 2. Il remplit le formulaire de création avec un code de tournée, un code de site, une liste de clients et les articles associés. 3. Le système vérifie que le code de la tournée n'est pas déjà utilisé. 4. La nouvelle tournée générique est enregistrée dans la base de données. 5. Un message de succès est affiché et la liste des tournées est mise à jour.
Scénarios alternatifs/d'erreur	- Si le code de la tournée existe déjà, le système affiche un message d'erreur « Code tournée déjà existant ».

8.4.4 Description textuelle du cas d'utilisation "Création automatique d'un voyage"

La table suivante décrit le cas d'utilisation de création automatique de voyages à partir des tournées génériques, détaillant le scénario nominal, les postconditions et les scénarios alternatifs. Ce cas d'utilisation décrit comment le système traite automatiquement les ordres de transport (OT) nouvellement importés pour les regrouper dans des voyages en se basant sur des tournées génériques prédéfinies.

TABLE 8.5: Description textuelle du cas d'utilisation "Création automatique d'un voyage"

Titre	Création automatique d'un voyage à partir des tournées génériques
Acteur(s)	Système (déclenché par l'Utilisateur via l'importation)
Précondition(s)	L'utilisateur est authentifié. De nouveaux OT sont importés. Des tournées génériques sont configurées.
Postcondition	Les OT correspondants à des tournées sont regroupés dans des voyages. Les autres OT restent non planifiés.

Scénario nominal	1. L'utilisateur lance l'importation de nouveaux OT. 2. Pour chaque OT importé, le système recherche une tournée générique correspondante (basée sur le site, client, etc.). 3. Si une correspondance est trouvée, le système vérifie si un voyage pour cette tournée existe déjà à la date du jour. 4. Si un voyage existe, le système y ajoute l'OT. 5. Si aucun voyage n'existe, le système en crée un nouveau et y associe l'OT. 6. Si aucune correspondance n'est trouvée, l'OT reste non planifié. 7. L'interface est mise à jour pour afficher les OT regroupés par voyage, prêts à être finalisés par l'utilisateur.
Scénarios alternatifs/d'erreur	- Si une erreur survient durant l'importation ou la communication avec la base de données, le processus est interrompu et une notification est affichée.

8.4.5 Autres cas d'utilisation

La gestion des tournées génériques inclut également la modification et la suppression, permettant à l'administrateur d'ajuster ou de retirer des modèles de tournées existants. De plus, le système offre la possibilité de télécharger un rapport de voyage détaillé. Cette fonctionnalité s'appuie sur un modèle prédéfini dans Jaspersoft Studio pour générer un document PDF récapitulant toutes les informations pertinentes du voyage.

8.5 Conception

La conception de ce sprint s'articule autour des entités Voyage et TournéeGénérique, en introduisant la logique d'optimisation et de planification.

8.5.1 Diagramme de classe de sprint

Le diagramme de classes ci-dessous illustre les nouvelles entités et leurs relations avec les entités existantes comme les Ordres de Transport, les Chauffeurs et les Camions.

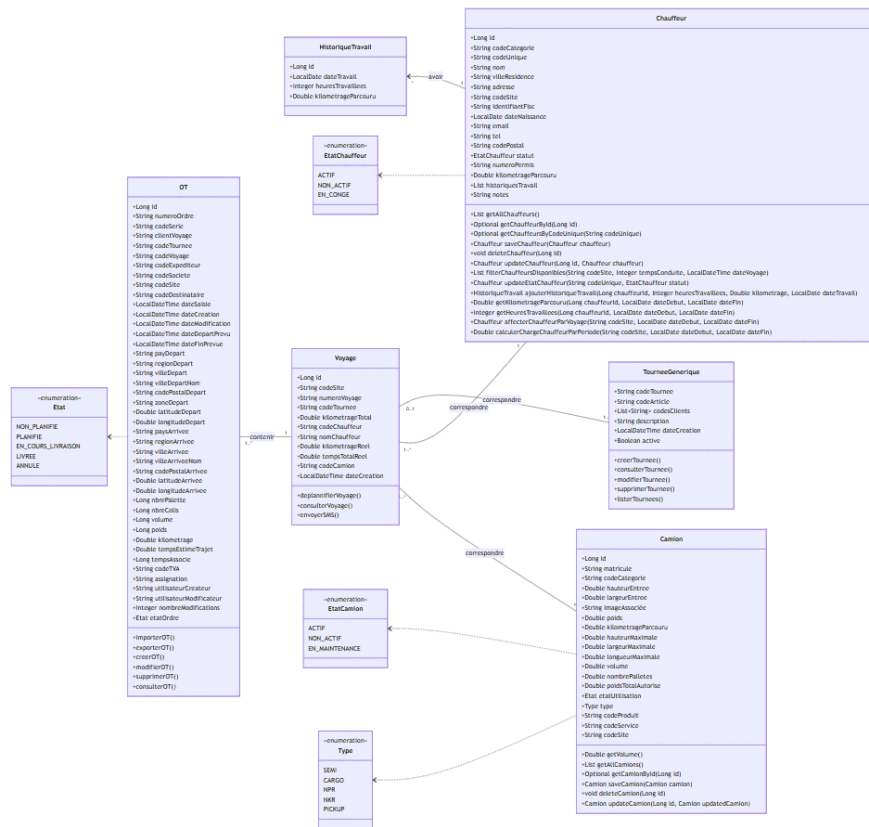


FIGURE 8.2 – Diagramme de classe du Sprint 5

8.5.2 Diagrammes de séquence

Les diagrammes suivants détaillent les interactions pour les principaux cas d'utilisation de ce sprint.

8.5.2.1 Diagramme de séquence “Créer un voyage”

Ce diagramme montre le processus de création manuelle d'un voyage, depuis la sélection des OT jusqu'à l'assignation d'un chauffeur et d'un camion.

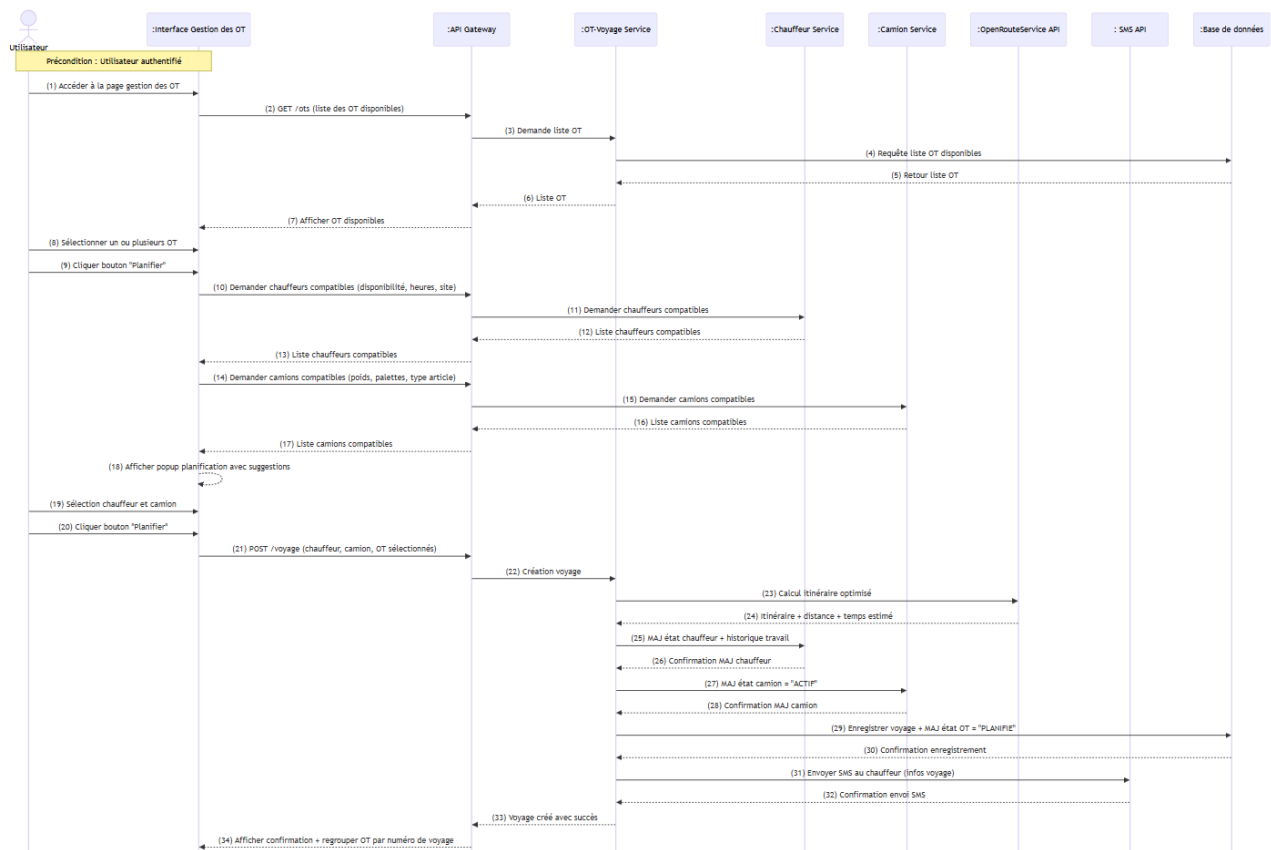


FIGURE 8.3 – Diagramme de séquence – Création d'un voyage

8.5.2.2 Diagramme de séquence “Déplanifier un voyage”

Ce diagramme illustre les étapes d'annulation d'un voyage planifié, incluant la libération des ressources (chauffeur, camion) et la réinitialisation du statut des OT.

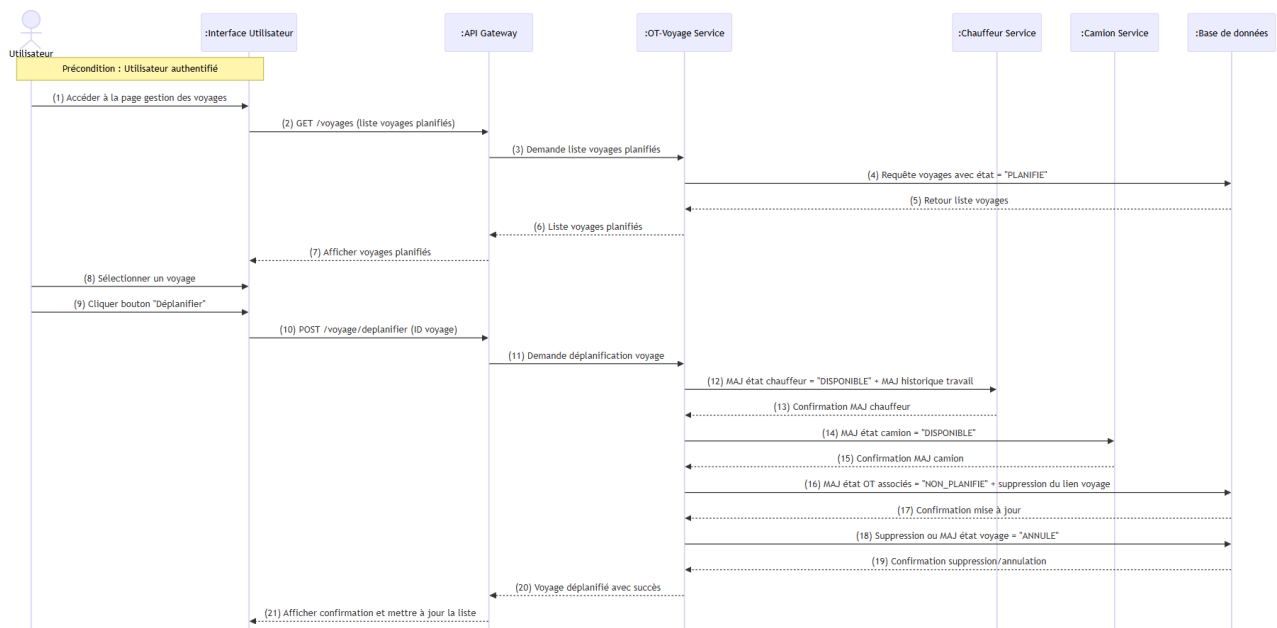


FIGURE 8.4 – Diagramme de séquence – Déplanification d'un voyage

8.5.2.3 Diagramme de séquence “Créer une tournée générique”

Ce diagramme détaille la procédure de création d'une tournée générique par l'administrateur, servant de modèle pour les voyages futurs.

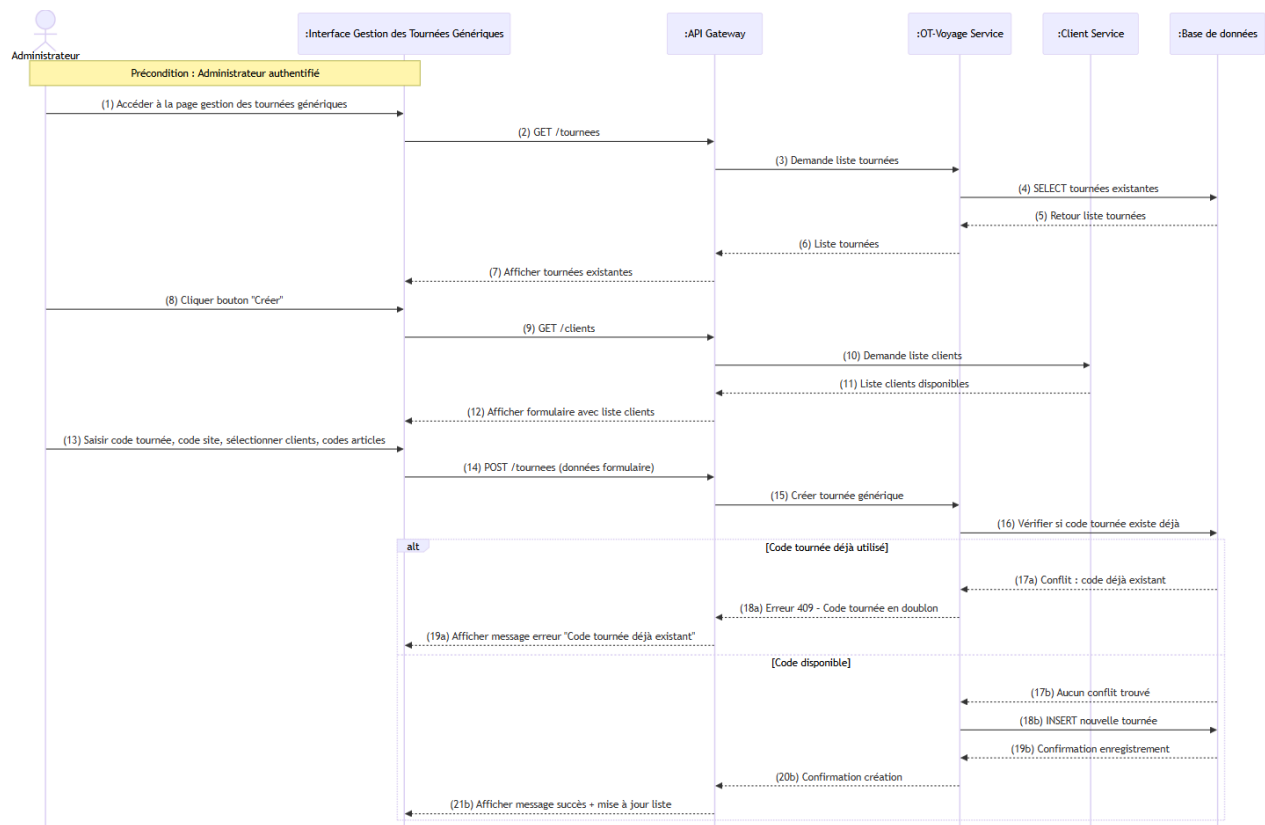


FIGURE 8.5 – Diagramme de séquence – Création d’une tournée générique

8.5.2.4 Diagramme de séquence “Création automatique d’un voyage à partir d’une tournée générique”

Ce diagramme détaille comment le système traite les OT importés pour les regrouper automatiquement dans des voyages en se basant sur les tournées génériques.

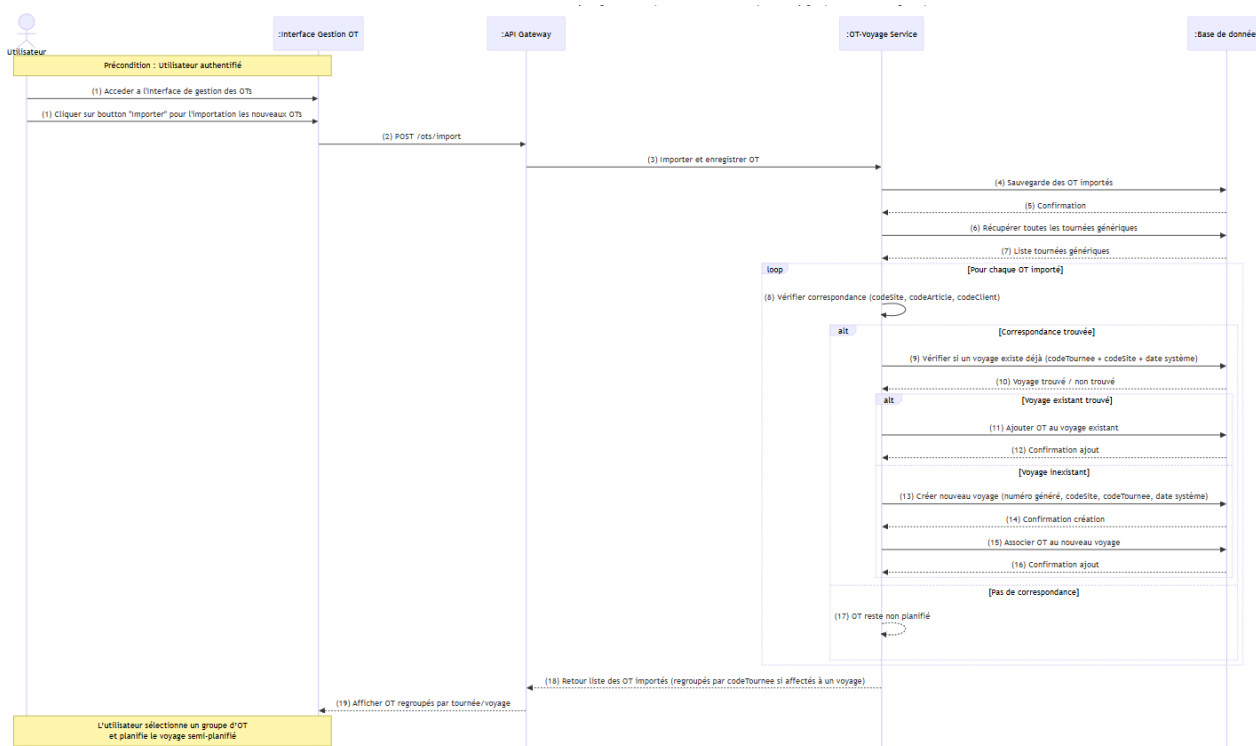


FIGURE 8.6 – Diagramme de séquence – Création automatique d'un voyage

8.6 Réalisation

La mise en œuvre des fonctionnalités de ce sprint a permis de doter l'application d'outils puissants pour la planification et l'automatisation des transports. Les interfaces ont été conçues pour offrir une expérience utilisateur fluide et efficace.

8.6.1 Interfaces de planification des voyages

Ces interfaces permettent aux utilisateurs de visualiser les voyages planifiés, de suivre leurs états et d'accéder aux détails de chaque itinéraire. Elles servent de tableaux de bord centraux pour la gestion des opérations de transport.

La figure suivante présente l'interface de planification des voyages, avec la liste des voyages planifiés et les options de suivi.

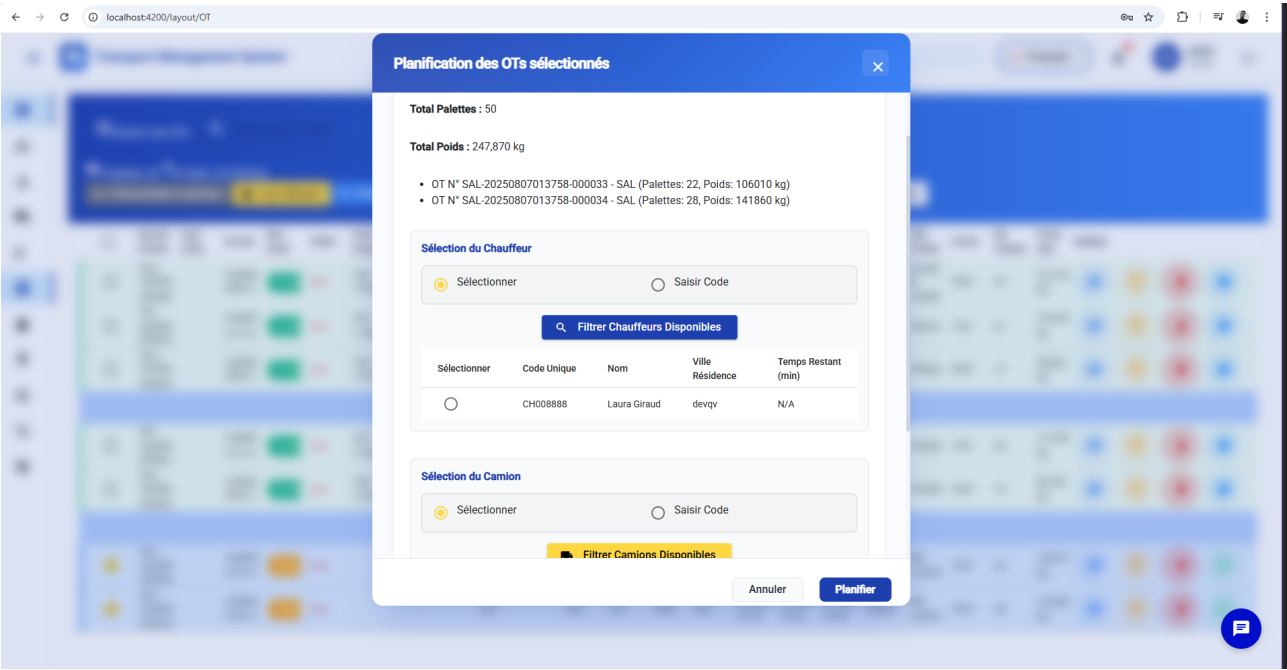


FIGURE 8.7 – Interface de planification des voyages

La figure suivante illustre l’interface de gestion des voyages, permettant de consulter et modifier les détails des voyages existants.

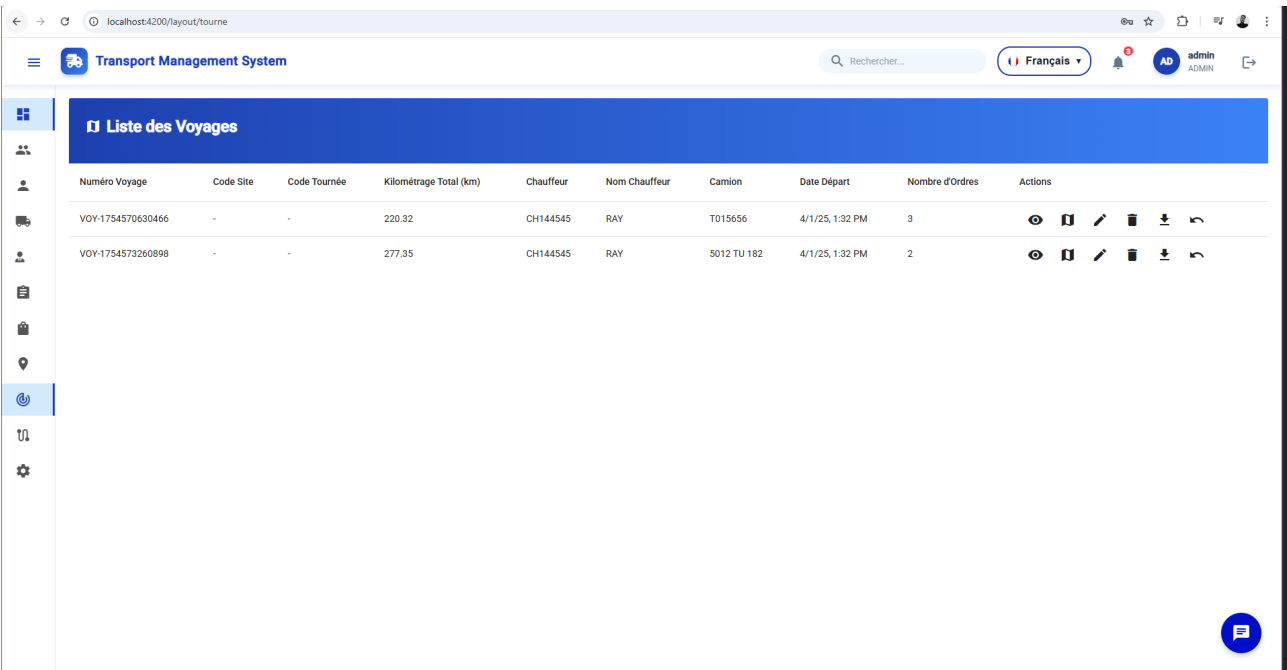


FIGURE 8.8 – Interface de gestion des voyages

La figure suivante montre l’affichage des itinéraires sur la carte pour chaque voyage planifié, incluant les chemins et points de passage.

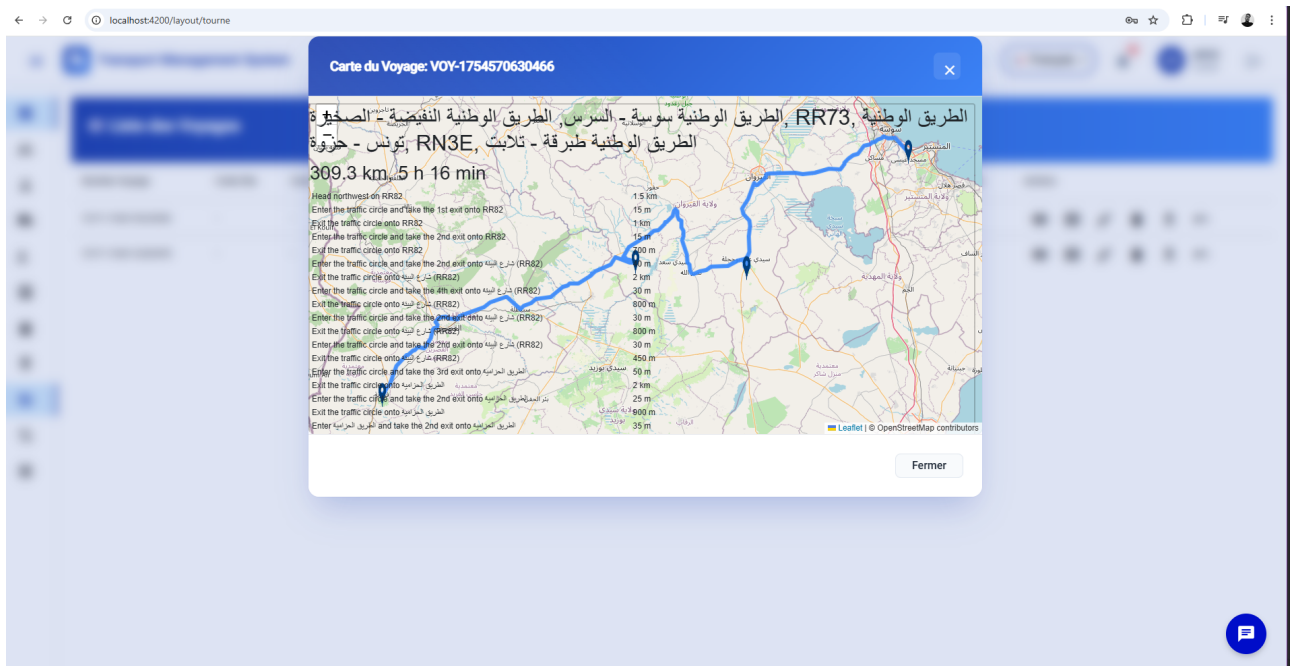


FIGURE 8.9 – Interface du chemin du voyage sur carte

8.6.2 Interface de gestion des tournées génériques

Cette interface permet aux administrateurs de créer, modifier et supprimer des modèles de tournées, servant de base à la génération automatique des voyages.

La figure suivante présente l’interface de gestion des tournées génériques.

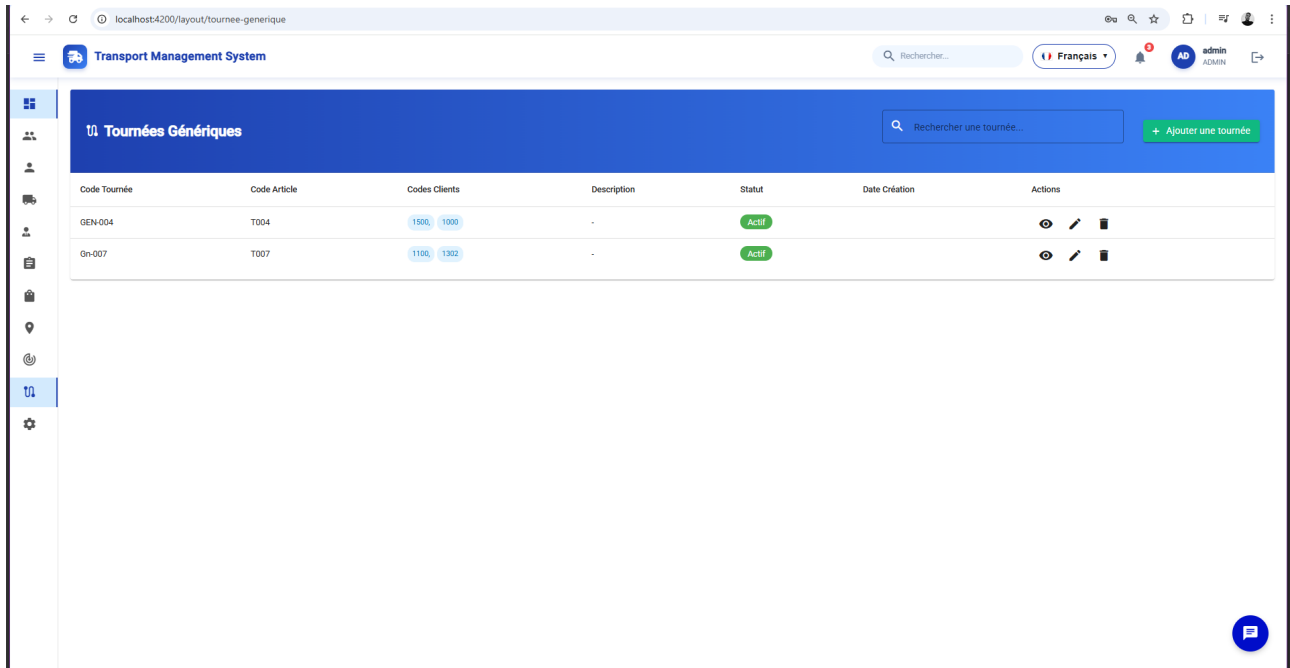


FIGURE 8.10 – Interface de gestion des tournées génériques

8.7 Conclusion

Le cinquième sprint marque une avancée majeure pour LUMTMS en transformant l'application d'un simple outil de gestion de données en une plateforme de planification logistique intelligente. L'intégration de l'optimisation d'itinéraires via OpenRouteService, la création de voyages, et l'automatisation via les tournées génériques permettent des gains d'efficacité significatifs.

Chapitre 9

Sprint 6 : Chatbot et Outils de consultation et d'analyse

9.1 Introduction

Le sixième et dernier sprint du projet, qui s'est déroulé sur quatre semaines, du 24 mai au 20 juin, se concentre sur l'ajout de fonctionnalités à haute valeur ajoutée. Il vise à transformer l'application en un véritable outil d'aide à la décision en introduisant un tableau de planning journalier, un tableau de bord analytique interactif, et un chatbot intelligent pour simplifier l'accès à l'information.

9.2 Objectif du sprint

L'objectif de ce sprint est de finaliser l'application en la dotant d'outils avancés pour l'utilisateur. Ces fonctionnalités visent à offrir une vue d'ensemble claire des opérations, à fournir des insights stratégiques grâce à des indicateurs de performance clés (KPIs), et à permettre une interrogation intuitive des données via un agent conversationnel, rendant l'information accessible sans effort.

9.3 Analyse et spécification

9.3.1 Diagramme des cas d'utilisation

Le diagramme ci-dessous présente les principaux cas d'utilisation développés durant ce sprint, illustrant les interactions de l'utilisateur avec les nouvelles fonctionnalités de planification, d'analyse et de conversation.

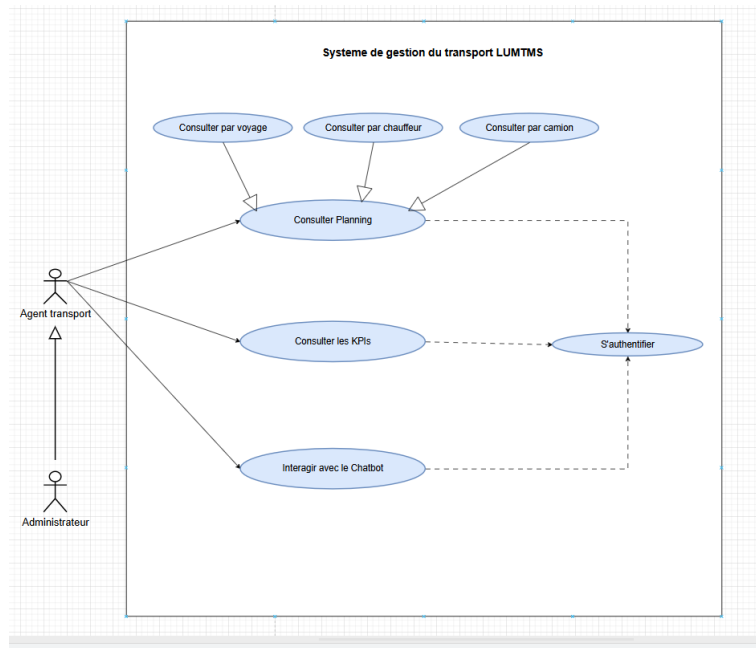


FIGURE 9.1 – Diagramme des cas d'utilisation du Sprint 6

9.3.2 Backlog du sprint

Le backlog de ce sprint final est défini dans le tableau ci-dessous.

TABLE 9.1: Backlog du Sprint 6

ID Fonct.	Scénario Utilisateur	ID Tâche	Tâches	Estim.
US-23	En tant qu'utilisateur, je souhaite consulter un tableau de planification journalier.	T-45	Développer l'interface du tableau.	2 j
		T-46	Créer le point de terminaison backend.	2 j
		T-47	Intégrer et tester le planning.	1 j
US-24	En tant qu'utilisateur, je souhaite consulter un tableau de bord avec graphiques.	T-48	Concevoir les requêtes SQL pour les KPIs.	2 j
		T-49	Développer les points de terminaison pour les KPIs.	2 j

ID Fonct.	Scénario Utilisateur	ID Tâche	Tâches	Estim.
		T-50	Intégrer Chart.js dans le frontend.	2 j
		T-51	Connecter les graphiques aux données.	1 j
US-25	En tant qu'utilisateur, je souhaite interagir avec un chatbot pour obtenir de l'aide.	T-52	Configurer le script du chatbot.	4 j
		T-53	Créer le service d'orchestration.	3 j
		T-54	Développer l'interface du chatbot.	2 j
		T-55	Tester l'intégration de bout en bout.	2 j

9.3.3 Description textuelle des cas d'utilisation

9.3.3.1 Description textuelle du cas d'utilisation "Consulter le planning journalier"

Le tableau suivant décrit le scénario complet pour consulter le planning journalier, incluant les étapes et conditions préalables à l'affichage du planning.

TABLE 9.2: Description textuelle - Consulter le planning journalier

Titre	Consulter le planning journalier
Acteur(s)	Administrateur, Agent
Précondition(s)	L'utilisateur est authentifié.
Postcondition	Le planning des activités de la journée sélectionnée est affiché.

Scénario nominal	1. L'utilisateur accède à la page de planification. 2. Il saisit ou sélectionne une date. 3. Il choisit un mode de regroupement (par voyage, chauffeur ou camion). 4. Le système affiche un tableau horaire (de 00 :00 à 23 :00) montrant les voyages planifiés pour la date et le regroupement choisis.
Scénarios alternatifs/d'erreur	- Si aucune donnée n'est disponible pour la date sélectionnée, un message l'indique.

9.3.3.2 Description textuelle du cas d'utilisation "Consulter le tableau de bord"

Le tableau suivant présente la consultation du tableau de bord analytique, montrant les KPIs et graphiques interactifs pour la période et le site sélectionnés.

TABLE 9.3: Description textuelle - Consulter le tableau de bord

Titre	Consulter le tableau de bord analytique
Acteur(s)	Administrateur, Agent
Précondition(s)	L'utilisateur est authentifié.
Postcondition	Les KPIs et graphiques sont affichés pour la période et le site sélectionnés.
Scénario nominal	1. L'utilisateur accède à la page du tableau de bord. 2. Il utilise les filtres pour sélectionner une plage de dates et un code site. 3. Le système charge et affiche les graphiques interactifs (Chart.js) représentant les KPIs : nombre d'OT, de voyages, de palettes, taux de planification, utilisation des ressources, etc.
Scénarios alternatifs/d'erreur	- Si les données ne peuvent pas être chargées, un message d'erreur est affiché.

9.3.3.3 Description textuelle du cas d'utilisation "Interagir avec le chatbot"

Le tableau suivant décrit le scénario pour interagir avec le chatbot, incluant la saisie de la question par l'utilisateur, le traitement backend et l'affichage de la réponse.

TABLE 9.4: Description textuelle - Interagir avec le chatbot

Titre	Interagir avec le chatbot
Acteur(s)	Administrateur, Agent
Précondition(s)	L'utilisateur est authentifié. Le service de chatbot est opérationnel.
Postcondition	L'utilisateur reçoit une réponse textuelle à sa question.
Scénario nominal	1. L'utilisateur ouvre l'interface du chatbot. 2. Il saisit une question en langage naturel (ex : "Liste les voyages du chauffeur 'Dupont'"). 3. Le frontend envoie la question au backend (OT-Voyage service). 4. Le backend transmet la question à un script Python. 5. Le script, via LangChain et l'API Gemini, traduit la question en requête SQL. 6. La requête est exécutée sur la base de données MySQL. 7. Le résultat est reformaté en une réponse lisible. 8. La réponse est renvoyée au frontend et affichée à l'utilisateur.
Scénarios alternatifs/d'erreur	- Si le chatbot ne comprend pas la question ou si une erreur survient, il répond avec un message d'aide invitant l'utilisateur à reformuler sa question.

9.3.4 Principe de fonctionnement du script Python du chatbot intelligent

Cette section détaille l'architecture et les technologies qui animent le chatbot, lui permettant de traduire le langage naturel en actions concrètes sur la base de données.

9.3.4.1 Flux de traitement d'une question

Le traitement d'une question utilisateur suit un processus orchestré en plusieurs étapes clés :

1. **Configuration initiale** : Le script charge les configurations sécurisées depuis un fichier `‘.env‘`, notamment la clé d'API pour Google Gemini et l'URL de la base de données.
2. **Connexion à la base de données** : Une connexion est établie avec la base de données MySQL via `‘SQLDatabase‘` de LangChain, préparant le terrain pour l'interrogation des données.
3. **Initialisation du LLM** : Le modèle de langage `‘gemini-1.5-flash‘` est instancié via `‘ChatGoogleGenerativeAI‘`. La température est fixée à 0 pour garantir des réponses factuelles et déterministes.
4. **Création de l'agent SQL** : Un agent SQL est créé avec `‘create-sql-agent‘`. Cet agent est le cœur du système : il est capable de recevoir une question en langage naturel, de la convertir en une requête SQL valide, de l'exécuter sur la base de données, et de synthétiser le résultat en une réponse compréhensible.
5. **Exécution et journalisation** : La fonction principale encapsule ce processus, gère les erreurs potentielles et enregistre chaque interaction dans un fichier de log pour le suivi.

9.3.4.2 Composants technologiques clés

Le fonctionnement du chatbot repose sur un ensemble de bibliothèques Python spécialisées :

langchain : Le framework principal qui orchestre les interactions entre le modèle de langage (LLM), la base de données et les autres outils.

langchain-google-genai : Le connecteur spécifique qui intègre le modèle Gemini de Google au sein de l'écosystème LangChain.

google-generativeai : Le SDK officiel de Google pour interagir directement avec l'API de Gemini.

pymysql : Le pilote Python qui assure la communication effective avec la base de données MySQL.

python-dotenv : Un utilitaire essentiel pour la gestion sécurisée des secrets et des configurations.

9.4 Conception

Cette section présente les diagrammes de séquence qui modélisent les interactions entre les différents composants du système pour les principaux cas d'utilisation de ce sprint.

9.4.1 Diagrammes de séquence

9.4.1.1 Diagramme de séquence - Consulter le planning journalier

Le diagramme de séquence suivant illustre les interactions pour la consultation du planning journalier.

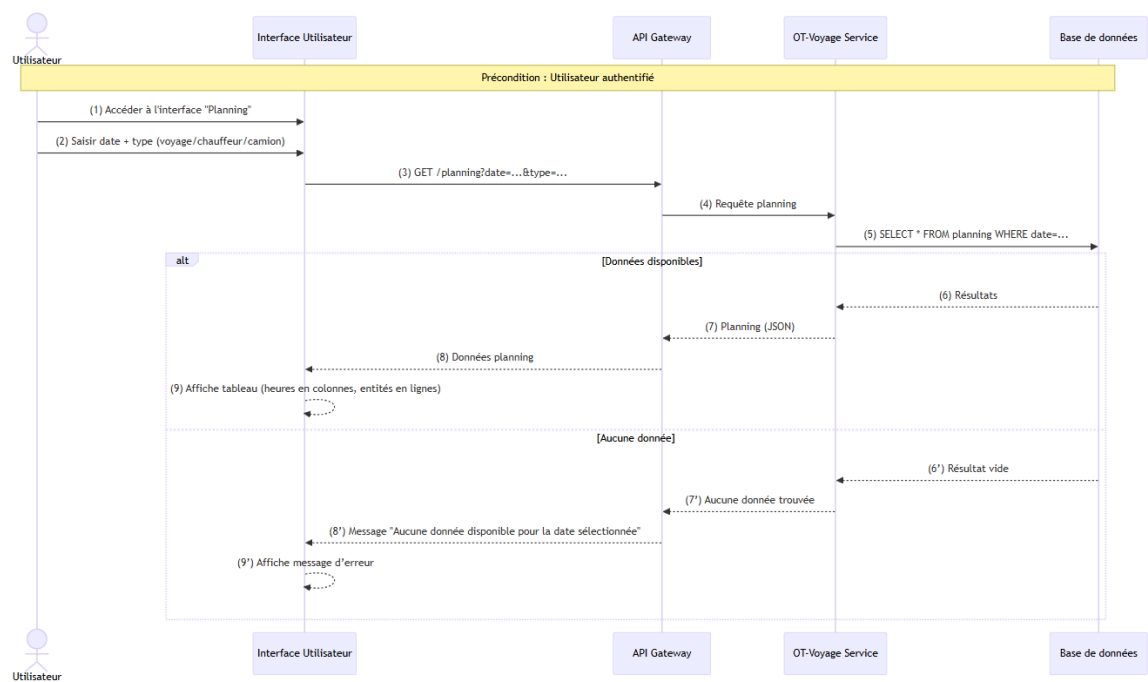


FIGURE 9.2 – Diagramme de séquence - Consulter le planning journalier

9.4.1.2 Diagramme de séquence - Consulter le tableau de bord analytique (KPIs)

Le diagramme ci-dessous détaille la consultation du tableau de bord analytique et de ses indicateurs de performance.

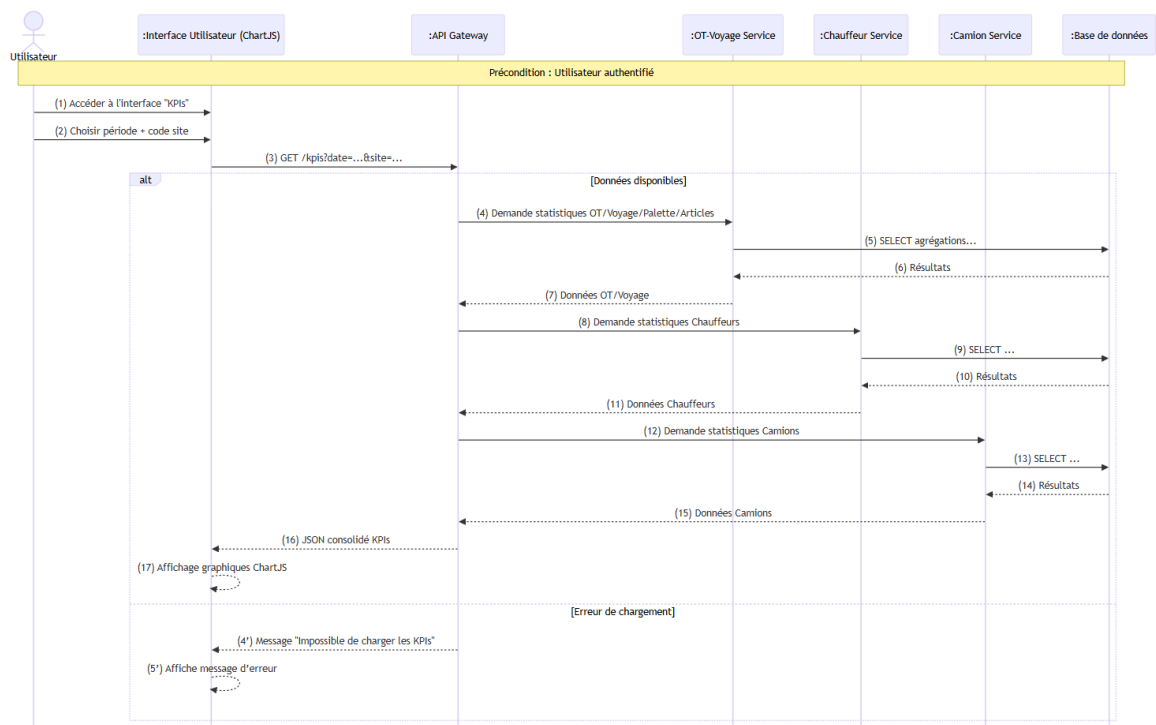


FIGURE 9.3 – Diagramme de séquence - Consulter le tableau de bord (KPIs)

9.4.1.3 Diagramme de séquence - Interagir avec le chatbot intelligent

Enfin, ce diagramme montre le flux complet d'une interaction avec le chatbot intelligent, de la question de l'utilisateur à la réponse générée.

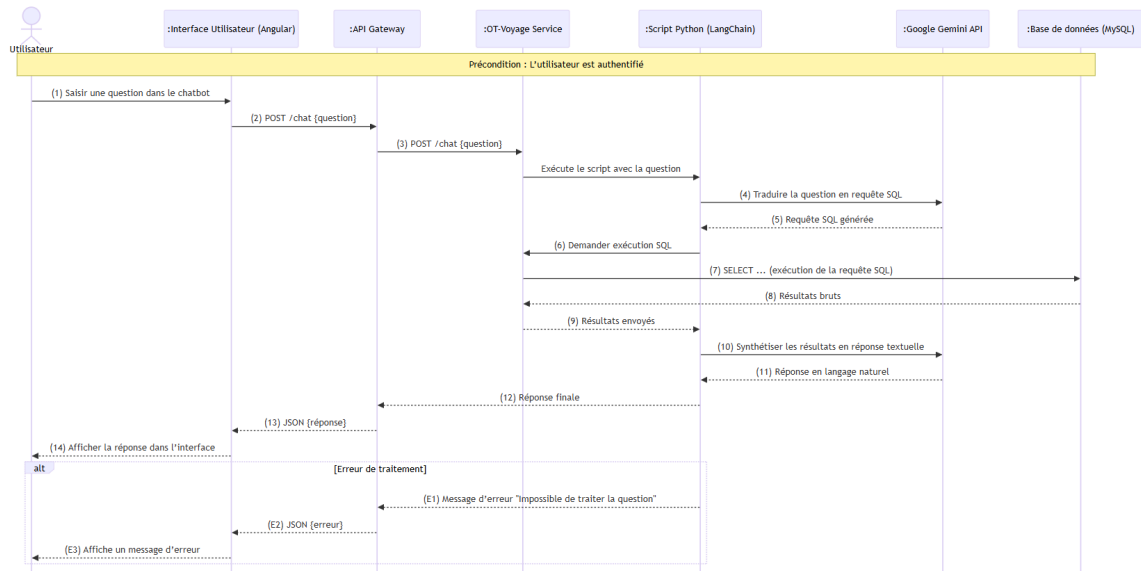


FIGURE 9.4 – Diagramme de séquence - Interagir avec le chatbot

9.5 Réalisation

Cette section présente les interfaces utilisateur finales de l'application LUMTMS, illustrant les fonctionnalités clés développées.

9.5.1 Tableau de Bord et Indicateurs de Performance (KPIs)

Le tableau de bord centralise les indicateurs de performance clés (KPIs) pour offrir une vue analytique sur les opérations, comme le nombre de voyages par statut ou la performance des chauffeurs.

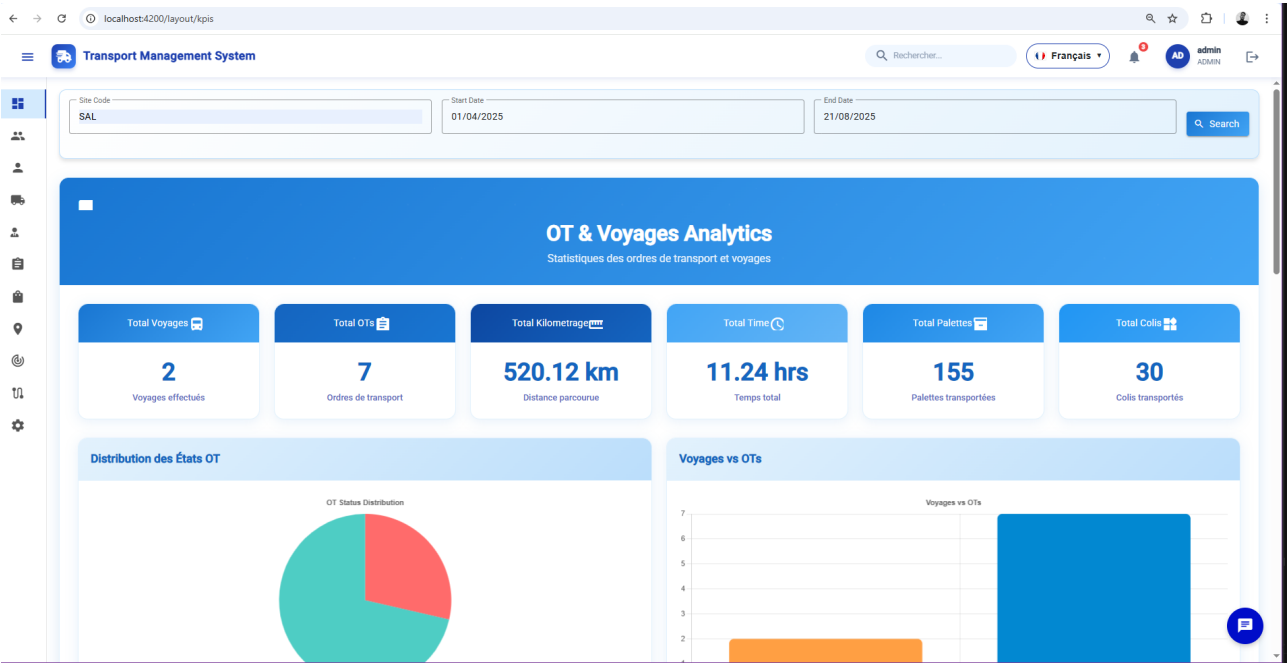


FIGURE 9.5 – Tableau de bord avec les KPIs principaux

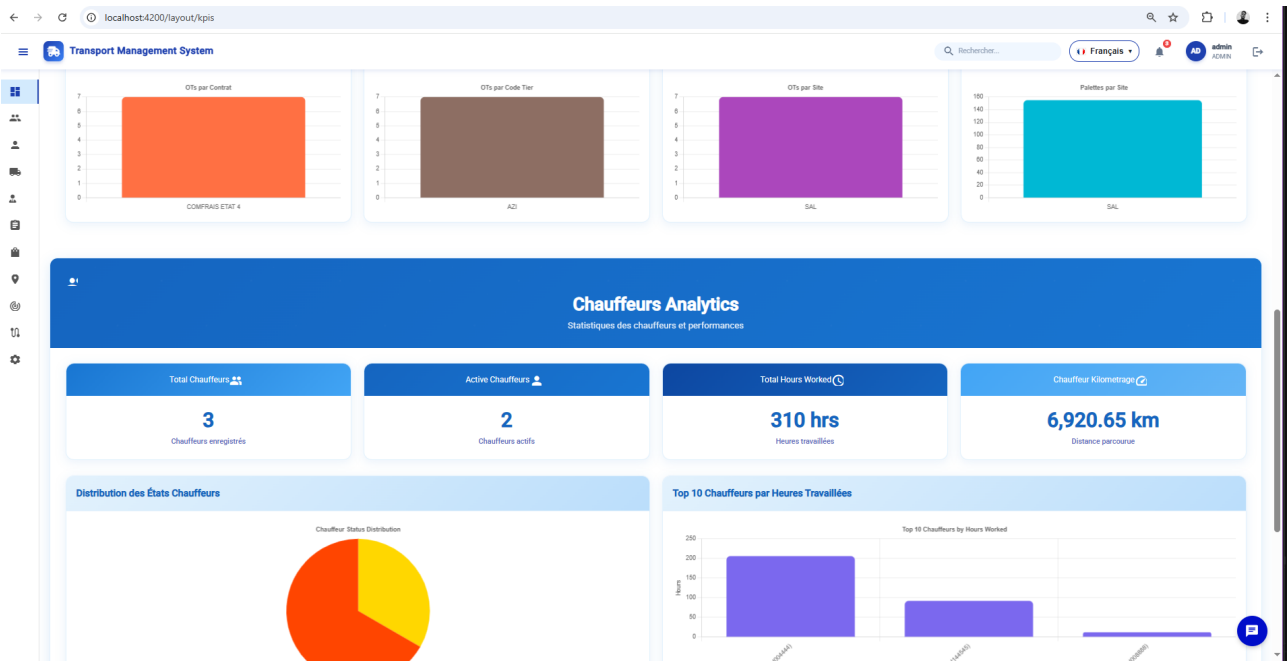


FIGURE 9.6 – Détail des indicateurs de performance

9.5.2 Interface de Consultation du Planning

L'interface de planning offre une vue d'ensemble des voyages planifiés, permettant aux gestionnaires de suivre les affectations et les états en temps réel.

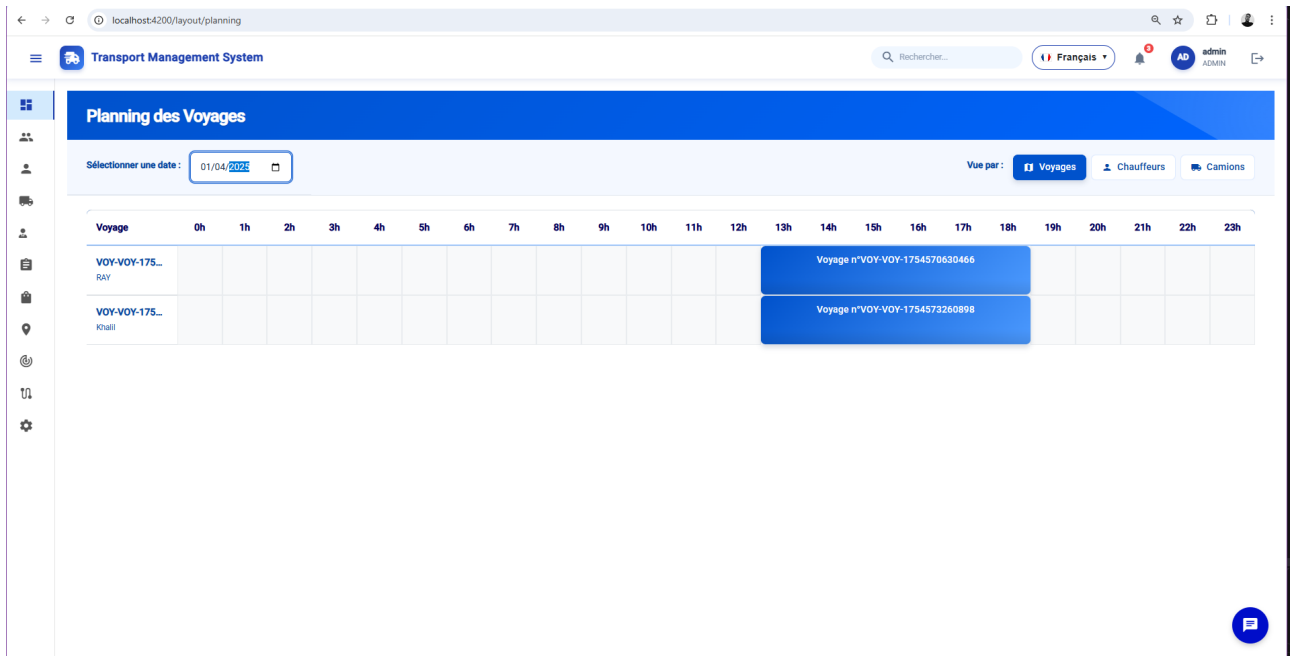


FIGURE 9.7 – Vue du planning des voyages

9.5.3 Interface du Chatbot Intelligent

Le chatbot, intégré à l'application, offre une assistance interactive aux utilisateurs. Il peut répondre aux questions concernant les voyages, les chauffeurs, ou l'état général du système.

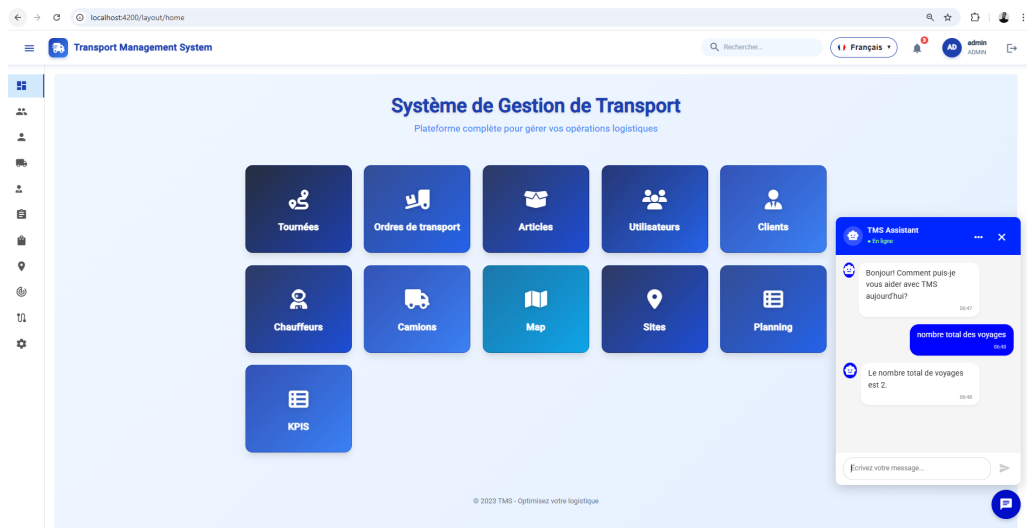


FIGURE 9.8 – Interaction avec le chatbot pour obtenir des informations

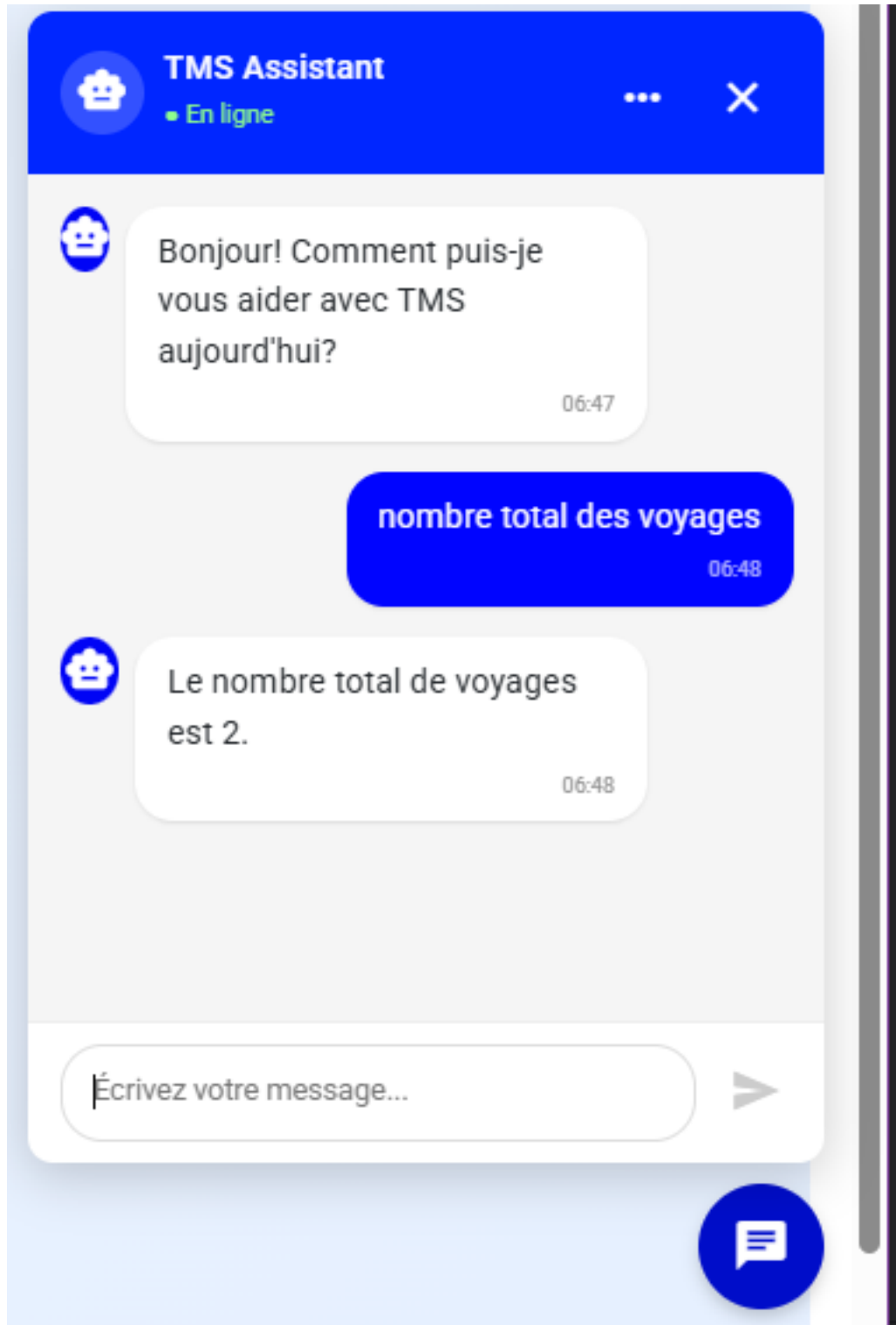


FIGURE 9.9 – Exemple de réponse structurée du chatbot

9.6 Conclusion

Le Sprint 6 a marqué l'aboutissement du projet en introduisant des fonctionnalités avancées qui transforment l'application d'un simple outil de gestion en une véritable plateforme d'aide à la décision. La mise en place du planning journalier, du tableau de bord analytique et du chatbot intelligent a permis de répondre aux objectifs fixés : offrir une meilleure visibilité opérationnelle, fournir des indicateurs de performance clés pour l'analyse stratégique, et simplifier l'accès à l'information. Avec la finalisation de ce sprint, le projet atteint sa pleine maturité, livrant une solution complète, intuitive et à haute valeur ajoutée pour l'utilisateur.

Chapitre 10

Sprint 7 – Déploiement et Automatisation du Système

10.1 Introduction

Ce chapitre décrit la phase finale du projet, axée sur le déploiement et l'automatisation. L'objectif est de mettre en place une infrastructure robuste et un processus de livraison continue pour garantir la stabilité, la scalabilité et la maintenabilité de l'application.

10.2 Objectif du sprint

L'objectif principal de ce sprint est de rendre l'application pleinement opérationnelle en production. Cela inclut la conteneurisation de tous les services, la mise en place d'un déploiement orchestré, et l'automatisation du cycle de vie de développement via une pipeline CI/CD.

10.3 Backlog du sprint

Le tableau 10.1 présente le backlog du Sprint 7 avec les User Stories, les tâches associées et les estimations de durée.

TABLE 10.1: Backlog du Sprint 7

ID Fonct.	Scénario Utilisateur	ID Tâche	Tâches	Estim.
US-26	En tant que développeur, je veux dockeriser l'application pour qu'elle soit portable et facile à déployer	T-US26-1	Analyser les dépendances et choisir une image de base	1 j
		T-US26-2	Créer un fichier Dockerfile	2 j
		T-US26-3	Tester l'image Docker localement	2 j
		T-US26-4	Documenter le processus de dockerisation	1 j
US-27	En tant que développeur, je veux utiliser Docker Compose pour orchestrer les services	T-US27-1	Identifier les services nécessaires	1 j
		T-US27-2	Créer un fichier docker-compose.yml	2 j
		T-US27-3	Tester le déploiement local avec Docker Compose	2 j
		T-US27-4	Documenter la configuration Docker Compose	1 j
US-28	En tant que développeur, je veux déployer le système sur un serveur distant	T-US28-1	Configurer un serveur distant	2 j
		T-US28-2	Installer Docker et Docker Compose sur le serveur	1 j
		T-US28-3	Déployer l'application via Docker Compose sur le serveur	2 j
		T-US28-5	Tester l'accès à l'application déployée	1 j
US-29	En tant que développeur, je veux mettre en place une pipeline CI/CD avec Jenkins	T-US29-1	Installer et configurer Jenkins	2 j

ID Fonct.	Scénario Utilisateur	ID Tâche	Tâches	Estim.
		T-US29-2	Créer un fichier Jenkinsfile	2 j
		T-US29-3	Configurer l'intégration avec un gestionnaire de code	1 j
		T-US29-4	Tester la pipeline (build, test, déploiement)	2 j
		T-US29-5	Documenter la pipeline CI/CD	1 j

10.4 Dockerisation du backend

La dockerisation est une étape cruciale pour standardiser les environnements de déploiement et simplifier la gestion des dépendances de chaque microservice.

10.4.1 Cycle de Vie d'une Application avec Docker

Le cycle de vie d'une application conteneurisée avec Docker suit un processus standardisé qui garantit la portabilité et la reproductibilité, depuis le développement jusqu'à la production[38]. Ce cycle peut être résumé en quatre étapes principales : la construction de l'image (Build), le partage via un registre (Push), le déploiement sur un hôte (Pull), et l'exécution du conteneur (Run).

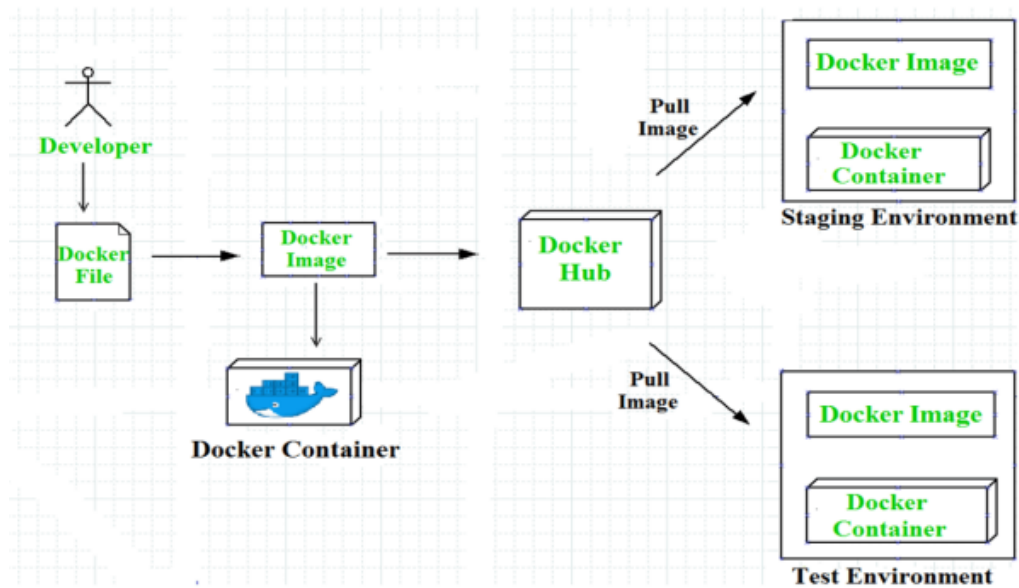


FIGURE 10.1 – Cycle de vie d’une application avec Docker

10.4.2 Configuration des Dockerfiles

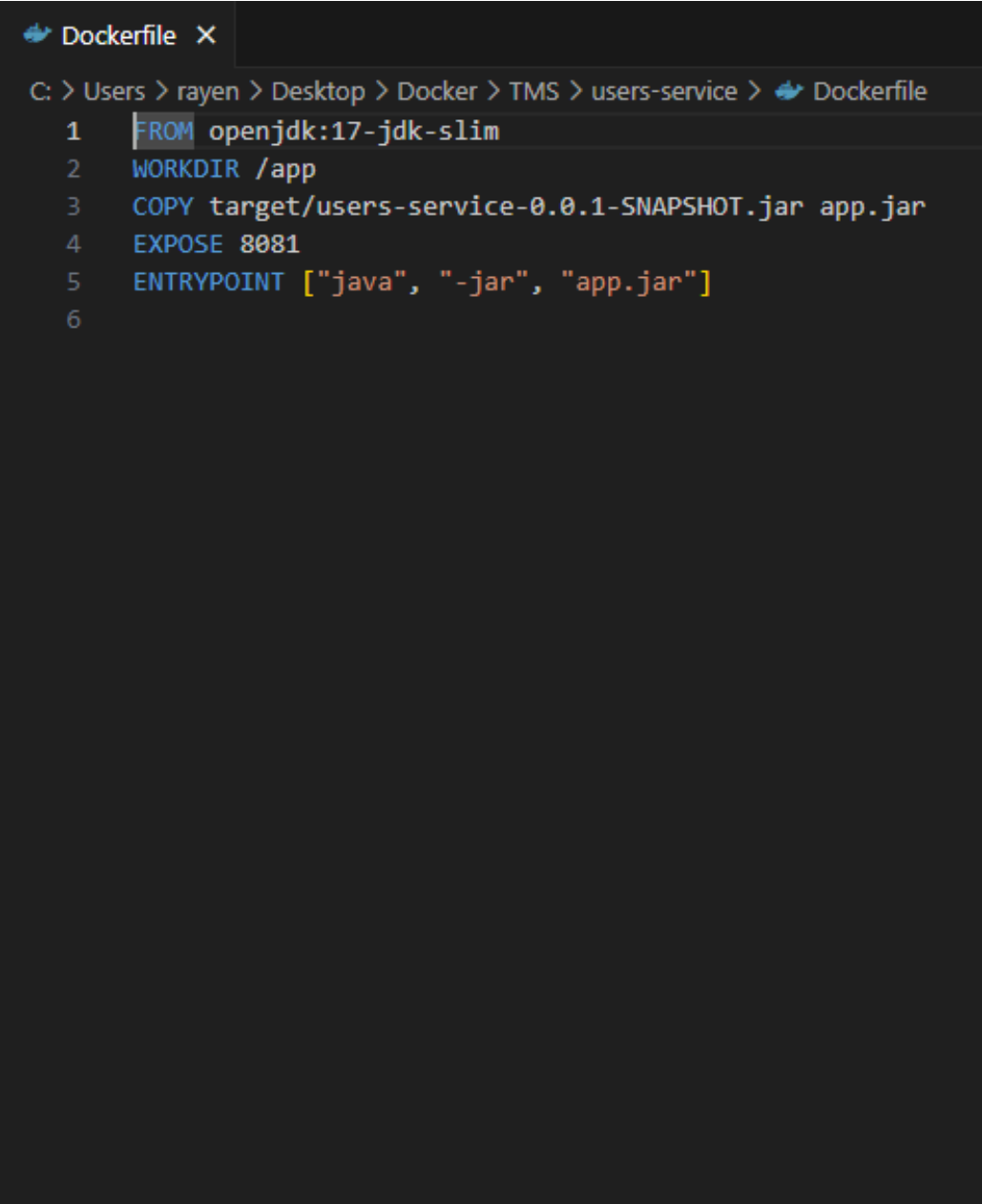
Chaque microservice est doté de son propre Dockerfile pour définir son image de conteneur. Ce fichier texte contient les instructions nécessaires pour assembler l’image, incluant le système d’exploitation de base, les dépendances, le code source et la commande de démarrage.

10.4.2.1 Stratégie de Dockerisation pour Spring Boot

Pour les microservices développés avec Spring Boot, une stratégie de construction multi-étapes (multi-stage build) a été adoptée. Cette approche permet de créer des images Docker légères, sécurisées et optimisées pour la production. Elle consiste à utiliser une première image (l’étape de build) contenant tout le nécessaire pour compiler le projet (JDK, Maven), puis à copier uniquement l’artefact généré (le fichier JAR) dans une seconde image beaucoup plus légère (l’étape finale) contenant uniquement l’environnement d’exécution Java (JRE).

10.4.2.2 Exemple de Dockerfile

Voici un exemple de ‘Dockerfile’ multi-étapes utilisé pour l’un des microservices Spring Boot :



```
Dockerfile X
C: > Users > rayen > Desktop > Docker > TMS > users-service > Dockerfile
1 FROM openjdk:17-jdk-slim
2 WORKDIR /app
3 COPY target/users-service-0.0.1-SNAPSHOT.jar app.jar
4 EXPOSE 8081
5 ENTRYPOINT ["java", "-jar", "app.jar"]
6
```

FIGURE 10.2 – Users-service Dockerfile

10.4.3 Dockerisation des Services Auxiliaires (MySQL)

Pour la base de données, l'image officielle 'mysql' de Docker Hub est utilisée. Sa configuration est gérée de manière externe via des variables d'environnement dans le fichier Docker Compose, ce qui permet de définir le mot de passe root, le nom de la base de données et de monter un volume pour assurer la persistance des données.

```
mysql:
  image: mysql:8.0
  container_name: mysql-container
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: tms_users
    MYSQL_USER: user
    MYSQL_PASSWORD: pass
  ports:
    - "3307:3306" # éviter conflit avec MySQL local
  volumes:
    - mysql-data:/var/lib/mysql
  networks:
    - tms-network
```

FIGURE 10.3 – Configuration du service MYSQL dans le fichier Docker Compose

10.5 Déploiement via Docker Compose

Docker Compose est un outil qui permet de définir et de gérer des applications Docker multi-conteneurs. À l'aide d'un simple fichier de configuration au format YAML ('docker-compose.yml'), il est possible de déclarer tous les services de l'application, leurs dépendances, les réseaux et les volumes nécessaires. Une seule commande ('docker-compose up') suffit ensuite pour créer et démarrer l'ensemble de l'infrastructure.

Voici un exemple simplifié du fichier 'docker-compose.yml' utilisé pour ce projet :

```
docker-compose.yml X
C:\Users\rayen\Desktop> Docker> TMS> docker-compose.yml
1  version: '3.8'
2
3  services:
4    mysql:
5      image: mysql:8.0
6      container_name: mysql-container
7      restart: always
8      environment:
9        MYSQL_ROOT_PASSWORD: root
10       MYSQL_DATABASE: tms_users
11       MYSQL_USER: user
12       MYSQL_PASSWORD: pass
13      ports:
14        - "3307:3306" # éviter conflit avec MySQL local
15      volumes:
16        - mysql-data:/var/lib/mysql
17      networks:
18        - tms-network
19
20    discovery-server:
21      build:
22        context: ./discovery-server
23      container_name: discovery-server
24      ports:
25        - "8761:8761"
26      networks:
27        - tms-network
28
29    users-service:
30      build:
31        context: ./users-service
32      container_name: users-service
33      restart: always
34      depends_on:
35        - mysql
36        - discovery-server
37      ports:
38        - "8081:8081"
39      environment:
40        MYSQL_HOST: mysql
41        SPRING_DATASOURCE_USERNAME: user
42        SPRING_DATASOURCE_PASSWORD: pass
43        EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
44      networks:
45        - tms-network
46
47    gateway-service:
48      build:
```

FIGURE 10.4 – Docker-compose.yml

10.6 Hébergement du système sur un serveur de l'entreprise

L'hébergement de l'application LUMTMS a été conçu pour garantir la robustesse, la cohérence entre les environnements et la simplicité de la maintenance. L'ensemble de l'architecture, incluant le backend et le frontend, est conteneurisé avec Docker.

10.6.1 Architecture de Déploiement

L'architecture retenue repose sur une approche entièrement conteneurisée, orchestrée par Docker Compose. Elle se compose des éléments suivants :

- **Microservices Backend** : Chaque microservice Spring Boot est encapsulé dans sa propre image

Docker, garantissant l'isolation et la scalabilité.

- **Base de données :** Le service de base de données MySQL est également un conteneur, avec un volume persistant pour garantir la durabilité des données.
- **Frontend Angular :** L'application cliente Angular est compilée en fichiers statiques, puis servie par un serveur web Nginx.

10.6.2 Déploiement Unifié avec Docker Compose

Le fichier 'docker-compose.yml' est mis à jour pour inclure le service frontend. Il définit l'ensemble de la pile applicative :

- Le service 'frontend-nginx' qui construit l'image Angular/Nginx.
- Les services backend ('users-service', 'ot-voyage-service', etc.).
- Le service 'db' pour la base de données.
- Un réseau commun ('tms-network') pour permettre la communication entre les conteneurs.

Le service Nginx est le seul à exposer un port (généralement le port 80) au serveur hôte. Les utilisateurs accèdent à l'application via ce point d'entrée unique, et Nginx se charge de distribuer les requêtes en interne. Le déploiement de l'ensemble de l'application se résume alors à une seule commande : 'docker-compose up --build -d'.

10.6.3 Étapes d'hébergement

Le processus de déploiement sur le serveur cible suit une procédure standardisée :

1. **Prérequis :** Installation de Docker et Docker Compose sur la VM serveur.
2. **Récupération du code :** Clonage du dépôt Git contenant le code source des microservices et le fichier 'docker-compose.yml'.
3. **Configuration de l'environnement :** Création d'un fichier '.env' à la racine du projet pour centraliser les variables d'environnement (identifiants de base de données, clés d'API, etc.).
4. **Lancement de l'application :** Exécution de la commande 'docker-compose up -d' pour construire les images (si elles n'existent pas localement) et démarrer tous les conteneurs en arrière-plan.
5. **Vérification :** Utilisation de la commande 'docker ps' pour vérifier que tous les conteneurs sont en cours d'exécution et consultation des logs avec 'docker-compose logs -f' pour s'assurer qu'il n'y a pas d'erreurs au démarrage.

10.7 Mise en place d'une pipeline CI/CD avec Jenkins

Pour automatiser le processus de build, de test et de publication des différents composants de l'application, une pipeline d'intégration et de déploiement continu (CI/CD) a été configurée à l'aide de Jenkins. Cette pipeline est définie dans un fichier 'Jenkinsfile' à la racine du projet, ce qui permet de versionner et de gérer la configuration de l'automatisation comme du code (*Pipeline as Code*).

10.7.1 Objectifs de la pipeline

L'objectif principal est d'automatiser entièrement le cycle de vie de chaque microservice, depuis la modification du code source jusqu'à la publication d'une image Docker prête à être déployée. La pipeline vise à :

- **Garantir la cohérence** : Chaque build suit exactement les mêmes étapes, éliminant les erreurs manuelles.
- **Accélérer les livraisons** : L'automatisation réduit considérablement le temps nécessaire pour préparer une nouvelle version.
- **Centraliser les artefacts** : Toutes les images Docker sont stockées et versionnées dans un registre central (Docker Hub).
- **Faciliter le déploiement** : La pipeline génère un fichier 'docker-compose.prod.yml' à jour, prêt à être utilisé pour le déploiement.

10.7.2 Configuration de l'Environnement Jenkins

Pour mettre en œuvre la pipeline CI/CD, une configuration rigoureuse de l'environnement Jenkins est nécessaire.

10.7.2.1 Installation et Configuration Initiale

Pour simplifier la gestion et éviter les dépendances liées aux conteneurs, Jenkins a été installé directement en local comme logiciel, à partir du package officiel disponible sur le site de Jenkins. Cette approche permet de bénéficier d'une intégration directe avec le système hôte tout en conservant la flexibilité et les fonctionnalités complètes de Jenkins. La configuration initiale, accessible via l'interface web de Jenkins, inclut la création d'un utilisateur administrateur ainsi que le paramétrage des options de base du système.

10.7.2.2 Configuration des Plugins

Afin de répondre aux besoins propres au projet, plusieurs plugins ont été ajoutés et paramétrés pour étendre les capacités de Jenkins.

- **Plugin Docker** : facilite l'intégration avec Docker pour la construction et le déploiement d'images. Plusieurs composants liés à Docker ont été ajoutés, notamment :
 - Docker API (version 3.5.0)
 - Docker Commons
 - Docker Pipeline
 - Docker Plugin
 - Docker Build Step
- **Plugin Git** : assure la connexion avec les dépôts Git, permettant ainsi la récupération du code source.
- **Plugin Pipeline** : offre la possibilité de créer des pipelines CI/CD en utilisant soit la syntaxe déclarative, soit la syntaxe scripted.

10.7.2.3 Gestion des Credentials et Sécurité

La sécurité constitue un élément essentiel dans la configuration de Jenkins. Afin de garantir un accès protégé aux différentes ressources, plusieurs types de **credentials** ont été définis :

- **Credentials GitHub** : utilisés pour s'authentifier auprès des dépôts GitHub, permettant ainsi la récupération du code source et la publication des statuts de build.
- **Credentials Docker Hub** : nécessaires pour l'authentification et l'envoi des images Docker générées vers le registre Docker Hub.

10.7.3 Description des étapes de la pipeline (Jenkinsfile)

Le 'Jenkinsfile' est structuré en plusieurs étapes (*stages*) qui s'exécutent séquentiellement :

1. **Checkout** : La première étape consiste à récupérer la dernière version du code source depuis le dépôt Git.
2. **Setup Docker** : L'environnement de build est préparé. Cette étape vérifie que Docker est fonctionnel et se connecte à Docker Hub en utilisant des identifiants sécurisés stockés dans Jenkins.
3. **Build des Microservices** : Pour chaque microservice de l'architecture (Discovery Server, Users Service, Gateway, etc.), une étape de build dédiée est exécutée. Ce processus est standardisé :

- Le projet Maven est compilé et packagé en un fichier JAR exécutable via la commande `'mvnw.cmd clean package'`.
 - Une image Docker est construite à partir du Dockerfile du microservice. L'image est taguée avec le numéro de build Jenkins (ex : `'rayenmeddeb20/tms-users-service :42'`) pour une traçabilité parfaite, ainsi qu'avec le tag `'latest'`.
4. **Push des Images sur Docker Hub :** Une fois que tous les microservices ont été compilés et conteneurisés avec succès, leurs images Docker (avec les deux tags, versionné et `'latest'`) sont poussées vers le registre Docker Hub. Elles deviennent ainsi accessibles pour le déploiement sur n'importe quel serveur.
 5. **Mise à jour du Docker Compose :** Une étape clé de la pipeline génère dynamiquement un fichier `'docker-compose.prod.yml'`. Ce fichier utilise les variables d'environnement de la pipeline pour référencer les images qui viennent d'être construites et poussées sur Docker Hub. Cela garantit que le fichier de déploiement est toujours synchronisé avec la dernière version validée par la pipeline.
 6. **Cleanup :** Pour maintenir un environnement propre sur l'agent Jenkins, toutes les images Docker créées localement pendant l'exécution de la pipeline sont supprimées. L'agent se déconnecte également de Docker Hub.

Cette pipeline robuste et automatisée constitue la pierre angulaire du processus de livraison continue du projet, assurant des mises à jour rapides, fiables et sécurisées.

L'automatisation réduit les erreurs manuelles, accélère les cycles de livraison et permet aux développeurs de se concentrer sur les fonctionnalités à valeur ajoutée.

10.8 Résultats et Démonstration

Cette section présente les preuves concrètes du succès du déploiement et de l'automatisation mis en place durant ce sprint.

10.8.1 Screenshots des containers en exécution

Une fois le déploiement effectué via la commande `'docker-compose up'`, la commande `'docker ps'` permet de lister tous les conteneurs en cours d'exécution. La capture d'écran ci-dessous montre que tous les services de l'application (microservices, base de données, Jenkins) sont actifs et fonctionnels.

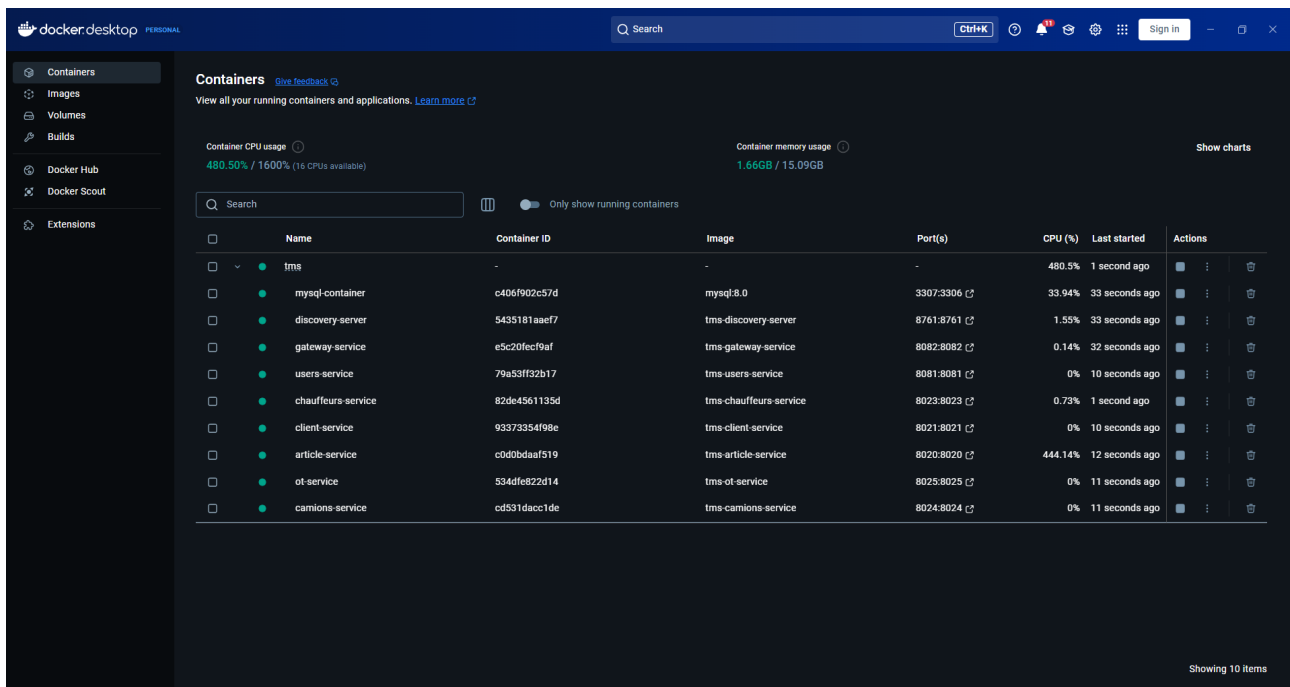


FIGURE 10.5 – Liste des conteneurs Docker en exécution sur le serveur

10.8.2 Exécution de la pipeline CI/CD

La pipeline Jenkins a été configurée pour se déclencher automatiquement à chaque ‘push’ sur la branche principale du dépôt Git. La vue "Blue Ocean" de Jenkins offre une visualisation claire des différentes étapes de la pipeline : Checkout, Build et Push. La figure ci-dessous montre une exécution réussie de la pipeline.

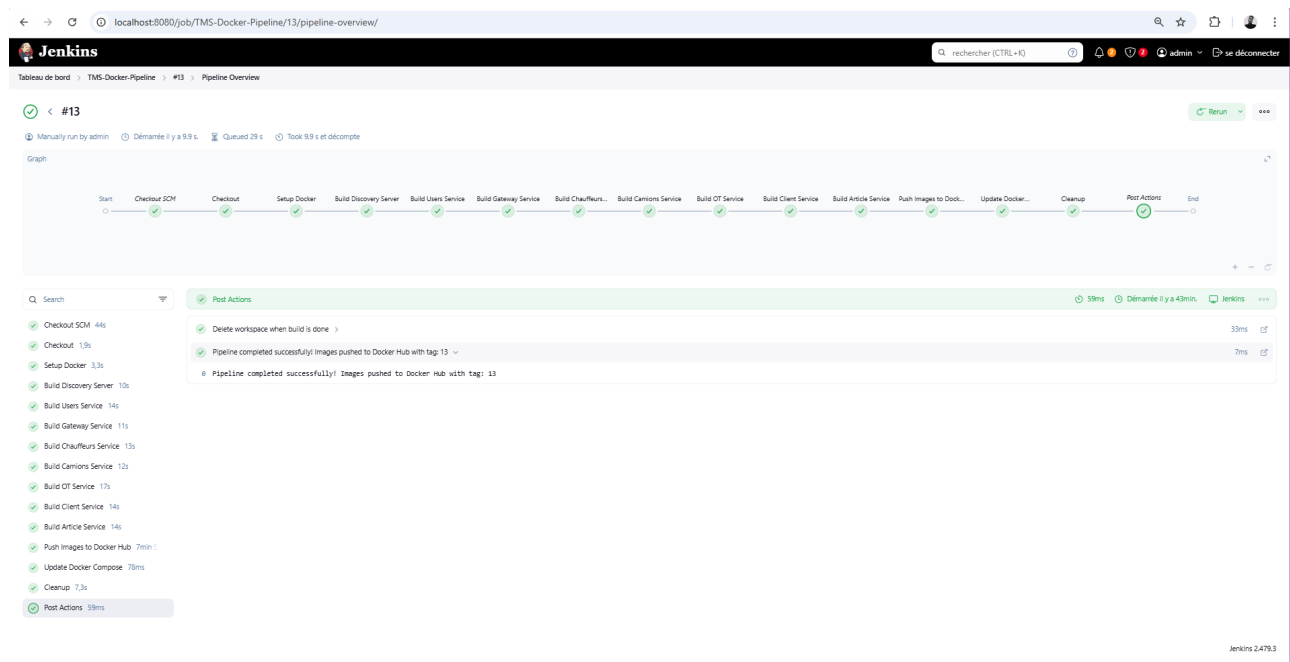


FIGURE 10.6 – Visualisation de la pipeline CI/CD dans Jenkins Blue Ocean

10.9 Conclusion

Ce sprint a permis de mettre en place une infrastructure de déploiement moderne et automatisée, essentielle pour la pérennité du projet. L'application est désormais prête pour une exploitation en production, avec des processus garantissant des mises à jour fiables et rapides.

Conclusion générale

Ce projet de fin d'études a permis de mener à bien la conception, le développement et le déploiement de LUMTMS, une solution de gestion de transport moderne et complète, spécifiquement adaptée aux besoins de Lumière Logistique. En remplaçant des processus manuels, fragmentés et sujets aux erreurs par une plateforme centralisée, automatisée et intelligente, ce projet a atteint son objectif principal : doter l'entreprise d'un outil stratégique pour optimiser ses opérations, améliorer sa qualité de service et renforcer sa compétitivité.

L'architecture microservices, combinée à des technologies robustes comme Spring Boot et Angular, et déployée via une infrastructure conteneurisée avec Docker et une pipeline CI/CD avec Jenkins, garantit non seulement la performance et la fiabilité de la solution actuelle, mais aussi sa maintenabilité et son évolutivité future. L'intégration de fonctionnalités avancées telles que le tableau de bord analytique et le chatbot intelligent positionne LUMTMS comme une solution à forte valeur ajoutée, allant au-delà de la simple gestion opérationnelle pour devenir un véritable outil d'aide à la décision.

Le système livré est aujourd'hui une base solide et fonctionnelle. Cependant, le domaine de la logistique étant en perpétuelle évolution, plusieurs pistes d'amélioration peuvent être envisagées pour enrichir davantage la solution.

Les perspectives d'évolution de LUMTMS s'orientent vers l'intégration de l'Intelligence Artificielle afin de rendre le système plus proactif et prédictif. Cela permettrait d'optimiser les tournées en tenant compte des conditions réelles, d'anticiper la maintenance des véhicules grâce à des modèles prédictifs, et d'offrir une estimation dynamique et fiable des heures d'arrivée (ETA). Par ailleurs, le développement d'une application mobile dédiée constitue une version chauffeurs pour la gestion des missions, la remontée d'informations et la navigation GPS, ainsi qu'une version clients pour le suivi en temps réel, la réception de notifications et l'amélioration de l'expérience utilisateur.

Bibliographie

- [1] The Importance of Supply Chain Management in Logistics. (2023). <https://www.apu.apus.edu/area-of-study/business-and-management/resources/the-importance-of-supply-chain-management-in-logistics/>
- [2] Statista. (2024). Size of the freight forwarding market worldwide. <https://www.statista.com/statistics/1066803/freight-forwarding-market-size-worldwide/>
- [3] Monk, E., Wagner, B. (2009). *Concepts in Enterprise Resource Planning*. 3rd ed. Cengage Learning.
- [4] SAP. What is a transportation management system (TMS)? <https://www.sap.com/products/scm/transportation-logistics/what-is-a-tms.html>
- [5] Oracle. What Is a Transportation Management System? <https://www.oracle.com/scm/logistics/transportation-management/what-is-transportation-management-system/>
- [6] Shipy. 5 Key Challenges in Transportation Management System. <https://shipy.io/blogs/5-key-challenges-in-transportation-management-system/>
- [7] Gartner. (2024). Magic Quadrant for Transportation Management Systems. <https://www.gartner.com/en/documents/5305563>
- [8] CTSI-Global. TMS Platforms Are Evolving : The Top Trends for 2025 and Beyond. <https://ctsi-global.com/2024/tms-platforms-are-evolving-the-top-trends-for-2025-and-beyond/>
- [9] G2. SAP Transportation Management Pros and Cons. <https://www.g2.com/products/sap-transportation-management/reviews?page=3&q=pros-and-cons>
- [10] G2. Oracle Transportation Management Cloud Pros and Cons. <https://www.g2.com/products/oracle-transportation-management-cloud/reviews?q=pros-and-cons>
- [11] SelectHub. Manhattan TMS Reviews 2025 : Pricing, Features & More. <https://www.selecthub.com/p/tms-software/manhattan-tms/>
- [12] Beck, K., et al. (2001). *Manifesto for Agile Software Development*. Disponible à : <https://agilemanifesto.org>.

- [13] AKANEA. (2024). *AKANEA, éditeur Supply-Chain*. Disponible à : <https://www.faq-logistique.com/Editeur-AKANEA.htm>.
- [14] Schwaber, K., Sutherland, J. (2020). *The Scrum Guide*. Disponible à : <https://scrumguides.org>.
- [15] Booch, G., Rumbaugh, J., Jacobson, I. (2005). *The Unified Modeling Language User Guide*. 2nd ed. Addison-Wesley.
- [16] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- [17] Newman, S. (2021). *Building Microservices : Designing Fine-Grained Systems*. 2nd ed. O'Reilly Media.
- [18] Angular. (2024). *Angular Documentation*. Disponible à : <https://angular.dev>.
- [19] Pivotal Software. (2024). *Spring Boot Documentation*. Disponible à : <https://spring.io/projects/spring-boot>.
- [20] Oracle. (2024). *MySQL Workbench Documentation*. Disponible à : <https://dev.mysql.com/doc/workbench/en/>.
- [21] OpenRouteService. (2024). *OpenRouteService API Documentation*. Disponible à : <https://openrouteservice.org>.
- [22] OpenStreetMap Foundation. (2024). *OpenStreetMap Documentation*. Disponible à : <https://www.openstreetmap.org>.
- [23] LangChain. (2024). *LangChain Documentation*. Disponible à : <https://python.langchain.com>.
- [24] Google. (2024). *Gemini API Documentation*. Disponible à : <https://ai.google.dev>.
- [25] Chart.js. (2024). *Chart.js Documentation*. Disponible à : <https://www.chartjs.org>.
- [26] TIBCO Software. (2024). *JasperReports Documentation*. Disponible à : <https://community.jaspersoft.com/project/jasperreports-library>.
- [27] JetBrains. (2024). *IntelliJ IDEA Documentation*. Disponible à : <https://www.jetbrains.com/idea/documentation>.
- [28] Microsoft. (2024). *Visual Studio Code Documentation*. Disponible à : <https://code.visualstudio.com/docs>.
- [29] Postman. (2024). *Postman Documentation*. Disponible à : <https://learning.postman.com/docs>.
- [30] GitHub. (2024). *GitHub Documentation*. Disponible à : <https://docs.github.com>.
- [31] diagrams.net. (2024). *draw.io Documentation*. Disponible à : <https://www.drawio.com/doc>.
- [32] Lamport, L. (1994). *LaTeX : A Document Preparation System*. 2nd ed. Addison-Wesley.
- [33] Oracle. (2024). *Java Documentation*. Disponible à : <https://docs.oracle.com/en/java>.
- [34] Microsoft. (2024). *TypeScript Documentation*. Disponible à : <https://www.typescriptlang.org/docs>.

- [35] W3C. (2024). *HTML Living Standard*. Disponible à : <https://html.spec.whatwg.org>.
- [36] W3C. (2024). *CSS Snapshot*. Disponible à : <https://www.w3.org/TR/CSS>.
- [37] Python Software Foundation. (2024). *Python Documentation*. Disponible à : <https://docs.python.org>.
- [38] Docker Inc. (2024). *Docker Documentation*. Disponible à : <https://docs.docker.com>.
- [39] Jenkins. (2024). *Jenkins Documentation*. Disponible à : <https://www.jenkins.io/doc>.
- [40] NGINX. (2024). *NGINX Documentation*. Disponible à : <https://nginx.org/en/docs>.

Résumé

Ce rapport présente le développement d'un Système de Gestion du Transport (TMS) moderne, LUMTMS, pour l'entreprise Lumière Logistique. Le projet s'appuie sur une analyse approfondie et une conception UML rigoureuse afin de répondre aux besoins métier et contraintes opérationnelles. Il repose sur une architecture microservices modulaire, scalable et maintenable, chaque service disposant de sa propre base de données. LUMTMS intègre des API externes et des algorithmes d'optimisation pour le calcul dynamique des itinéraires et l'affectation efficace des ressources, réduisant coûts et trajets à vide. Des tableaux de bord interactifs fournissent des indicateurs clés pour le suivi et la prise de décision. Un chatbot basé sur l'IA facilite l'accès aux informations en langage naturel. Enfin, le système est conteneurisé avec Docker et déployé via une pipeline CI/CD avec Jenkins, garantissant portabilité, automatisation et pratiques DevOps modernes. Ce projet offre ainsi une bonne solution logistique.

Abstract

This report presents the development of a modern Transport Management System (TMS), LUMTMS, for the company Lumière Logistique. The project is based on a thorough analysis and a rigorous UML design to meet business needs and operational constraints. It relies on a modular, scalable, and maintainable microservices architecture, with each service having its own database. LUMTMS integrates external APIs and optimization algorithms for dynamic route calculation and efficient resource allocation, reducing costs and empty trips. Interactive dashboards provide key indicators for monitoring and decision-making. An AI-based chatbot facilitates access to information in natural language. Finally, the system is containerized with Docker and deployed through a CI/CD pipeline with Jenkins, ensuring portability, automation, and modern DevOps practices. This project thus offers an effective and innovative logistics solution.